

This section covers the basics of software for FTC®.

10.1 Getting Started

Basic information for starting out on your programming journey.

Check out the official FTC SDK Example Programs here!¹⁵⁹

10.1.1 Fundamental Concepts of Programming

For almost any programming language, whether it's Java, Python, or Blocks, there are concepts in coding that transfer across languages. These ideas are foundational when learning to program and should be applicable in FTC® and beyond.

This section is primarily for people with limited Java experience. However, even if you are more experienced, it may still be helpful to skim through the section, as you might find concepts that have not yet been introduced to you.

Examples will mostly be in Java, where `//` indicates a comment which the program ignores and is used for people to read.

```
int number; // Declaring that number will contain an integer.  
number = 5; // Setting a value so that the variable holds something.  
  
int secondNumber = 6; // Doing both above.  
  
int total = number + secondNumber; // Math.  
System.out.println(total); // Printing, it will show up as 11.
```

¹⁵⁹ <https://github.com/FIRST-Tech-Challenge/FtcRobotController/tree/master/FtcRobotController/src/main/java/org/fIRSTinspires/ftc/robotcontroller/external/samples>

Java-Specific Exploratory Questions

- If I didn't set a value for number and then I printed it, what would it print?
- What other operations can I do with number and secondNumber?
- Can I set a decimal to number? If not, what happens?
- What is `System.out.println()` ;?
- Delete one character in the code. Remember the error (if any), and then undo it. Delete another part. How many different errors can you get?

There are different types of variables

- Numbers (Integers, Floats, Doubles)
- Strings (Text) or characters
- And a lot more depending on the language (Ex: Arrays)
- They help tell the program know the basis of what it should do with a variable.

```
String coolName = "Gluten Free";
String restOfSentence = " is epic.";

// Prints out the sentence by combining the strings, unlike adding if they // were
// integers
System.out.println(coolName + restOfSentence);

// Fun fact: Using + to add strings is called String Concatenation
```

Java-Specific Exploratory Questions

- Replace the text in `coolName` to something else. Your name, a phone number, your favorite anime. What about emotes and copypastas? What about characters in other languages?
- Try adding a number and a string, what happens?
- Is it possible to add multiple strings and numbers together?

Important Control Structures

Be sure to familiarize yourself with basic control structures (if/else statements, for loops, while loops, and for-each loops). These control structures are by far the most commonly encountered, and thus, familiarizing yourself with these principles is extremely important (not just for FTC, but programming in general). However, there are a few control structures that are far less common that are extremely useful in FTC; namely *Finite State Machines* (page 344).

Data Structures (Arrays)

Data structures are a method of organizing and storing large amounts of data. There are a lot of different types of data structures that mostly differ in the relationships between data points, and we would recommend that you read into them. We will only go over a few here.

Arrays¹⁶⁰

Arrays are the most basic and simple data structure. When an array is initialized, its size must be set, and it cannot be changed.

If you wish to expand an array, a new one must be created and all of the old data copied over. Elements of an array are stored adjacent to each other in memory, so when they are accessed the number you want to access times the amount of bits in the object in the array is added to the starting address, and data is accessed from there.

This means that arrays are incredibly efficient at reading data in a nonlinear order.

ArrayList¹⁶¹

Object Oriented Programming in Java¹⁶²

10.1.2 Options for Programming

There are three recommended options for teams to use when programming a robot to do various tasks: Blocks, OnBot Java, and Android Studio.

Blocks

The Blocks Programming Tool is a simple way to code simple actions for the robot. It has a colorful and lego-like design and includes sounds of block pieces of snapping together.

Using a device and a robot controller phone, connect your device to the phone's Wi-Fi by clicking the three dots in the top right of the [RC](#) and select "Program & Manage". This will display the WiFi SSID and password for you to connect to. Go onto your browser (preferably Chrome, though other modern browser will likely work) and enter the address shown on the [RC](#).

Advantages

- Good for beginners: one of the easiest and simplest options to use.
- Easy to set up: requires a device (laptop, chromebook, tablet) and a phone in the Program & Manage screen
- Programs can be saved directly to the phone
 - Changes can be made quickly
- Doesn't require an external Wi-Fi connection

¹⁶⁰ <https://www.geeksforgeeks.org/arrays-in-java/>

¹⁶¹ <https://www.geeksforgeeks.org/arraylist-in-java/>

¹⁶² <https://www.geeksforgeeks.org/classes-objects-java/>

Disadvantages

- Not recommended if you're already familiar with some type of programming
 - Primarily a teaching tool for people with no programming experience
- This will change your primary Wi-Fi network to the *Robot Controller's* Direct Wi-Fi network, and as such, you will be unable to access the internet while connected to this particular network.
 - Teams can get around this by purchasing an external Wi-Fi module that allows computers to run 2 Wi-Fi networks at once.
- Sacrifices flexibility and application for simplicity
- Don't ever use it on a phone, unless you're out of options. (Not phone-friendly)
 - You may make a bigger mess if you try to.

OnBot Java

OnBot Java uses a similar method of using a browser to code and save directly to the phone. The difference being that a programming language called Java is used instead.

Advantages

- Recommended if you're learning or have learned some programming, even better if you know a little Java.
- Greater flexibility than Blocks
- There are a lot more resources available in case you need help
- More applicable to the real-world than Blocks
- Maintains most of the advantages of Blocks

Disadvantages

- Connecting to the robot controller's Wi-Fi network will prevent you from using anything on the internet including video tutorials and online communication
- Using external libraries is difficult and borderline impossible
- Steeper learning curve than Blocks

Android Studio

Android Studio is a comprehensive Integrated Development Environment (IDE) that uses Java to program the phones. Instead of using a browser to upload code, Android Studio will compile your Robot Controller code into a .apk file (an app installer), and install that on the phone.

Advantages

- Recommended if you're learning or have learned some programming, even better if you know a little Java.
- Much greater flexibility than Blocks.
- Much easier to integrate libraries like [EasyOpenCV¹⁶³](#), [FTC Dashboard¹⁶⁴](#), [FTCLib¹⁶⁵](#), and [Road Runner¹⁶⁶](#).
- Can use plugins like [Road Runner¹⁶⁷](#).
- Can use either a USB connection to the [RC](#) phone, or a wireless connection to upload code.

Note

Deploy times can be sped up by using [OpenRC Turbo¹⁶⁸](#).

- Can debug in real-time
- Many resources for Java, Android Studio, and IDEA
- Can use other programming languages

Disadvantages

- Connecting to the robot controller's Wi-Fi network will prevent you from using anything on the internet including video tutorials and online communication, unless you have a second Wifi adapter (cheap and easy)
- Relatively easy setup process, but time consuming and is a hefty install (3GB of files between Android Studio, ftc_app, and other libraries)
- Issues can be difficult to diagnose and solve

Other Programming Languages

Kotlin

Kotlin is a relatively new and rapidly growing programming language from JetBrains, the creator of the IntelliJ IDE, which Android Studio is based off. Kotlin was made to be completely compatible with Java but be easier to work with. Google recently announced it as an official Android language, then announced they are going "Kotlin first" but still keeping Java support.

¹⁶³ <https://github.com/openftc/easyopencv>

¹⁶⁴ <https://github.com/acmerobotics/ftc-dashboard>

¹⁶⁵ <https://github.com/ftclib/ftclib>

¹⁶⁶ <https://github.com/acmerobotics/road-runner>

¹⁶⁷ <https://github.com/acmerobotics/road-runner>

¹⁶⁸ <https://github.com/OpenFTC/OpenRC-Turbo>

Advantages

- Concise, readable, easy to edit code
 - Easy to write as it takes much less code to do the same thing
- Both optional type inference and a stronger type system than Java
- Null safety
- Thread safety
- Functional programming
- Seamless integration with Java code and libraries
- Very easy to transition from Java

Disadvantages

- Not widely used in FTC® yet
- New and has fewer community resources for training
- Not recommended for programmers who need large amounts of help from other teams

C and C++

C and C++ are native programming languages compatible with Android. Very few teams have used C++. This is typically used for only part of the code, with the majority being Java or Kotlin.

Advantages

- Fast execution for extremely resource-intensive applications.
- Supports more libraries

Disadvantages

- Rarely needed
- Very difficult to set up
- Difficult to debug code
- Very few teams can help you
- Very few online resources

10.1.3 LinearOpMode vs OpMode

There are two OpMode classes within the FTC® SDK: OpMode and LinearOpMode. The one you use affects how you write the program. For examples of how to use OpMode and LinearOpMode, [refer to the example OpModes in the SDK](#)¹⁶⁹.

LinearOpMode Methods

- `runOpMode()`: Code inside this method will run exactly once after you press the INIT button. This is where you should put all code for the OpMode.
- `waitForStart()`: This method pauses the Op-Mode until you press the START button on the driver station.
- `isStarted()`: returns `true` if the START button has been pressed, otherwise it returns `false`.
- `isStopRequested()`: returns `true` if the STOP button has been pressed, otherwise it returns `false`.
- `idle()`: calls `Thread.yield`, allowing other threads at the same priority level to run.
- `opModeIsActive()`: returns `isStarted() && !isStopRequested()` and calls `idle()`.
- `opModeInInit()`: returns `!isStarted() && !isStopRequested()` and does not call `idle()`.

OpMode Methods

- `init()`: Code inside this method will run exactly once after you press the INIT button on the driver station.
- `init_loop()`: Once the code in `init()` has been run, code inside this method will run continuously until the START button is pressed on the driver station.
- `start()`: Code inside this method will run exactly once after you press the START button on the driver station.
- `loop()`: Once the code in `start()` has been run, code inside this method will run continuously until the STOP button is pressed on the driver station.
- `stop()`: Code inside this method will run exactly once after you press the STOP button on the driver station.

Note

As of SDK version 8.1, when executing OpModes there is a negligible delay of one millisecond between calls of `loop()`. Previously, it had unpredictable delays, however since 8.1 it is similarly performant to LinearOpMode.

¹⁶⁹ <https://github.com/FIRST-Tech-Challenge/FtcRobotController/tree/master/FtcRobotController/src/main/java/org/fIRSTinspires/ftc/robotcontroller/external/samples>

Conclusion

Overall, the use of `LinearOpMode` or `OpMode` is up to preference. Game Manual 0 uses `LinearOpMode` everywhere for consistency.

10.1.4 Reading and Writing to Hardware

When using the FTC® SDK, there are a variety of built in hardware classes which can be used to communicate with hardware on the robot such as DC Motors, [Servos](#), and Sensors.

Creating and Instantiating Hardware Objects

The first thing required to properly create an object is to import its class. In Android Studio, if the class is referenced without being imported `Alt+Enter` can be pressed to automatically import it. After it is imported, the next step is to create the object:

```
private DcMotor liftMotor;
```

After the object is created, it must be instantiated. Part of the `OpMode` superclass is something called `hardwareMap`. `hardwareMap` is used in the FTC SDK to instantiate objects rather than calling a constructor.

It contains all of the information entered into the configuration on the Robot Controller, such as names of hardware and what port it is plugged into. Here is an example of instantiating the motor we created above:

```
liftMotor = hardwareMap.get(DcMotor.class, "Lift Motor");
```

Whatever sensor you are using, you will pass that class into the spot where `DcMotor.class` is. For example, if `liftMotor` was a `Servo`, `Servo.class` would be passed instead.

For the second argument, you pass whatever the device is named in the Robot Controller configuration. `hardwareMap` will then go find what port the device with that name is plugged into, which allows the hardware to be accessed.

Examples of Using Common Hardware Components

DC Motor

```
DcMotor leftMotor = hardwareMap.get(DcMotor.class, "Left Motor");  
DcMotor rightMotor = hardwareMap.get(DcMotor.class, "Right Motor");  
  
DcMotor elevatorMotor = hardware.get(DcMotor.class, "Elevator Motor");  
DcMotor intakeMotor = hardware.get(DcMotor.class, "Intake Motor");
```

After a `DcMotor` is instantiated, there are a few variables you can set to affect how the DC Motor runs. The first of these is `direction`:

```
leftMotor.setDirection(DcMotor.Direction.REVERSE);  
rightMotor.setDirection(DcMotor.Direction.FORWARD);
```


Changing the direction of the motor does exactly what should be expected, it changes the direction. If a power of 1 is applied to the motor while it is in forward mode, it will turn one direction. If it is in reverse, a power of 1 will spin it in the other direction. If you face the shaft of the motor towards you, forward is counterclockwise (with the exception of NeveRest motors).

Next, there are two zero power behaviors that can be adjusted:

```
leftMotor.setZeroPowerBehavior(DcMotor.ZeroPowerBehavior.BRAKE);
rightMotor.setZeroPowerBehavior(DcMotor.ZeroPowerBehavior.FLOAT);
```

Changing this variable affects how the DC Motor behaves while a power of 0 is applied. BRAKE will cause the motor to try and slow itself down if it is moving (it will NOT cause the motor to hold its position if not already moving), while FLOAT causes the motor to glide to a stop, letting friction do all the work.

Finally, there are four different run modes that can be used with DC motors:

```
leftMotor.setMode(DcMotor.RunMode.RUN_WITHOUT_ENCODER);
rightMotor.setMode(DcMotor.RunMode.RUN_USING_ENCODER);

elevatorMotor.setMode(DcMotor.RunMode.RUN_TO_POSITION);
intakeMotor.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
```

It is important to note that encoder values can be read in any of these modes provided an encoder is properly plugged in. These modes just change how the motor reacts to these encoder values. [The REV Robotics documentation has an explanation of all four run modes](#)¹⁷⁰.

Warning

RUN_TO_POSITION can be a convenient way to control a single-motor mechanism, as it offloads all control work; however, since every motor is dealt with independently, it is inadvisable to use this on mechanisms with multiple motors, especially drivetrains.

Encoders

Term

Encoder

An encoder refers to a device that tracks (generally) rotational movement around an axis.

There are both absolute and relative encoders. An absolute encoder will report at exactly what angle the shaft is compared to its absolute “zero”. A relative encoder will report how far the *shaft* has rotated since it started tracking (for example, when autonomous starts). Relative encoders will have a quadrature output, whereas absolute encoders generally have analog or i2c outputs.

Encoders are used to help find the position of where the robot, or one of its mechanisms, is.

While all FTC legal motors contain built in relative quadrature encoders, they must be wired separately and are not required for use. External encoders may be used and plugged into an encoder port so long as they use the quadrature communication protocol.

Accessing encoders requires calling one method on the DcMotor object, `getCurrentPosition()`, which returns the current position of the encoder plugged into the port. This number may be arbitrary at the begin-

¹⁷⁰ <https://docs.revrobotics.com/duo-control/programming/using-encoder-feedback#choosing-a-motor-mode>

ning of an opmode, and is not reset to 0 unless STOP_AND_RESET_ENCODERS is used or power is cycled to the expansion hub.

Important

There is no real standardized terminology when dealing with quadrature encoders. The SDK uses “CPR” or Counts Per Revolution by default,. You may also see some datasheets list “PPR” or Pulses per Revolution. One pulse can be equivalent to anywhere from 1 to 4 SDK “counts”. Be careful when reading datasheets!

Warning

Encoders with high numbers of Counts per Revolution, such as the REV Through Bore Encoder, can lose steps if plugged into ports 1 or 2. In addition, calls to `getVelocity()` on a `DcMotorEx` object may overflow with high counts per revolution encoders, due to the returned number only being a 16 bit signed integer.

Servo

```
Servo relicServo = hardwareMap.get(Servo.class, "Release Servo");
```

After instantiating a Servo, there are two main functions that can be called: `setPosition()` and `getPosition()`.

```
releaseServo.setPosition(0.75);  
telemetry.addData("Release Servo Target", releaseServo.getPosition());
```

`setPosition()` sets the position of the *servo*. The SDK will use a built-in control loop with the *servo's* potentiometer to drive the *servo* to that position and hold that position. `setPosition()` takes in a double between 0 and 1, where 0 is the *servo's* lower limit of rotation and 1 is the *servo's* upper limit of rotation. Everything between is directly proportional, so 0.5 is the middle, 0.75 is 3/4 the way up, etc.

`getPosition()` does not return the *servo's* current position, rather its current target position. If a variable for the *servo's* current target position is stored properly, this function should never be needed.

Continuous Rotation Servo

```
CRServo intakeServo = hardwareMap.get(CRServo.class, "Intake Servo");
```

A `CRServo` has one main method; `setPower()`. This works very similarly to `DcMotor` 's `setPower()`, meaning that passing it 0 makes it stop, passing it 1 makes it go forward at full speed, passing it -1 makes it go backwards at full speed, and everything in between.

```
intakeServo.setPower(0.75);
```

Digital IO

```
DigitalChannel digitalDevice = hardwareMap.get(DigitalChannel.class, "digital device
↔");
```

A DigitalChannel has a couple main methods. `setMode()` is used to set the port as either an OUTPUT or INPUT port, `getState()` returns the current state of the port (only works in INPUT mode), and `setState()` sets the state of the port (only works in OUTPUT mode)

Tip

Digital ports start by default in INPUT mode

Danger

Digital ports are pulled UP to prevent floating. This means that there is a resistor between the port and 3.3V so the port reads HIGH by default when nothing is connected. As a result digital devices **MUST** connect the digital pin to ground when closed, then leave it unconnected when open. For limit switches, this means connecting one lead to ground and the other to the digital port. **Connecting this wrong (connecting 3.3V to the digital port) may cause instability and can cause your expansion hub to crash**

Analog Input

```
AnalogInput analogInput = hardwareMap.get(AnalogInput.class, "analog input");
```

An AnalogInput has one main method: `getVoltage()` which is used to get the current input voltage to the port.

Note

Although `getMaxVoltage()` returns 3.3v, the expansion and control hub analog input ports can safely handle up to 5v.

A Note on Hardware Call Speed

Every hardware call you make, (whether it be setting the power for a motor, setting a [servo](#) position, reading an encoder value, etc.) will take approximately 3 milliseconds to execute, except for I2C calls which can take upwards of 7ms. This is because behind the scenes, the SDK may need to make multiple hardware calls in order to perform the I2C operation.

Note

When using a Control Hub, you may see considerably faster hardware call times because the Control Hub uses a direct UART connection to the Lynx board instead of going through USB and a middle-man FTDI as happens when using a phone.

These times may seem fast, but they add up quickly. Consider a control loop to drive forward for N encoder counts while maintaining heading using the IMU. This would require 5 normal hardware calls (4 set power + 1 read encoder) and an I2C call (IMU) which means that the loop cycle would take approximately 22ms to execute, and thus run at approximately 45Hz.

This means that it is critical to minimize the amount of hardware calls you make in order to keep your control loops running fast. For instance, do not read a sensor more than once per loop. Instead, read it once and store the value to a variable if you need to use it again at other points in the same loop cycle.

Using a bulk read hardware call can help with this problem. A bulk read takes the same 3ms to execute as any other normal hardware call, but it returns far more data. In order to be able to use bulk reads, you must be running SDK v5.4 or higher. See [Bulk Reads](#) (page 333) for more information

10.1.5 Using Android Studio

[Android Studio](#)¹⁷¹ is an integrated development environment (IDE) for Android app development based on IntelliJ. It compiles your code to an apk which is then installed onto the Robot Controller: either the Control Hub or a legal Android phone.

Downloading Android Studio

If you've already downloaded Android Studio, you can move on to the next step, which is *setting up the SDK* (page ??).

The steps to download and setup Android Studio are:

1. Check to make sure your system meets the [necessary requirements](#)¹⁷²
2. Install the *latest* version of Android Studio from <https://developer.android.com/studio/index.html>
3. Run the executable, follow the setup wizard, and use any and all recommended development kits

Setting up the SDK

Now that you have Android Studio installed, you're going to want to use the current season's SDK (software development kit) where you will create your team's code.

Downloading the SDK

The SDK is publicly released to a GitHub repository every season. The current season's SDK can be found in the [FtcRobotController](#)¹⁷³ repository.

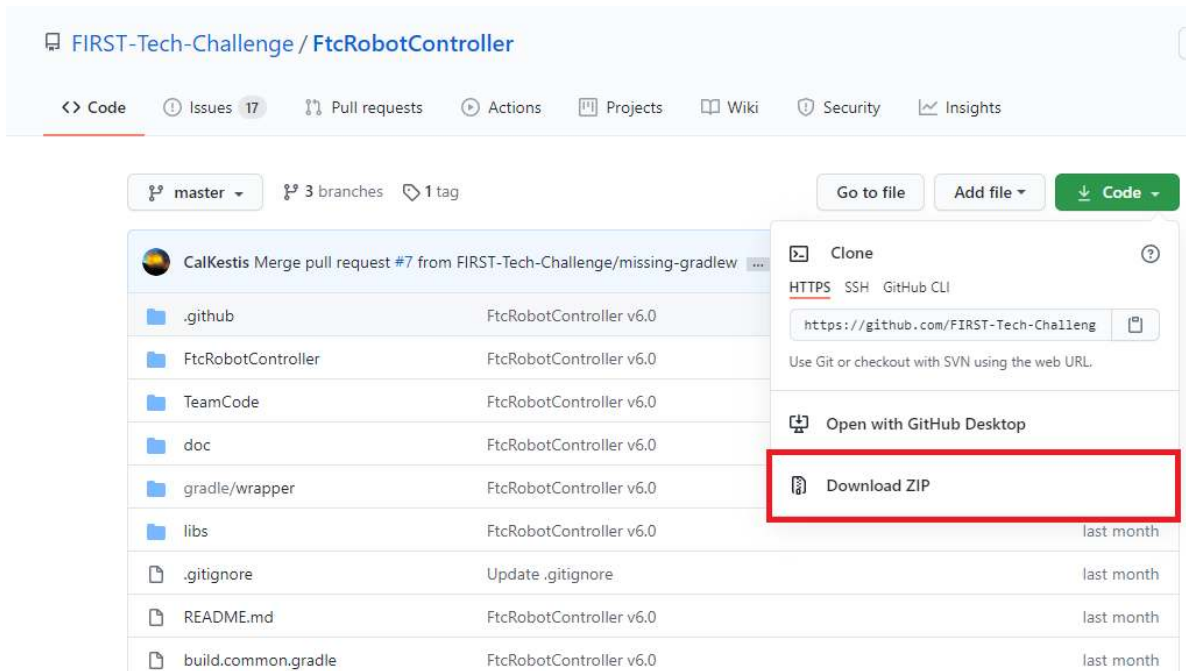
¹⁷¹ <https://developer.android.com/studio/intro>

¹⁷² <https://developer.android.com/studio#Requirements>

¹⁷³ <https://github.com/FIRST-Tech-Challenge/FtcRobotController>

Downloading the ZIP

1. When you're at the repository, click the green "code" button. Then, select "Download ZIP."



2. Then, save it to the desired location in your computer.



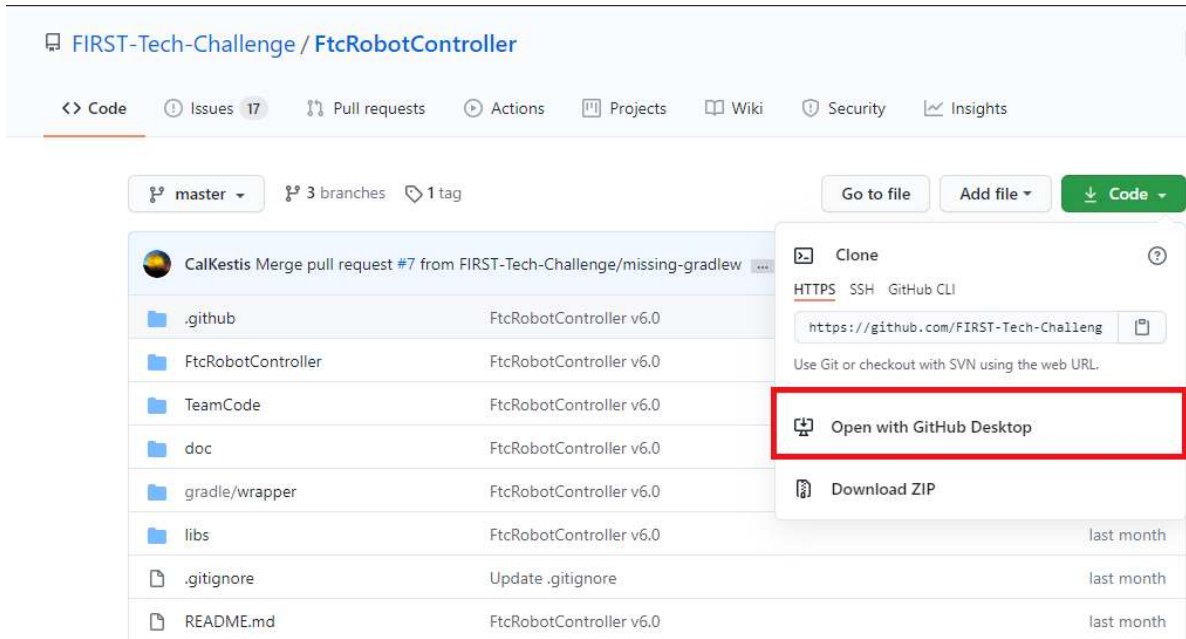
3. After it is saved, extract the contents of the ZIP and place them into whatever desired location. You should see the contents of the SDK inside of the folder location.

Using GitHub Desktop

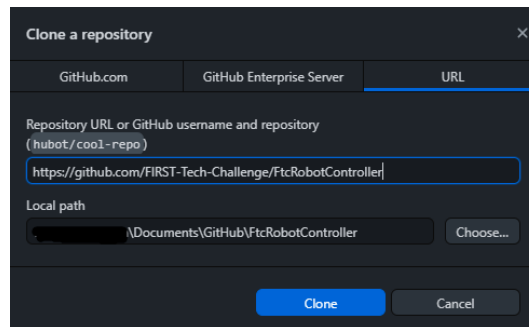
1. Install GitHub Desktop¹⁷⁴
2. Open the SDK repository¹⁷⁵ in a browser.
3. Click the green "code" button, and then select "Open with GitHub Desktop."

¹⁷⁴ <https://docs.github.com/en/desktop/installing-and-configuring-github-desktop/installing-and-authenticating-to-github-desktop>

¹⁷⁵ <https://github.com/FIRST-Tech-Challenge/FTCRobotController>



4. Clone the project.



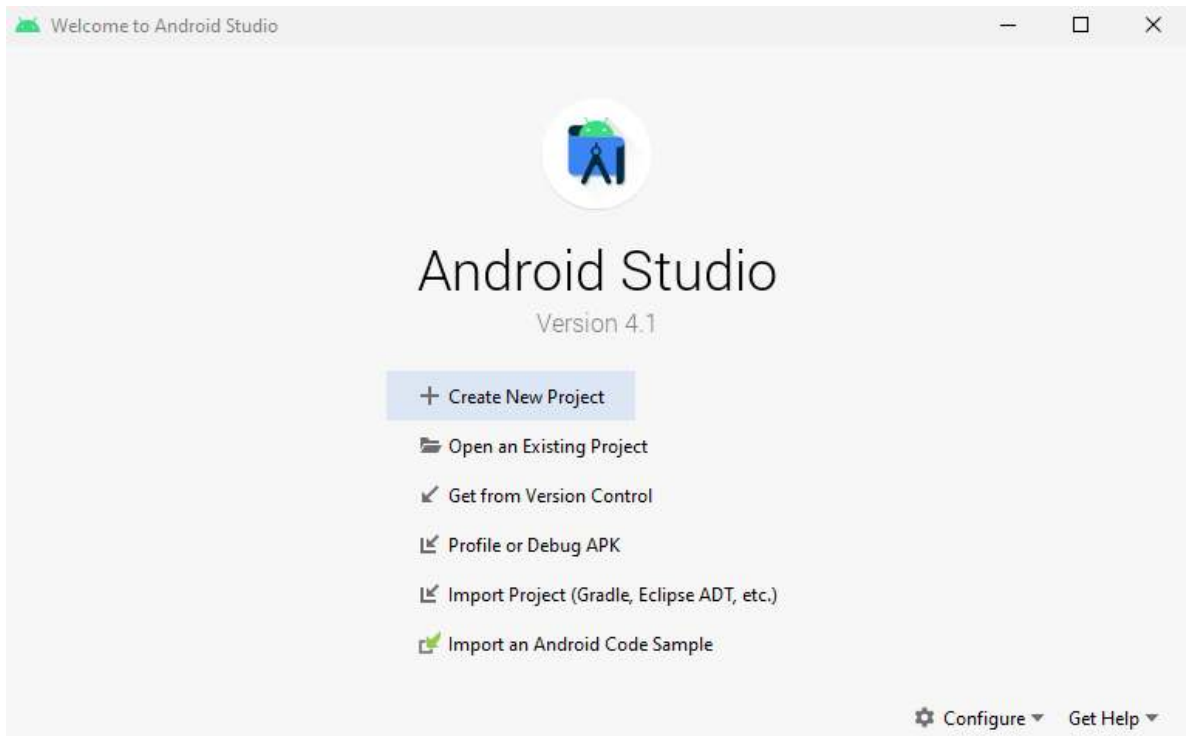
From the Command Line

1. Install git¹⁷⁶
2. Open the terminal (probably bash) in the desired resource location.
3. Use `$ git clone https://github.com/FIRST-Tech-Challenge/FtcRobotController.git`

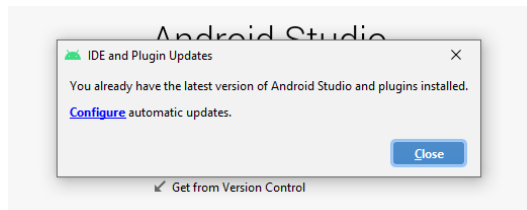
¹⁷⁶ <https://github.com/git-guides/install-git>

Opening the SDK on Android Studio

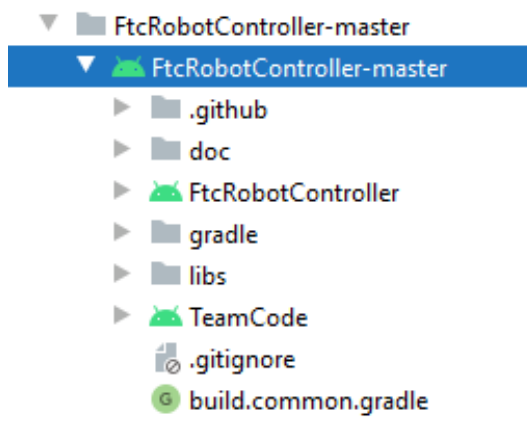
1. Open Android Studio. If you have another project open, close it.



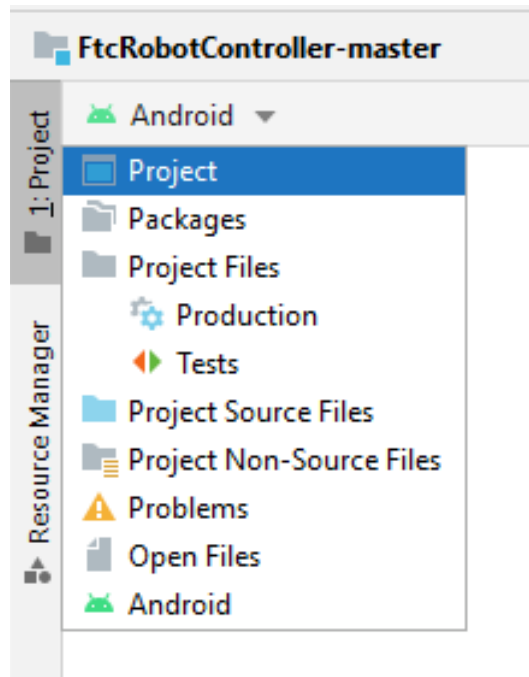
2. Check for updates. Click on the “configure” dropdown and select “check for updates.” If you do not have the latest version, download the updates.



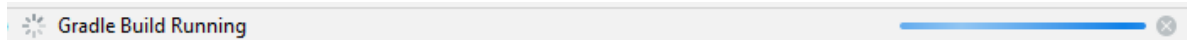
3. Select “Import Project.” Navigate to where you have the SDK saved on your computer. Choose the directory that has the Android logo.



4. Change to project view. In the top left corner should be a dropdown that allows you to change the way you are looking at your project.

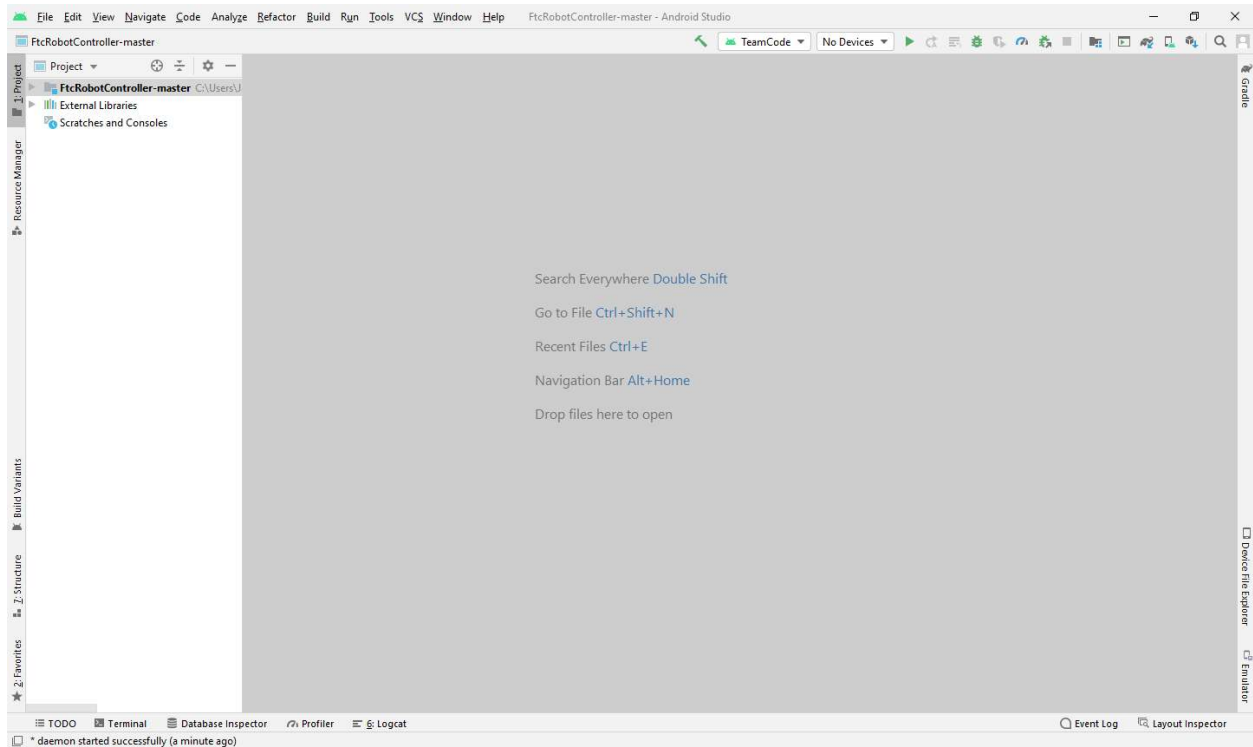


5. Wait for [Gradle](#) (page 284) to complete the build. This indicator should be located at the bottom of the window by default.



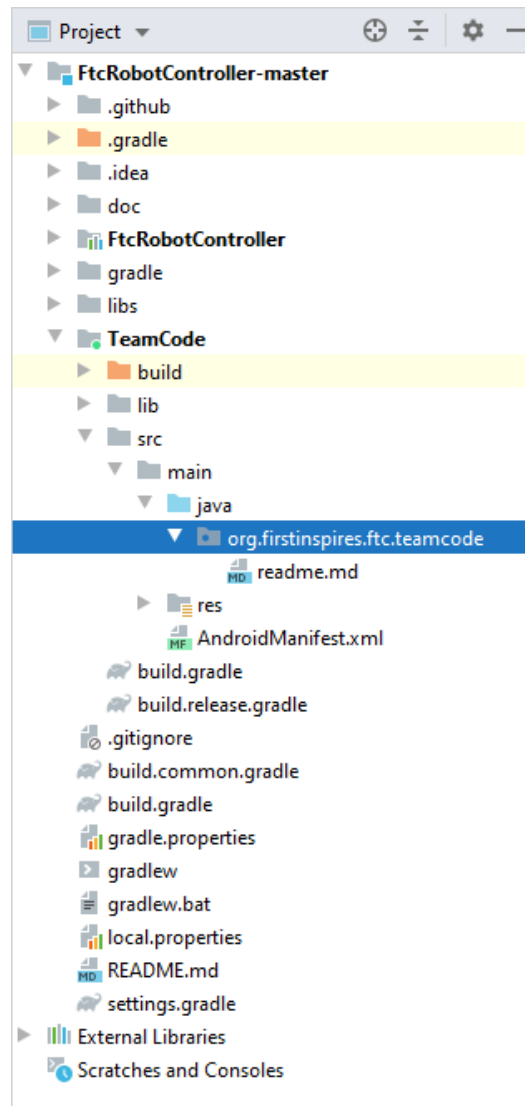
Layout

Android Studio can look intimidating at first glance, but there are only a few features needed to use it properly.

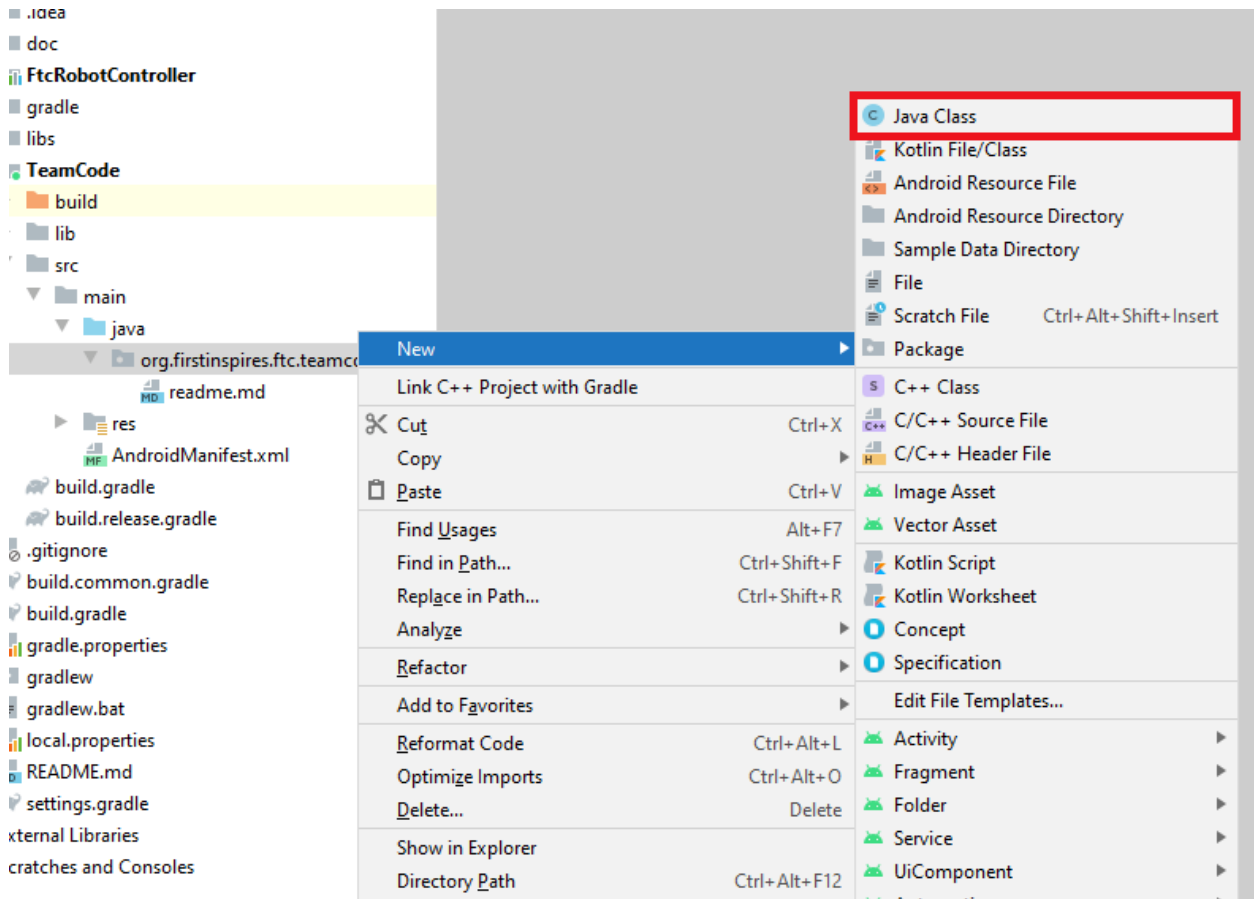


Creating Classes

The first thing to note in the project view is the dropdown with the name of the project. If you drop that down, you will see all of the Gradle files and directories. Navigate to the TeamCode folder. In the teamcode folder you will see an `org.firstinspires.ftc.teamcode` package.



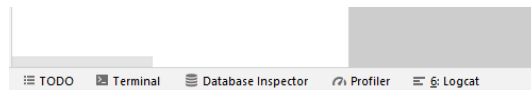
This is where you will create your code for the robot. To create a new Java class, right click on the package, select New, and then choose “Java Class.”



Alternatively, you can select the “Package” option if you want to create a subfolder for organization purposes. Then, you can create classes in those packages.

Terminal and Logcat

Near the bottom left of the application, you will find tabs for the local terminal and logcat. These are useful tools for working with your program.



Some useful information on using the logcat can be found [here](https://developer.android.com/studio/debug/am-logcat)¹⁷⁷.

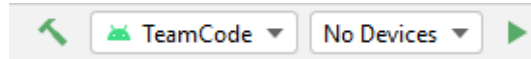
Note

You can build your program through the commandline via the local terminal. Click on the terminal tab and then input `gradlew :TeamCode:clean :TeamCode:build`. This will delete the previously compiled files and build your TeamCode module.

¹⁷⁷ <https://developer.android.com/studio/debug/am-logcat>

Installing Your Program

To install your program onto the Robot Controller, you will use the play button located near the top right of the application window.



Next to it you will see a dropdown for devices. When you connect your Robot Controller to your computer (using the correct cable), the device should appear in the dropdown after some time. Then, click the play button and your program will install onto the device.

Tip

Occasionally the app will fail to start on the robot controller, leaving the driver station in a disconnected state. If this occurs you can open the terminal and run

```
adb shell am start -n com.qualcomm.ftcrobotcontroller/org.firstinspires.ftc.  
robotcontroller.internal.PermissionValidatorWrapper
```

to remotely start the Robot Controller app.

If you run into any problems with this process, refer to the official [REV documentation](#)¹⁷⁸. Some useful pages from the REV site are:

- [Troubleshooting the Control System](#)¹⁷⁹
- [Deploying Code Wirelessly](#)¹⁸⁰

If you're still stuck you can ask for help in the [FTC|reg| Discord](#)¹⁸¹.

Gradle

Gradle is a build tool for software development. In the scope of FTC, it is used to build and manage dependencies for your project.

When you update any of your Gradle files you will need to perform a Gradle sync, which syncs your project to the changes and rebuilds it. In your `build.common.gradle`, you will find information for how your robot controller application is built.

Rebuilding

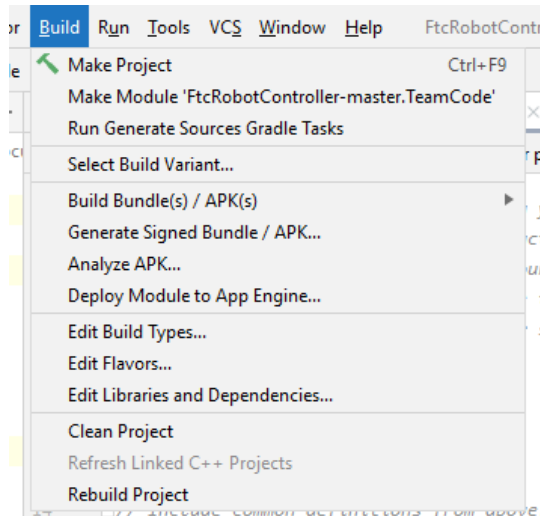
You can rebuild your project easily with the build dropdown.

¹⁷⁸ <https://docs.revrobotics.com/duo-control/>

¹⁷⁹ <https://github.com/FIRST-Tech-Challenge/FtcRobotController/wiki/Android-Studio-Tutorial>

¹⁸⁰ <https://docs.revrobotics.com/duo-control/programming/android-studio-using-wireless-adb>

¹⁸¹ <https://discord.com/invite/first-tech-challenge>



To rebuild from a clean project, press the clean project option. This removes old compiled files from your project so you can completely rebuild your project. It clears any production files, generated files, etc. This is useful for making sure old artifacts don't break anything when you build your code. When you next build your project, it will do so from scratch with no old compiled files to which it can refer. To rebuild your project, press the rebuild option.

Invalidate and Restart

Sometimes you can get errors after moving things around, refactoring, etc. The first step is to try cleaning the project and doing a rebuild. If this doesn't work, you might have confused Android Studio because it caches information about your project structure.

The most common way to fix these errors is to do an invalidate and restart. In the file dropdown, there will be an option for this and then you will choose Invalidate and Restart. This clears the cache and restarts your Android Studio, which then should perform a Gradle rebuild.

Adding Dependencies

If you want to add dependencies to your project, you can do so in the `build.gradle` file in the `TeamCode` directory.

There should be a dependencies block at the bottom of the file.

```
// Include common definitions from above.
apply from: '../build.common.gradle'
apply from: '../build.dependencies.gradle'

dependencies {
    implementation project(':FtcRobotController')
    annotationProcessor files('lib/OpModeAnnotationProcessor.jar')
}
```

Some dependencies require changes to your other Gradle files. Make sure to read the installation instructions for whatever dependency you want to add.

Next, you add a line in the dependencies block to implement the dependency. This is generally done with `implementation 'com.package.name'`.

```
dependencies {
    implementation project(':FtcRobotController')
    annotationProcessor files('lib/OpModeAnnotationProcessor.jar')

    implementation 'com.package.name:name:version'
}
```

Refer to the instructions of whatever library you are using for the correct package name and version.

Finally, perform a Gradle sync.

Upgrading to Java 8

By default, the SDK's version of Java is set to 7. Java 8 is also supported. You might want to upgrade your version of Java from 7 to 8 if you want to use features such as lambdas or generics. Some libraries may also require you to change your Java version.

To upgrade to Java 8, navigate to your `build.common.gradle` file. Scroll down until you find this block:

```
compileOptions {
    sourceCompatibility JavaVersion.VERSION_1_7
    targetCompatibility JavaVersion.VERSION_1_7
}
```

Change the 7 to 8, like so:

```
compileOptions {
    sourceCompatibility JavaVersion.VERSION_1_8
    targetCompatibility JavaVersion.VERSION_1_8
}
```

Then, perform a Gradle sync.

Android Debug Bridge

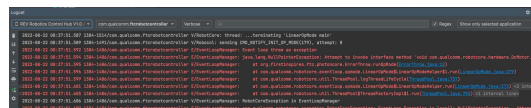
Note

On macOS, Linux, or using PowerShell you will have to change any commands that start with `adb` to start with `./adb` if you are in the `platform-tools` directory.

Logcat

Logcat is extremely useful for debugging issues with your code at runtime or figuring out what went wrong. For example, if your app activity crashes and you pull up the log seeing 5000 lines of the same error, there is probably infinite recursion in your code!

To use logcat, plug in your device (or connect via ADB). Then, select the app you want to view the logs for. Your window should look like this.



If you have an issue you don't understand, you can take a screenshot of the log or select and copy the error and ask a question in the [FTC discord](#)¹⁸².

Wireless Communication

Android Debug Bridge (ADB) is a command-line tool that allows for wireless communication between the robot controller (phone or Control Hub).

ADB should come with the platform tools in Android Studio. Navigate to your `local.properties` file in the root of your project and you should see a path to the Android SDK on your computer, such as `C:\Users\Woodie\AppData\Local\AndroidSdk`. Then navigate to `platform-tools` and an application called `adb` should be there. To use it, open CLI (like PowerShell or command prompt) and run either `adb devices` or `./adb devices`.

For more information on ADB, you can look at the [developers page](#)¹⁸³.

Setting Up ADB

1. Ensure USB debugging is enabled on your device and it is in developer mode.
2. Make sure you have ADB installed. If you do not, follow the instructions at [this link](#)¹⁸⁴

Note

You can use `logcat` via ADB with the `adb logcat` command. This is extremely useful for debugging as it allows you to look at the logs wirelessly which saves time. Remember, `logcat` is the *best* way to debug your software.

Add ADB To PATH

Adding variables to PATH:

- [Windows](#)¹⁸⁵
- [Linux/Unix \(bash\)](#)¹⁸⁶
- [macOS \(zsh\)](#)¹⁸⁷

If you want to use ADB from any directory, add it to PATH. Follow an online tutorial for adding to PATH and set the PATH to the `platform-tools` directory. Once you do that, you can run ADB commands from anywhere on your system.

¹⁸² <https://discord.com/invite/first-tech-challenge>

¹⁸³ <https://developer.android.com/studio/command-line/adb>

¹⁸⁴ <https://www.xda-developers.com/install-adb-windows-macos-linux/>

¹⁸⁵ <https://docs.alfresco.com/content-services/latest/admin/troubleshoot/>

¹⁸⁶ <https://unix.stackexchange.com/questions/26047/how-to-correctly-add-a-path-to-path>

¹⁸⁷ <https://koenwoortman.com/zsh-add-directory-to-path/>

Connecting to a Phone Wirelessly

1. Plug the robot controller phone into your computer.
2. Run the command `adb devices` in the `platform-tools` directory and see if the phone shows up.
3. Run `adb usb` and then `adb tcpip 5555`. You can then unplug the phone.
4. Connect to the same WiFi network the device is either hosting or on. The WiFi direct network created by the phone should be called "TEAMNUMBER-RC" or some small derivation of that. It may include extra letters if you have multiple devices per team. Refer to RS01 in Game Manual Part 1 for further details on the network naming scheme.
5. Connect to the phone using `adb connect 192.168.49.1:5555`. If this doesn't work, recheck the IP address of the phone and try again with that IP address if it is different.

Connecting to a Control Hub Wirelessly

1. Connect to the WiFi hotspot hosted by the Control Hub. The hotspot should be called "TEAMNUMBER-RC" or some small derivation of that. It may include extra letters if you have multiple devices per team. Refer to RS01 in Game Manual Part 1 for further details on the network naming scheme.
2. Once you're connected to a Control Hub's network, you simply need to connect to it using `adb connect 192.168.43.1:5555`.

Once a connection is established, it should appear in the device dropdown in Android Studio.

Wireless Configuration

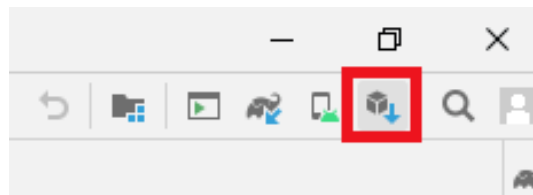
You can set up a configuration on the Driver Station or Robot Controller like usual. However, you can also create a valid configuration XML file in `TeamCode/src/main/res/xml`. You can find your configuration files in the `/sdcard/FIRST` folder as an XML file with the same name as the configuration.

To get these XML files wirelessly, you can use `adb pull /sdcard/FIRST/config.xml /fully/qualified/path/res/xml`.

If a valid configuration XML file is in `res/xml` it will show up as a configuration you can use for the robot when you push it to the Robot Controller or a Control Hub.

SDK Manager

You can find the SDK manager in the top right corner of your Android Studio.



Accepting Licenses

If you get a warning complaining about licenses not being accepted, follow these steps:

1. Go to the SDK manager and under SDK Platforms.
2. Select the version with the API level specified by the warning.
3. Click “Apply” and wait for the components to install.
4. Once this finishes, press “Finish,” then “Ok.” Wait for Android Studio to index if it is.
5. Restart Android Studio.

Installing SDK Tools

To install any SDK tools such as platform tools or build tools, open the SDK manager and go to SDK Tools. Select the tools you want to install and install them the same way you would for the SDK platforms.

Version Control

Version control is an extremely useful tool. It allows for looking at (and reverting to) previous versions of code, easy collaboration, having multiple versions of code that can be merged together, etc.

As far as version control systems go, we strongly recommend git, especially when used with a GUI like Android Studio’s built in VCS tools or Github Desktop. While a git tutorial is out of scope for Game Manual 0, here are some git resources:

- [The official git tutorial](https://git-scm.com/docs/gittutorial)¹⁸⁸
- [GitHub’s collection of git resources](https://docs.github.com/en/get-started/quickstart/set-up-git)¹⁸⁹
- [GitHub’s guide to installing git](https://github.com/git-guides/install-git)¹⁹⁰
- [GitHub Desktop, a git GUI](https://desktop.github.com/)¹⁹¹
- [Android Studio’s/IntelliJ’s git integration documentation](https://www.jetbrains.com/help/idea/version-control-integration.html)¹⁹²

10.1.6 Common Issues

Warning

Be careful what code you take as a reference from this page! Some of it is intentionally buggy to demonstrate potential easy-to-make errors.

¹⁸⁸ <https://git-scm.com/docs/gittutorial>

¹⁸⁹ <https://docs.github.com/en/get-started/quickstart/set-up-git>

¹⁹⁰ <https://github.com/git-guides/install-git>

¹⁹¹ <https://desktop.github.com/>

¹⁹² <https://www.jetbrains.com/help/idea/version-control-integration.html>

Exceptions

Exceptions are events that occur during the execution of a program, disrupting the normal flow of instructions, used in error events or problems that arise during runtime. A exception can be caught to avoid propagation, otherwise any exception that's not handled will cause the program flow to stop immediately.

Some common types of exceptions include:

- **NullPointerException**

- It occurs when trying to call a method or getting a property of an object from a variable with a *null* value, which basically means that the variable doesn't hold a value yet, or the value doesn't exist.
- This exception is one of the most common in FTC®, below is an example that throws a `NullPointerException`:

```
public class CrashyOpMode extends OpMode {

    // This call to the "get" method here will throw a NullPointerException.
    //
    // The value of the "hardwareMap" variable is null at this point, due to
    // the way the SDK is limited to define the value of this variable, its
    // value is defined right before the init() (or runOpMode() in LinearOpModes)
    // method is called.
    Servo clawServo = hardwareMap.get(Servo.class, "claw");

    @Override
    public void init() {
        // This statement won't ever be reached due to the
        // thrown NullPointerException, explained above,
        // since it happens before the OpMode starts execution.
        clawServo.setPosition(0.5);
    }

    // ...

}
```

- This can be fixed moving the “Servo” variable value definition to the `init` (or `runOpMode()` in `LinearOpModes`) method as follows:

```
public class WorkingOpMode extends OpMode {

    Servo clawServo = null;

    @Override
    public void init() {
        // This won't throw a NullPointerException since the value of the
        // "hardwareMap" variable is defined at this point, but note that
        // the "get" method will return null if the name "claw" isn't
        // configured. (consult the "Using the SDK" section)
        clawServo = hardwareMap.get(Servo.class, "claw");

        // This statement should be reached and executed now.
        //
        // Note that if the "claw" servo is not configured, the value returned
        // by the hardwareMap will be null as explained before, therefore,
        // a NullPointerException would be thrown here if that happens.
        clawServo.setPosition(0.5);
    }

}
```

(continues on next page)

(continued from previous page)

```

    }

    // ...

}

```

• TargetPositionNotSetException

- This exception type is a custom one from the SDK. It means you changed the motor RunMode to RUN_TO_POSITION before setting a target position:

```

// This will throw a "TargetPositionNotSetException" here!
motor.setRunMode(DcMotor.RunMode.RUN_TO_POSITION);

// And this statement won't be reached.
motor.setTargetPosition(1120);

```

- It is fixed by simply switching the order of the statements; setting target position first, then changing the RunMode:

```

// Setting the target position first
motor.setTargetPosition(1120);

// Then switching the RunMode
motor.setRunMode(DcMotor.RunMode.RUN_TO_POSITION);

```

• ArithmeticException

- Occurs when performing any illegal arithmetic operations such as dividing by zero:

```

int number = 128 / 0; // This will throw an ArithmeticException!

```

- It can be handled by enclosing the code likely to throw this type of exception with a try catch block¹⁹³:

```

int number; // Declaring the variable in the outside scope

try {
    // Giving it a value that will possibly throw an ArithmeticException
    number = 128 / 0;
} catch (ArithmeticException e) {
    // Do something when the ArithmeticException happens.
    // (The value of the "number" variable will remain 0)
}

```

• InterruptedException

- It means that the SDK requested the OpMode to stop, and it's considered part of normal operation. An interrupt means that the current thread has been requested to end, so don't panic when you see a spam of those in *logcat* (page 286)!
- If you call a method that possibly throws an InterruptedException (such as `Thread.sleep()`) it should be handled like this, with the try catch syntax mentioned before:

¹⁹³ https://www.w3schools.com/java/java_try_catch.asp

```
try {
    // Block for 500 milliseconds
    Thread.sleep(500);
} catch (InterruptedException e) {
    // Tells the current thread (OpMode) to
    // end the execution as soon as possible
    Thread.currentThread().interrupt();
}
```

- Note that LinearOpMode already contains a shorthand `sleep()`¹⁹⁴ method that does this under the hood. (And you shouldn't be using sleeps in OpMode since they're more strictly controlled. Read next sections for further information)

How the SDK handles exceptions

Except for `InterruptedExceptions` and some other internal special cases, which simply cause the OpMode to end, the FTC SDK performs an “emergency stop” routine when an exception is thrown and not handled properly. This halts the OpMode and displays the full stacktrace on screen. The stacktrace can also be viewed through *Logcat* (page 286) when using Android Studio.

Note

Before SDK 8.0, only the first line of the error would be displayed and selecting “Restart Robot” from the menu would be required before running an OpMode again.

It's generally a good idea to debug all OpModes extensively before any official match, as these exceptions are disruptive.

Stuck in start, loop, stop...

OpModes are *strictly controlled programs*, in the sense that the SDK requires them to flow in a certain way with the methods `init()`, `loop()`, etc. If you take more than a specific time (5 seconds, or 900 milliseconds in stop commands¹⁹⁵) executing an action in any of these methods, the SDK will perform the “emergency stop” routine explained before, with the “stuck in action” error message.

```
public class StuckyOpMode extends OpMode {

    // ...

    @Override
    public void loop() {
        // Don't do this in a normal iterative OpMode!
        // This will cause a "stuck in stop" error after
        // 5 seconds, since iterative OpModes shouldn't
        // be blocked by loops or any lengthy operation.
        while(true) {
            // ...
        }
    }
}
```

(continues on next page)

¹⁹⁴ <https://github.com/OpenFTC/Extracted-RC/blob/f47d6f15fa1b59faaf509a522e0ec04f223ec125/RobotCore/src/main/java/com/qualcomm/robotcore/eventloop/opmode/LinearOpMode.java#L96>

¹⁹⁵ <https://github.com/OpenFTC/Extracted-RC/blob/f47d6f15fa1b59faaf509a522e0ec04f223ec125/RobotCore/src/main/java/com/qualcomm/robotcore/eventloop/opmode/OpMode.java#L189>

(continued from previous page)

```

    }
}

```

If you need to run any sort of lengthy action in your OpMode, another option would be using a LinearOpMode instead.

LinearOpModes are less strict since their single `runOpMode()` method can flow more freely, but they still need to be cooperative to stop requests. Take the following code as an example:

```

public class StuckyLinearOpMode extends LinearOpMode {

    @Override
    public void runOpMode() {
        // Wait for the driver to press PLAY on the DS
        waitForStart();

        while(true) {
            // Do stuff infinitely
        }
    }
}

```

This code isn't cooperative to stop requests, since the while loop doesn't have an exit condition to cooperate with the OpMode stopping, therefore, this code will cause a "stuck in stop" error once it's stopped in the Driver Station.

To cooperate with the stopping of the OpMode, an `opModeIsActive()` or `!isStopRequested()` condition is required to be added to all the blocking loops executed in the `runOpMode()` method. Consult the [LinearOpMode vs OpMode](#) (page 271) page for more information about these methods.

An example for a cooperative LinearOpMode would be as follows:

```

public class CooperativeLinearOpMode extends LinearOpMode {

    @Override
    public void runOpMode() {
        while(someCondition && !isStopRequested()) {
            // Do something while the "someOtherCondition"
            // is true and the OpMode is not stopped.
        }

        // Wait for the driver to press PLAY on the DS
        waitForStart();

        while(someOtherCondition && opModeIsActive()) {
            // Do something while the "someCondition" is true
            // and the OpMode is running (started and not stopped).
        }
    }
}

```

10.2 Tutorials

Tutorials for common FTC® programs.

10.2.1 Mecanum TeleOp

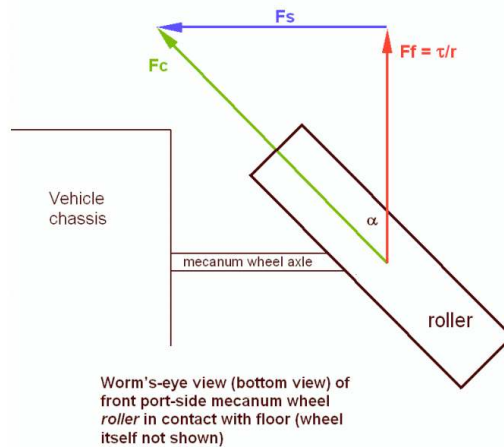
Mecanum Physics

Mecanum drive is a very popular drivetrain type in FTC®. Mecanum drivetrains enables holonomic movement. This means that the drivetrain is able to move in any direction while rotating: forwards, backwards, side to side, translating while rotating, etc. [Here is a neat video](#)¹⁹⁶ demonstrating such movement.

Note

Some common COTS mecanum drivetrain kits are the [goBILDA Strafer Chassis Kit](#)¹⁹⁷ and the [REV Mecanum Drivetrain Kit](#)¹⁹⁸.

Mecanum wheels have rollers at a 45° angle to the rest of the wheel. Since these are in contact with the ground instead of something solid like in a *traction wheel*, instead of the wheel creating a force parallel to the orientation of the wheel, it creates one 45° from parallel. Depending on how the wheels are driven, X or Y components of the force vectors can cancel which allows movement in any direction.



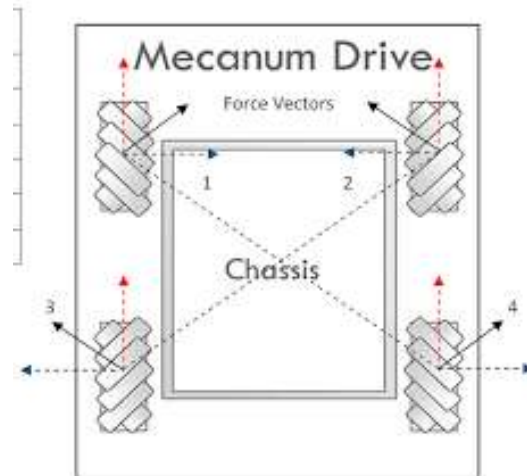
¹⁹⁶ <https://www.youtube.com/watch?v=pP8ajNMx84k>

¹⁹⁷ <https://www.gobilda.com/strafer-chassis-kit-v5/>

¹⁹⁸ <https://www.revrobotics.com/rev-45-2470/>

Using Vectoring to Create Omnidirectional Movement

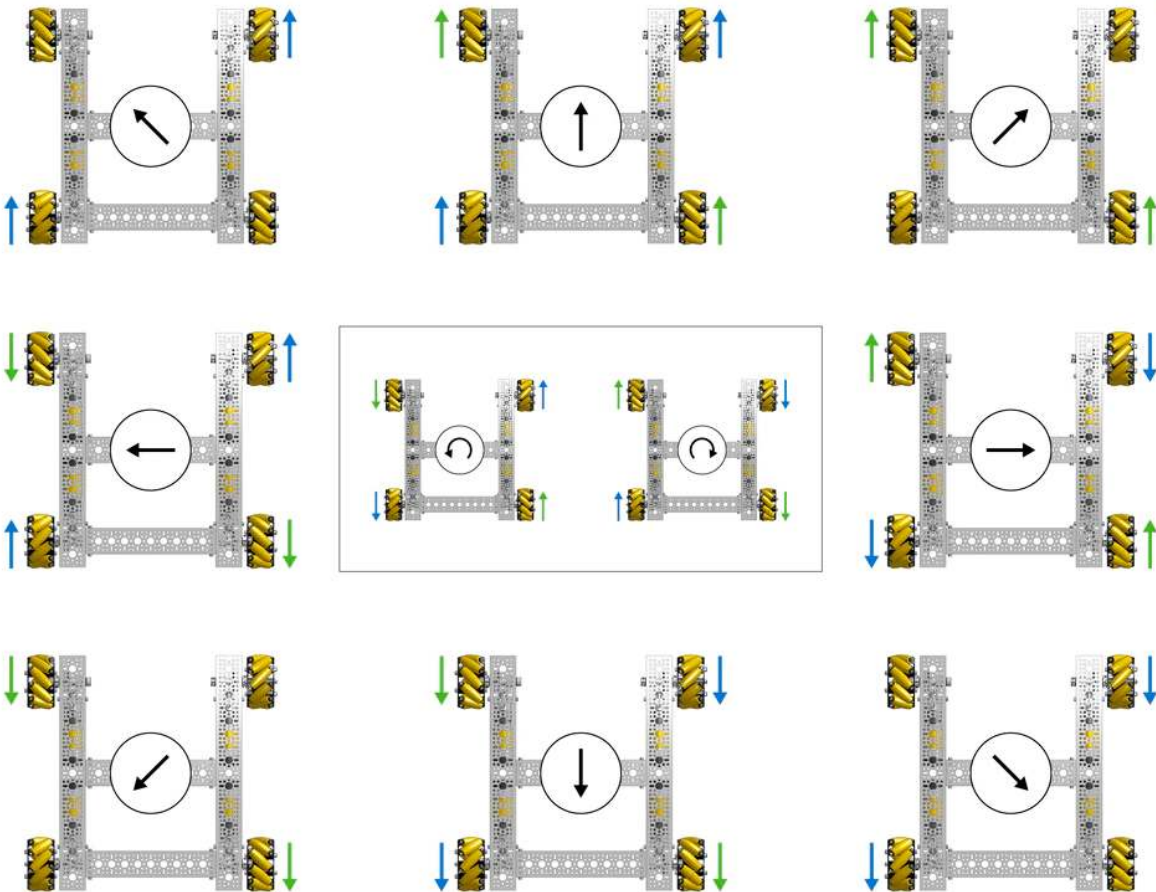
A standard mecanum drive configuration possesses 4 mecanum wheels oriented in an “X” shape. This means that the rollers are angled towards the center when looking at it from above. This configuration allows one to add up the force vectors generated by the offset rollers and derive movement in any direction. It is important to note that because of friction, perfect movement isn’t possible in every direction, so a *mecanum drivetrain* will be able to drive slightly faster forwards/backwards than any other directions. Combining translation and rotation will also result in slower movement.



In the image above, vectors 1, 2, 3, and 4 are the force vectors created by the *mecanum wheels* when the chassis is instructed to drive towards the top of the image. All motors are driving forward. The blue and red lines are their X and Y components, respectively. Here are a few examples of how the wheels must be driven to achieve different movements:



PRODUCT INSIGHTS



⚠ Attention

It is strongly advised to not hardcode these movements in; there is a much better way described below that allows for true holonomic movement and is much more elegant.

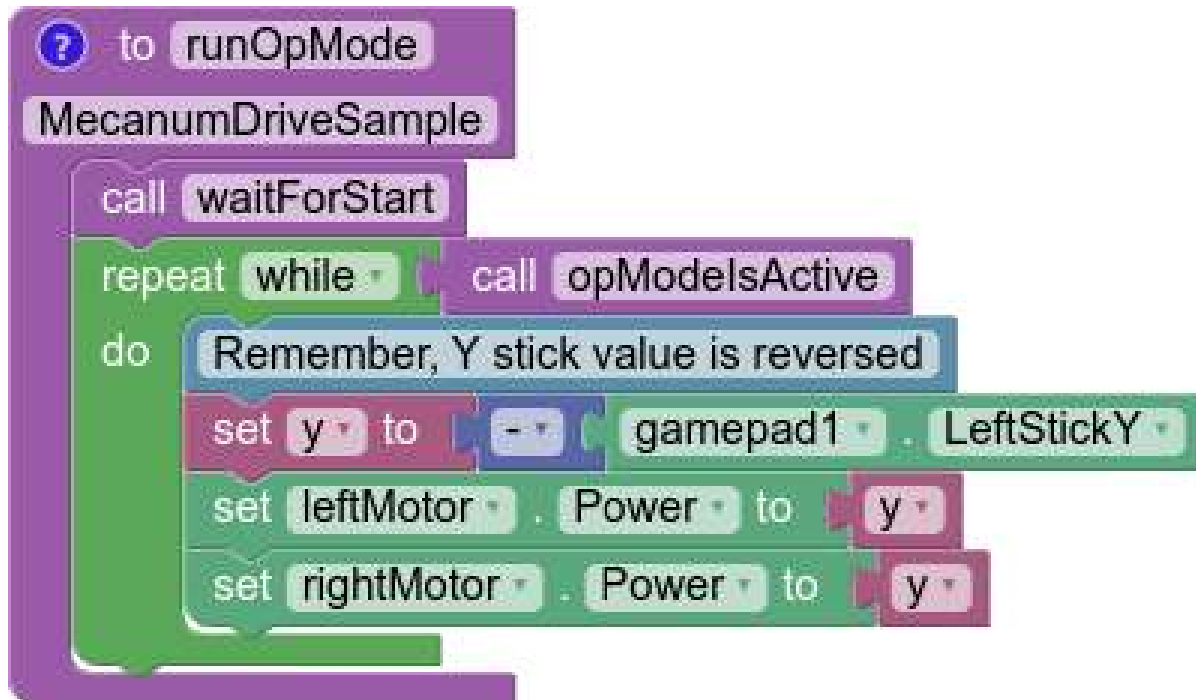
Deriving Mecanum Control Equations

Before thinking about mecanum, envision a scenario where you have a 2 motor tank drivetrain which you want to control using the left stick Y axis for forward/backward movement, and the right stick X axis for pivot turning. The motors are configured so that positive is clockwise for the right motor when the body is facing away from you, and the left motor is the opposite. To control only forward/backward movement, you simply need to set the motor powers to the Y stick value (flip the sign since Y is reversed):

Java

```
double y = -gamepad1.left_stick_y; // Remember, Y stick is reversed!
leftMotor.setPower(y);
rightMotor.setPower(y);
```

Blocks



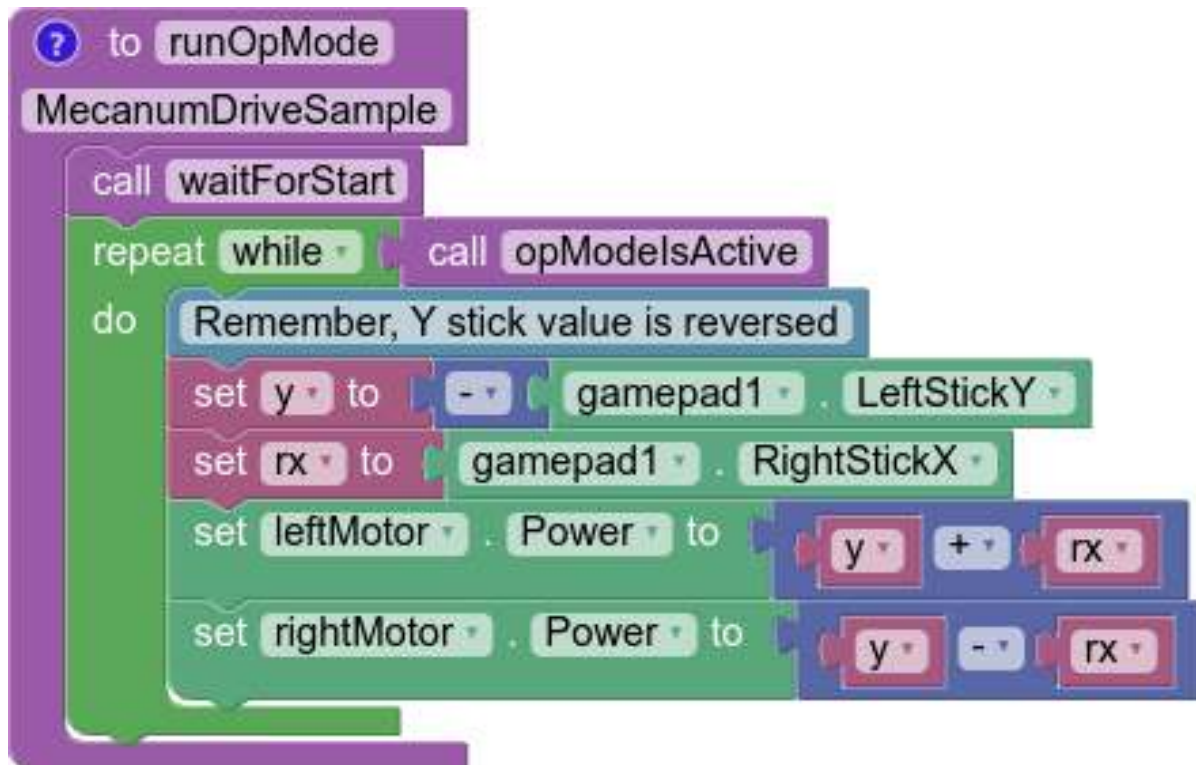
Although at first adding rotation might seem like a difficult task, it's actually super simple. All you need to do is subtract the right X stick value from the right wheels, and add it to the left:

Java

```
double y = -gamepad1.left_stick_y; // Remember, Y stick is reversed!
double rx = gamepad1.right_stick_x;

leftMotor.setPower(y + rx);
rightMotor.setPower(y - rx);
```

Blocks



Here, if the left stick is pressed upwards, both of the motors will be fed a positive value, causing the robot to move forward. If it is pressed downwards, both of the motors will be fed a negative value, causing the robot to move backwards. A similar principle applies for rotation: if the right stick is pushed rightward, the left wheels will spin forward while the right spin backward, causing rotation. The opposite applies for pushing the stick left. If both sticks are pushed at the same time, say the left Y stick is at 1 and the right X stick is also at 1, the value of the left wheels will be $1 + 1 = 2$ (which gets clipped to 1 in the SDK) and the right wheels will be $1 - 1 = 0$, which causes a rightward curve.

Applying omnidirectional movement with *mecanum wheels* operates under the same principle as adding turning into the tank example. The left stick X values will be added or subtracted to each wheel depending on how that wheel needs to rotate to get the desired movement. The only difference from turning is that rather than wheels on the same side being the same sign, wheels diagonal to each other will be the same sign.

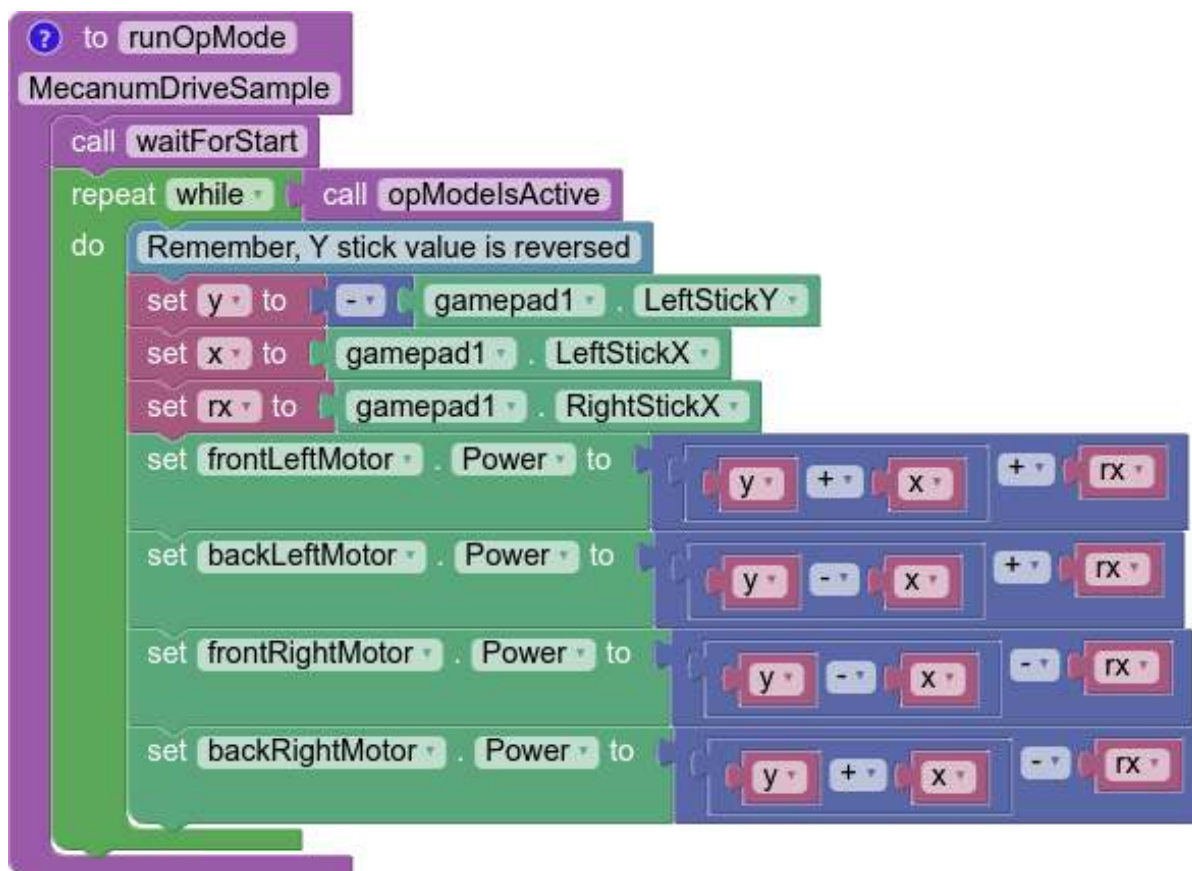
We want a positive left stick X value to correlate to rightward strafing. If we refer back to the vectoring image, this means that the front left and back right need to rotate forward, while the back left and front right need to rotate backwards. So, we should add the x value to the front left and back right and subtract it from the back right and front left:

Java

```
double y = -gamepad1.left_stick_y; // Remember, Y stick is reversed!
double x = gamepad1.left_stick_x;
double rx = gamepad1.right_stick_x;

frontLeftMotor.setPower(y + x + rx);
backLeftMotor.setPower(y - x + rx);
frontRightMotor.setPower(y - x - rx);
backRightMotor.setPower(y + x - rx);
```

Blocks



Important

Most FTC motors spin counterclockwise when viewed from their face when given positive power by default, with the exception of NeveRests. If your drivetrain uses an even number of gears, this will reverse the direction the motors spin in.

On most drivetrains, you will need to reverse the left side for positive power to move forwards with most motors, and reverse the right side with NeveRests. The presence of gearing between the motor gearbox and the wheel may swap this, which is the case for the goBILDA Strafer and the REV Mecanum Drivetrain Kit.

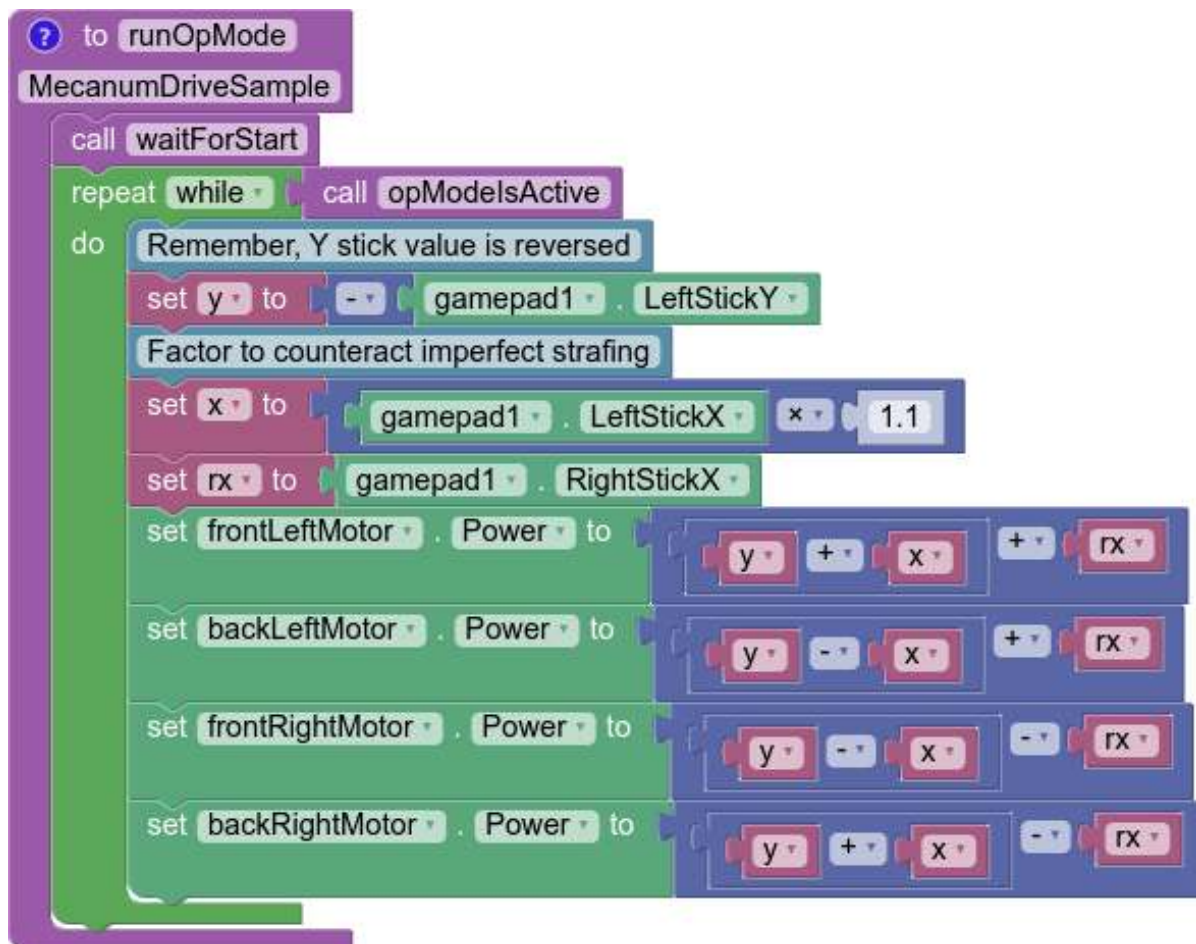
This is the same as the tank example, except now with 4 motors and the strafing component added. Similarly to the tank example, the Y component is added to all wheels, and the right X (rx) is added to the left wheels and subtracted from the right. Now, we have added a left X component (x) that allows us to strafe rightward. In doing that, however, we have actually allowed for strafing in any direction. If you think about it, pressing the left joystick to the left will do the same thing in reverse, which is what is needed to strafe left. If it is pressed at 45 degrees, the x and y components of the joystick will be equal. This will cause two diagonal motors to cancel, allowing for diagonal movement. This same effect applies to every angle of the joystick.

Now that we have a functioning mecanum driving program, there are a few things that can be done to clean it up. The first of these would be multiplying the left X value by something to counteract imperfect strafing. Doing this will make the drive feel more accurate on non axis aligned directions, and make field centric driving more accurate. In this tutorial, we will use 1.1, but it's really up to driver preference.

Java

```
double y = -gamepad1.left_stick_y; // Remember, Y stick is reversed!
double x = gamepad1.left_stick_x * 1.1; // Counteract imperfect strafing
double rx = gamepad1.right_stick_x;
```

Blocks



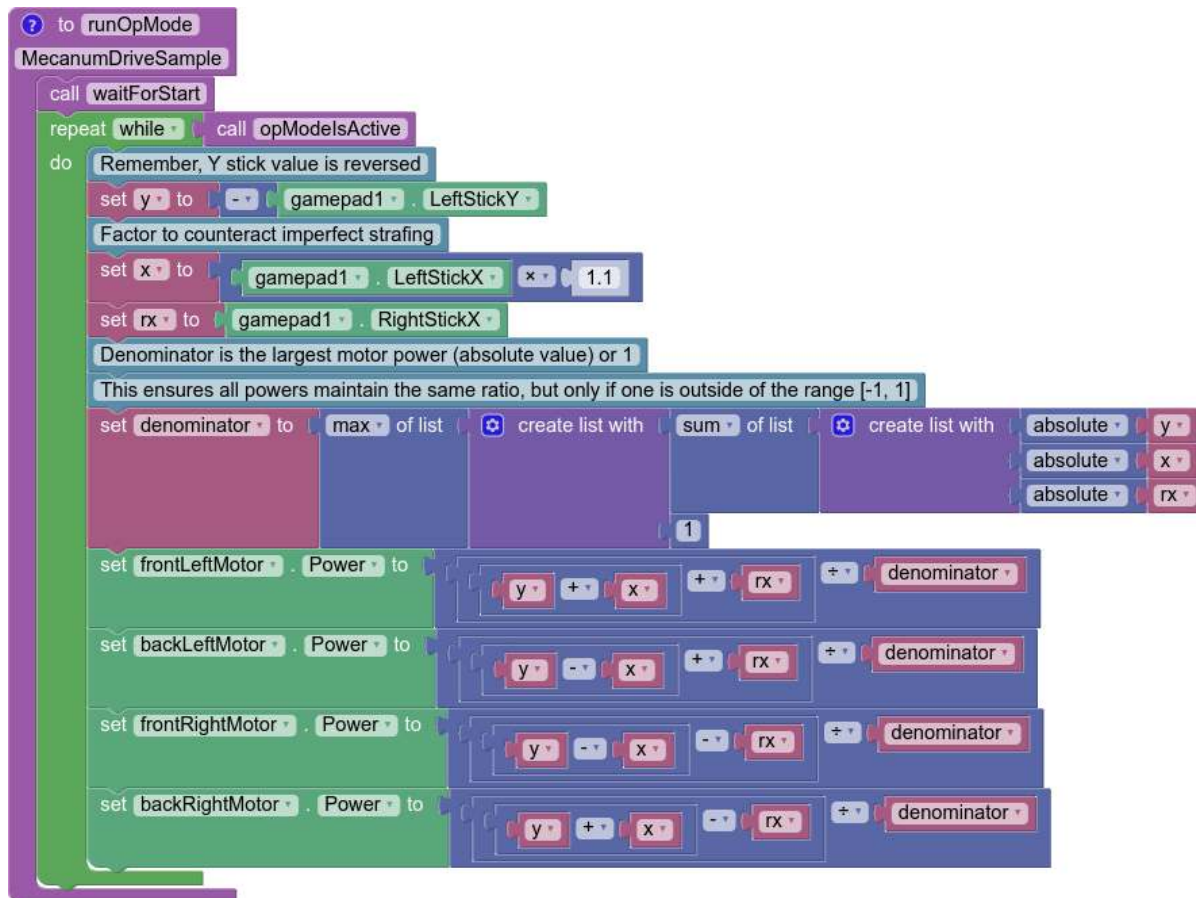
The other improvement we can make is scale the values into the range of -1 to 1.

Since the SDK simply clips (limits) the powers to that range, we can lose the ratio we are looking for unless we proactively put all the numbers back in that range while still maintaining our calculated ratio. For example, if we calculate values of 0.4, 0.1, 1.1, and 1.4, they will be clipped to 0.4, 0.1, 1.0, and 1.0, which is not the same ratio. Instead, we need to divide all of them by the largest power's absolute value when it exceeds 1:

Java

```
// Denominator is the largest motor power (absolute value) or 1
// This ensures all the powers maintain the same ratio, but only when
// at least one is out of the range [-1, 1]
double denominator = Math.max(Math.abs(y) + Math.abs(x) + Math.abs(rx), 1);
double frontLeftPower = (y + x + rx) / denominator;
double backLeftPower = (y - x + rx) / denominator;
double frontRightPower = (y - x - rx) / denominator;
double backRightPower = (y + x - rx) / denominator;
```

Blocks



Make sure to set the powers on your motor and update this every loop in an opmode!

Robot-Centric Final Sample Code

Java

```

package org.firstinspires.ftc.teamcode;

import com.qualcomm.robotcore.eventloop.opmode.LinearOpMode;
import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
import com.qualcomm.robotcore.hardware.DcMotor;
import com.qualcomm.robotcore.hardware.DcMotorSimple;

@TeleOp
public class MecanumTeleOp extends LinearOpMode {
    @Override
    public void runOpMode() throws InterruptedException {
        // Declare our motors
        // Make sure your ID's match your configuration
        DcMotor frontLeftMotor = hardwareMap.dcMotor.get("frontLeftMotor");
        DcMotor backLeftMotor = hardwareMap.dcMotor.get("backLeftMotor");
        DcMotor frontRightMotor = hardwareMap.dcMotor.get("frontRightMotor");
        DcMotor backRightMotor = hardwareMap.dcMotor.get("backRightMotor");

        // Reverse the right side motors. This may be wrong for your setup.
        // If your robot moves backwards when commanded to go forwards,
        // reverse the left side instead.
        // See the note about this earlier on this page.
        frontRightMotor.setDirection(DcMotorSimple.Direction.REVERSE);
        backRightMotor.setDirection(DcMotorSimple.Direction.REVERSE);

        waitForStart();

        if (isStopRequested()) return;

        while (opModeIsActive()) {
            double y = -gamepad1.left_stick_y; // Remember, Y stick value is reversed
            double x = gamepad1.left_stick_x * 1.1; // Counteract imperfect strafing
            double rx = gamepad1.right_stick_x;

            // Denominator is the largest motor power (absolute value) or 1
            // This ensures all the powers maintain the same ratio,
            // but only if at least one is out of the range [-1, 1]
            double denominator = Math.max(Math.abs(y) + Math.abs(x) + Math.abs(rx), 1);

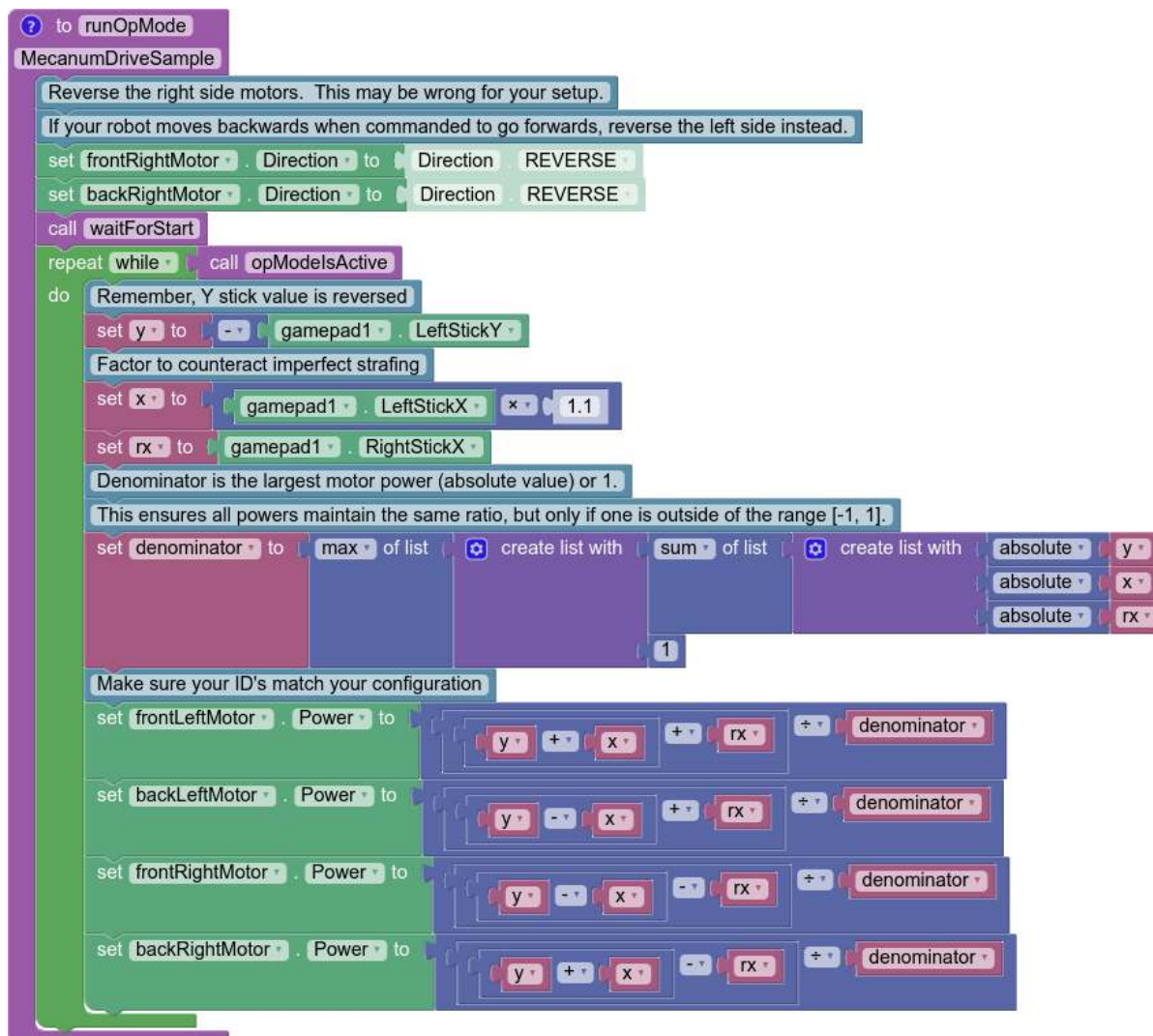
            double frontLeftPower = (y + x + rx) / denominator;
            double backLeftPower = (y - x + rx) / denominator;
            double frontRightPower = (y - x - rx) / denominator;
            double backRightPower = (y + x - rx) / denominator;

            frontLeftMotor.setPower(frontLeftPower);
            backLeftMotor.setPower(backLeftPower);
            frontRightMotor.setPower(frontRightPower);
            backRightMotor.setPower(backRightPower);
        }
    }
}

```

Blocks

Blocks file download



Field Centric

With field centric mecanum drive, the translation joystick controls the direction of the robot relative to the field, as opposed to the robot frame. This is preferred by some drivers, and make some evasive action easier, as one can spin while translating in a given direction easier. To do this, the x/y components of the joysticks are rotated counter to the robot's angle, which is given by the IMU.

There is an IMU inside of Control Hubs (and older models of Expansion Hubs). Unlike most other hardware, it is recommended to do more than `hardwareMap.get()` to begin using it. Note, this is configured when creating a new configuration by default as `imu`. See the [FTC doc page covering the IMU interface and its parameters](https://ftc-docs.firstinspires.org/programming_resources/imu/imu.html)¹⁹⁹ for more information. The way the IMU will be initialized here is:

¹⁹⁹ https://ftc-docs.firstinspires.org/programming_resources/imu/imu.html

```
// Retrieve the IMU from the hardware map
imu = hardwareMap.get(IMU.class, "imu");
// Adjust the orientation parameters to match your robot
IMU.Parameters parameters = new IMU.Parameters(new RevHubOrientationOnRobot(
    RevHubOrientationOnRobot.LogoFacingDirection.UP,
    RevHubOrientationOnRobot.UsbFacingDirection.FORWARD));
// Without this, the REV Hub's orientation is assumed to be logo up / USB forward
imu.initialize(parameters);
```

The angle needs to be read every loop. In addition to this, while the IMU keeps a consistent zero position between OpModes (relevantly, including between autonomous and teleop), adding a bind to reset the angle is important to counteract drift and because the zero can change due to some types of disconnects.

Note

BN0055 objects will reset the IMU zero when initialize is called. The BN0055 class is not recommended for new development. The IMU class does not have this behavior, and is the appropriate replacement as of SDK v8.1.

```
// This button choice was made so that it is hard to hit on accident,
// it can be freely changed based on preference.
// The equivalent button is start on Xbox-style controllers.
if (gamepad1.options) {
    imu.resetYaw();
}

double botHeading = imu.getRobotYawPitchRollAngles().getYaw(AngleUnit.RADIANS);
```

Then, the translation joystick values need to be counterrotated by the robot heading. The IMU returns heading, however we need to rotate the movement counter to the robot's rotation, so its negative is taken. The joystick values are a vector, and rotating a vector in 2D requires this formula ([proved here²⁰⁰](https://matthew-brett.github.io/teaching/rotation_2d.html)), where x_1 and y_1 are the components of the original vector, β is the angle to rotate by, and x_2 and y_2 are the components of the resultant vector.

$$x_2 = x_1 \cos \beta - y_1 \sin \beta$$

$$y_2 = x_1 \sin \beta + y_1 \cos \beta$$

```
// Rotate the movement direction counter to the bot's rotation
double rotX = x * Math.cos(-botHeading) - y * Math.sin(-botHeading);
double rotY = x * Math.sin(-botHeading) + y * Math.cos(-botHeading);
```

Then, these rotated values can be put into the mecanum kinematics shown earlier.

```
double denominator = Math.max(Math.abs(rotY) + Math.abs(rotX) + Math.abs(rx), 1);
double frontLeftPower = (rotY + rotX + rx) / denominator;
double backLeftPower = (rotY - rotX + rx) / denominator;
double frontRightPower = (rotY - rotX - rx) / denominator;
double backRightPower = (rotY + rotX - rx) / denominator;
```

²⁰⁰ https://matthew-brett.github.io/teaching/rotation_2d.html

Field-Centric Final Sample Code

```

package org.firstinspires.ftc.teamcode;

import com.qualcomm.robotcore.eventloop.opmode.LinearOpMode;
import com.qualcomm.robotcore.hardware.IMU;
import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
import com.qualcomm.robotcore.hardware.DcMotor;
import com.qualcomm.robotcore.hardware.DcMotorSimple;
import com.qualcomm.hardware.rev.RevHubOrientationOnRobot;
import org.firstinspires.ftc.robotcore.external.navigation.AngleUnit;

@TeleOp
public class FieldCentricMecanumTeleOp extends LinearOpMode {
    @Override
    public void runOpMode() throws InterruptedException {
        // Declare our motors
        // Make sure your ID's match your configuration
        DcMotor frontLeftMotor = hardwareMap.dcMotor.get("frontLeftMotor");
        DcMotor backLeftMotor = hardwareMap.dcMotor.get("backLeftMotor");
        DcMotor frontRightMotor = hardwareMap.dcMotor.get("frontRightMotor");
        DcMotor backRightMotor = hardwareMap.dcMotor.get("backRightMotor");

        // Reverse the right side motors. This may be wrong for your setup.
        // If your robot moves backwards when commanded to go forwards,
        // reverse the left side instead.
        // See the note about this earlier on this page.
        frontRightMotor.setDirection(DcMotorSimple.Direction.REVERSE);
        backRightMotor.setDirection(DcMotorSimple.Direction.REVERSE);

        // Retrieve the IMU from the hardware map
        IMU imu = hardwareMap.get(IMU.class, "imu");
        // Adjust the orientation parameters to match your robot
        IMU.Parameters parameters = new IMU.Parameters(new RevHubOrientationOnRobot(
            RevHubOrientationOnRobot.LogoFacingDirection.UP,
            RevHubOrientationOnRobot.UsbFacingDirection.FORWARD));
        // Without this, the REV Hub's orientation is assumed to be logo up / USB
        ↪ forward
        imu.initialize(parameters);

        waitForStart();

        if (isStopRequested()) return;

        while (opModeIsActive()) {
            double y = -gamepad1.left_stick_y; // Remember, Y stick value is reversed
            double x = gamepad1.left_stick_x;
            double rx = gamepad1.right_stick_x;

            // This button choice was made so that it is hard to hit on accident,
            // it can be freely changed based on preference.
            // The equivalent button is start on Xbox-style controllers.
            if (gamepad1.options) {
                imu.resetYaw();
            }

            double botHeading = imu.getRobotYawPitchRollAngles().getYaw(AngleUnit.

```

(continues on next page)

(continued from previous page)

```

↪RADIANS);

    // Rotate the movement direction counter to the bot's rotation
    double rotX = x * Math.cos(-botHeading) - y * Math.sin(-botHeading);
    double rotY = x * Math.sin(-botHeading) + y * Math.cos(-botHeading);

    rotX = rotX * 1.1; // Counteract imperfect strafing

    // Denominator is the largest motor power (absolute value) or 1
    // This ensures all the powers maintain the same ratio,
    // but only if at least one is out of the range [-1, 1]
    double denominator = Math.max(Math.abs(rotY) + Math.abs(rotX) + Math.
↪abs(rx), 1);
    double frontLeftPower = (rotY + rotX + rx) / denominator;
    double backLeftPower = (rotY - rotX + rx) / denominator;
    double frontRightPower = (rotY - rotX - rx) / denominator;
    double backRightPower = (rotY + rotX - rx) / denominator;

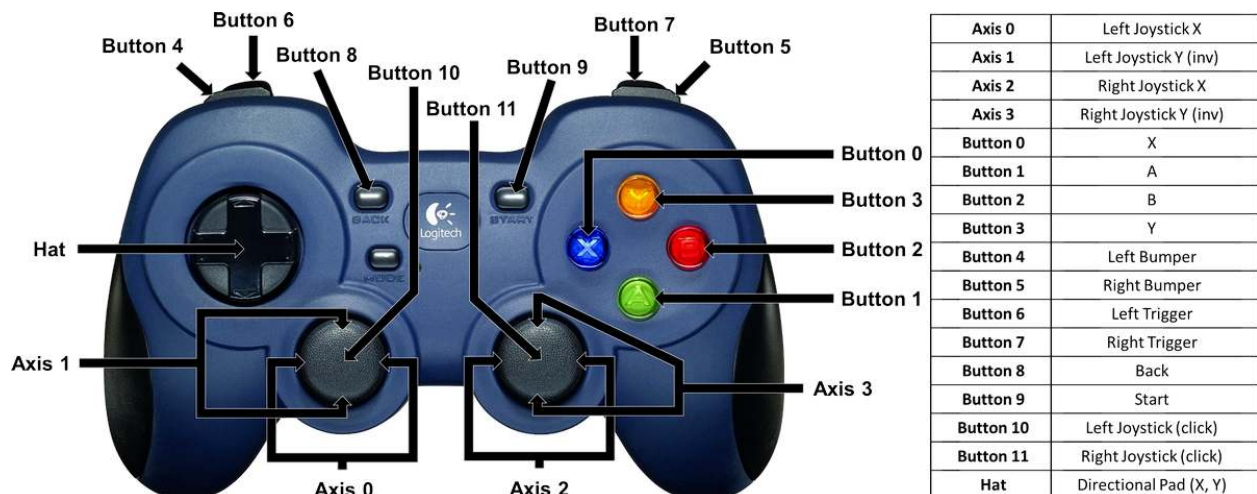
    frontLeftMotor.setPower(frontLeftPower);
    backLeftMotor.setPower(backLeftPower);
    frontRightMotor.setPower(frontRightPower);
    backRightMotor.setPower(backRightPower);
  }
}

```

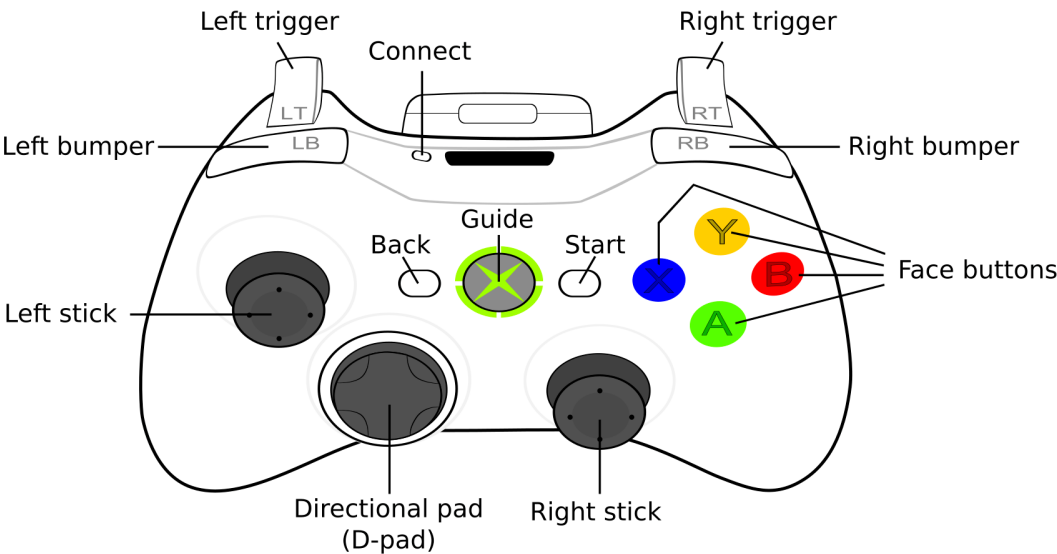
10.2.2 Gamepad Usage

Gamepad Layout

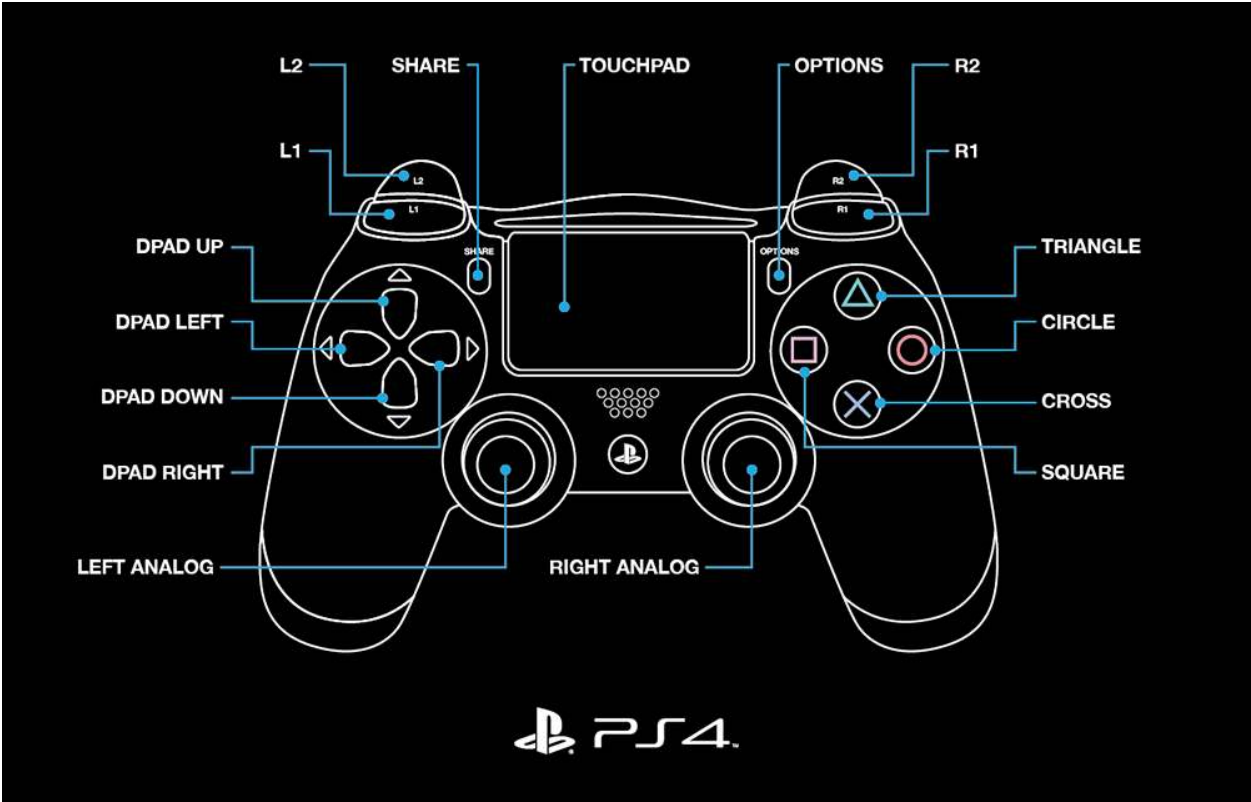
Logitech F310



Xbox 360



PS4/Etpark



Note

L1 is left_bumper, L2 is left_trigger, R1/R2 are the right-hand equivalents.

Button Aliases

Since both PS4-style and Xbox-Style controllers are FTC® legal, there are [aliases in the FTC SDK²⁰¹](#) between PS4-style and Xbox-style button naming.

PS4	Xbox
circle	b
cross	a
triangle	y
square	x
share	back
options	start
ps	guide

Boolean Inputs

TeleOp OpModes are generally written in iterative style, with a loop that contains code that is called over and over. Under this paradigm, a simple handling of user input could look like

```
if (gamepad1.a) {
    motor.setPower(1);
}
else {
    motor.setPower(0);
}
```

In this situation, this likely does what the driver wants: while the button is held, the motor's power is set to 1, and otherwise the power is set to 0. Since writing the same power to a motor multiple times has no effect on the motor's behavior, this works perfectly fine. However, issues arise when wanting to do something once when a button is pressed. For example, it is tempting to write something like this to get a press of a or b to adjust a servo.

```
if (gamepad1.a) {
    servo.setPosition(servo.getPosition()+0.1);
}
else if (gamepad1.b) {
    servo.setPosition(servo.getPosition()-0.1);
}
```

However, this will behave unpredictably, as each time the button is pressed, the setPosition method will be called multiple times as the loop frequency changes as does the button press length. There are a few techniques to avoid this, however they all require comparing the gamepad state to the gamepad state in the previous loop; therefore, it is necessary to store it.

²⁰¹ <https://github.com/OpenFTC/Extracted-RC/blob/c960dd7de34d49a66c00a345636175392f936b9e/RobotCore/src/main/java/com/qualcomm/robotcore/hardware/Gamepad.java#L892>

Storing Gamepad State

While each gamepad input's previous state could be stored individually in a variable, e.g. `boolean previousA`, this very quickly gets annoying. Luckily, the FTC SDK provides a way to copy gamepad states, with `gamepad.copy(gamepadToCopy)`.

Note

In addition to storing the gamepad state for the previous iteration of the loop, the gamepad state for the current iteration of the loop is also stored. This is necessary because if the state of a button was always read from `gamepad1/gamepad2`, it could change between reading the value and storing the previous value. This is because `gamepad1/gamepad2` update concurrently for `LinearOpMode`, and so can change during a loop iteration.

In a `LinearOpMode` based TeleOp program, storing both current and previous gamepad state could look like:

```
public void runOpMode() {
    // By setting these values to new Gamepad(), they will default to all
    // boolean values as false and all float values as 0
    Gamepad currentGamepad1 = new Gamepad();
    Gamepad currentGamepad2 = new Gamepad();

    Gamepad previousGamepad1 = new Gamepad();
    Gamepad previousGamepad2 = new Gamepad();

    // other initialization code goes here

    while (opModeIsActive()) {
        // Store the gamepad values from the previous loop iteration in
        // previousGamepad1/2 to be used in this loop iteration.
        // This is equivalent to doing this at the end of the previous
        // loop iteration, as it will run in the same order except for
        // the first/last iteration of the loop.
        previousGamepad1.copy(currentGamepad1);
        previousGamepad2.copy(currentGamepad2);

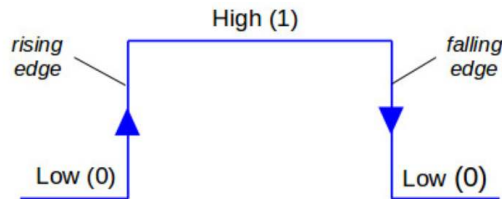
        // Store the gamepad values from this loop iteration in
        // currentGamepad1/2 to be used for the entirety of this loop iteration.
        // This prevents the gamepad values from changing between being
        // used and stored in previousGamepad1/2.
        currentGamepad1.copy(gamepad1);
        currentGamepad2.copy(gamepad2);

        // Main teleop loop goes here
    }
}
```

Rising Edge Detector

Why is it called a rising edge detector?

A signal edge is a transition in a digital signal. In this case, the digital signal is the gamepad input, which is low when not held and high when held. The rising edge is the transition from low to high, and the falling edge is the transition from high to low.



The most commonly used technique is a rising edge detector. It allows code to be run only once when the button is initially pressed, as opposed to every loop while it is held down. It works by checking that the button is currently pressed, but was not pressed in the previous loop. For example, inside of a TeleOp loop:

```
if (currentGamepad1.a && !previousGamepad1.a) {  
    servo.setPosition(servo.getPosition() + 0.1);  
}
```

This will increase the servo position by 0.1 exactly once per press of a.

Falling Edge Detector

A very similar technique is a falling edge detector. It allows code to be run only once when the button is released, as opposed to every loop while it is held down. It works by checking that the button is currently not pressed, but was pressed in the previous loop. For example, inside of a TeleOp loop:

```
if (!currentGamepad1.b && previousGamepad1.b) {  
    servo.setPosition(servo.getPosition() - 0.1);  
}
```

This will decrease the servo position by 0.1 exactly once per release of b.

Note

One button can run different code on the rising and falling edge. This is mainly useful for fairly complex interactions and so is not demonstrated here.

Toggles

One common use case for rising edge detectors is to control toggles. Toggles can be used to have a button for the robot to switch between states; for example, turning an intake on and off. This can be done for any number of states but is most commonly done between two. To make a toggle between two states, a rising edge detector is used to set a boolean to its opposite and then that boolean is used to control an action.

Example

Within the initialization code:

```
boolean intakeToggle = false;
```

Inside of the corresponding TeleOp loop:

```
// Rising edge detector
if (currentGamepad1.a && !previousGamepad1.a) {
    // This will set intakeToggle to true if it was previously false
    // and intakeToggle to false if it was previously true,
    // providing a toggling behavior.
    intakeToggle = !intakeToggle;
}

// Using the toggle variable to control the robot.
if (intakeToggle) {
    intakeMotor.setPower(1);
}
else {
    intakeMotor.setPower(0);
}
```

This will turn on the intake when a is pressed, and leave it on until it is pressed again.

Note

The less a driver has to keep in their head about the state of the robot the less they can screw up. Since toggles mean that a button does different things every time it is pressed, consider alternate solutions. This is especially true for toggles with more than two states.

Gamepad Feedback

Gamepad feedback (i.e. rumble and LED control) can be a helpful way for robots to communicate status to a driver during a match. The degree to which the legal gamepads support this functionality varies:

Logitech F310

- Rumble: none
- LED Control: none

Xbox 360

- Rumble: large (whomp whomp) and small (bzzz)
- LED Control: none

DualShock4 (PS4)

- Rumble: large (whomp whomp) and small (bzzz)
- LED Control: control of RGB lightbar (solid color or pattern)

EtPark

- Rumble: contains both left and right rumble motors, but both seem to be only small weight (bzzz)
- LED Control: control of RGB LED (solid color or pattern). LED is fairly small and dim and may not be a good choice.

Tip

Gamepad feedback can be used to alert drivers of: start of endgame, intake loaded, automatic alignment complete, etc.

Rumble

The SDK provides both a simple and more complex API for controlling rumble according to the desired use case.

Note

- Rumble power is specified as a floating-point value in the range [0.0, 1.0].
- Rumble duration is specified in milliseconds as an integer. The constant `Gamepad.RUMBLE_DURATION_CONTINUOUS` may be used to indicate that the rumble should continue until another rumble action is commanded.

Note

All rumble actions are completed asynchronously; i.e. the function calls will return immediately. Any call to a rumble API will immediately displace any currently running rumble action. That is, if you command

a gamepad to rumble for 750ms and then immediately command a rumble for 250ms, the gamepad will rumble for 250ms from the time the second command was issued.

Simple API

The simplest way to command rumble: rumble motor 1 at 100% power for a specified duration:

```
gamepad1.rumble(int durationMs);
```

If control over both rumble motors and rumble intensity is desired:

```
gamepad1.rumble(double rumble1, double rumble2, int durationMs);
```

To make a gamepad rumble for a certain number of “blips” (the notion of what a “blip” is being predefined by the SDK):

```
gamepad1.rumbleBlips(int numBlips);
```

Helper functions:

The public boolean `isRumbling()` function provides an educated guess about whether there is a rumble action ongoing on this gamepad. The Robot Controller does not know for sure whether a rumble action is ongoing or not, because once the command is sent to the Driver Station, the Driver Station handles running the gamepad effects and the Robot Controller is “hands off”.

The public void `stopRumble()` function may be used to stop any ongoing rumble action for a gamepad (perhaps most useful in conjunction with a rumble of continuous duration).

Advanced API

To create more advanced rumble behavior, a `RumbleEffect` may be created, which is composed of “Steps” which specify the power and duration each rumble motor should operate at. When a gamepad is commanded to run a `RumbleEffect`, it will perform each of the “Steps” in series.

To create a `RumbleEffect`, the `RumbleEffect.Builder` class must be used. The builder provides the `addStep(double rumble1, double rumble2, int durationMs)` function which is used to add a step to the sequence, and the `build()` function to create a `RumbleEffect` from the sequence of steps.

Using an anonymous instance of the builder class is the cleanest way to construct a `RumbleEffect`, for example:

```
Gamepad.RumbleEffect effect = new Gamepad.RumbleEffect.Builder()
    .addStep(0.0, 1.0, 500) // Rumble right motor 100% for 500 mSec
    .addStep(0.0, 0.0, 300) // Pause for 300 mSec
    .addStep(1.0, 0.0, 250) // Rumble left motor 100% for 250 mSec
    .addStep(0.0, 0.0, 250) // Pause for 250 mSec
    .addStep(1.0, 0.0, 250) // Rumble left motor 100% for 250 mSec
    .build();
```

Once a `RumbleEffect` has been created, it can be sent to a gamepad by calling:

```
gamepad1.runRumbleEffect(effect);
```

LED Control

Note

- RGB LED component (i.e. red, green, blue) intensity is specified as a floating-point value in the range [0.0, 1.0].
- LED duration is specified in milliseconds as an integer. The constant `Gamepad.LED_DURATION_CONTINUOUS` may be used to indicate that the LED should remain the specified color until another command is issued.

Note

All LED actions are completed *asynchronously*; i.e. the function calls will return immediately. Any call to an LED API will immediately displace any currently running LED action. That is, if you command the LED green for 750ms and then immediately command purple for 250ms, the LED will light purple for 250ms from the time the second command was issued.

To set the LED color for a fixed duration:

```
gamepad1.setLedColor(double r, double g, double b, int durationMs);
```

To create more advanced LED behavior, an `LedEffect` may be created, which is composed of “Steps” which specify a color and the duration for which to maintain it. When a gamepad is commanded to run an `LedEffect`, it will perform each of the “Steps” in series.

To create an `LedEffect`, the `LedEffect.Builder` class must be used. The builder provides the `addStep(double r, double g, double b, int durationMs)` function which is used to add a step to the sequence, and the `build()` function to create an `LedEffect` from the sequence of steps.

Using an anonymous instance of the builder class is the cleanest way to construct an `LedEffect`, for example:

```
Gamepad.LedEffect rgbEffect = new Gamepad.LedEffect.Builder()  
    .addStep(1, 0, 0, 250) // Show red for 250ms  
    .addStep(0, 1, 0, 250) // Show green for 250ms  
    .addStep(0, 0, 1, 250) // Show blue for 250ms  
    .addStep(1, 1, 1, 250) // Show white for 250ms  
    .build();
```

Once an `LedEffect` has been created, it can be sent to a gamepad by calling:

```
gamepad1.runLedEffect(rgbEffect);
```

10.2.3 Using Telemetry

The SDK has “telemetry” that can be sent from the robot controller to the drivers station to display basic text. This telemetry is accessible through the Telemetry class, and the `telemetry` variable accessible in both `OpMode` and `LinearOpMode`.

Building Telemetry

There are two main methods used to add things to telemetry. `addData()` takes two parameters, a string for the caption, and then a value that can be any object. This is then printed on the phone screen in the format `caption : value`. These lines are keyed, so adding a secondary `addData()` with the same caption will override the value set.

```
telemetry.addData("Caption 1", 2.5);
telemetry.addData("Caption 2", "value");
```

The second method used to add things to telemetry is `addLine()`. Add line can be used to add a line with no parameters, or can take a single string to add as a line.

```
telemetry.addLine("This is a line!");
```

Updating Telemetry

Once all telemetry items have been added (typically at the end of an `opmode` loop), `telemetry.update()` must be called. Calling this method is what pushes the values to the phone, so without it telemetry will not appear on the phone screen. Note that `OpMode` will call `telemetry.update()` automatically once per call of its loop.

Tip

By default, telemetry is only refreshed on the phone every 250 ms. Any calls to `telemetry.update()` during this window will be saved and overwritten if `update()` is called again before 250 ms have elapsed. `setMsTransmissionInterval()` may be used to change the amount of time the SDK will wait between sending updates to the driver station.

10.2.4 Encoders

What Are Encoders?

Very commonly in FTC®, you want to know where something is. Whether that is how many times your drive-train wheel has rotated, what angle your arm is at, or how far your string slides have gone, rotational encoders can help you. In FTC, an encoder refers to any sensor that tracks the rotational angle of a mechanism.

There are two types of encoders commonly used in FTC, relative and absolute encoders. Relative encoders are the ones covered here as they are the more common type.

Relative Encoders

Ranging from the built in encoder in every FTC legal motor to common external encoders like the REV Through Bore encoder, these encoders track the relative position of the shaft or mechanism they are attached to. **What this means is that the position output is relative to the initial position at robot power on, meaning position information is lost between power cycles.**

Tip

Relative encoders don't reset to zero at the beginning of OpModes! You can use `STOP_AND_RESET_ENCODERS` to ensure that your encoders are always at zero at the beginning of an OpMode (see below).

All relative encoders in FTC use the “quadrature” protocol to send position information to the expansion hub. As a result, relative encoders must be plugged into the encoder ports located near the motor ports in order to function.

Terminology

Count: A “count” (sometimes referred to as a “tick”) refers to one increment of the encoder's position. Relative encoders report their position as a number equal to the number of “ticks” or “counts” the encoder has moved from its starting angle.

Counts Per Revolution: The number of “counts” that an encoder reports after it has gone exactly one revolution. This value is commonly used to convert encoder “counts” into degrees or revolutions.

Warning

Quadrature terminology can get very confusing! Some encoders may report “pulses per revolution.” One pulse can either equal 1 or 4 counts. Other encoders may report “cycles per revolution”, which confusingly have the same acronym as counts per revolution. The best way to check is to plug the encoder into the REV Hub and turn it 1 full revolution, then check what it reports.

Programming Encoders

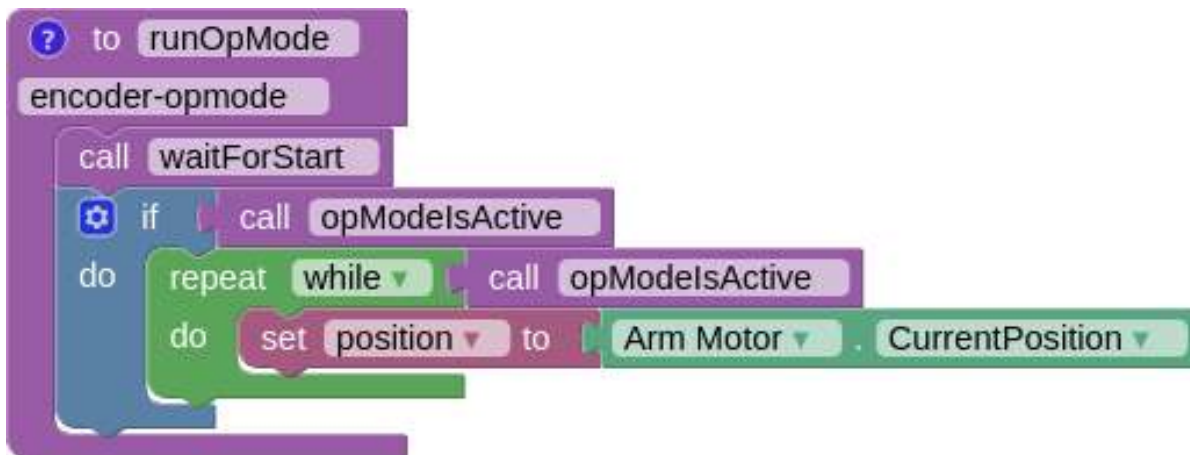
Reading Encoders

In FTC software, quadrature encoders and motors are accessed with the same motor object. What this means is that we can access an encoder's position like so:

Java

```
int position = motor.getCurrentPosition();
```

Blocks

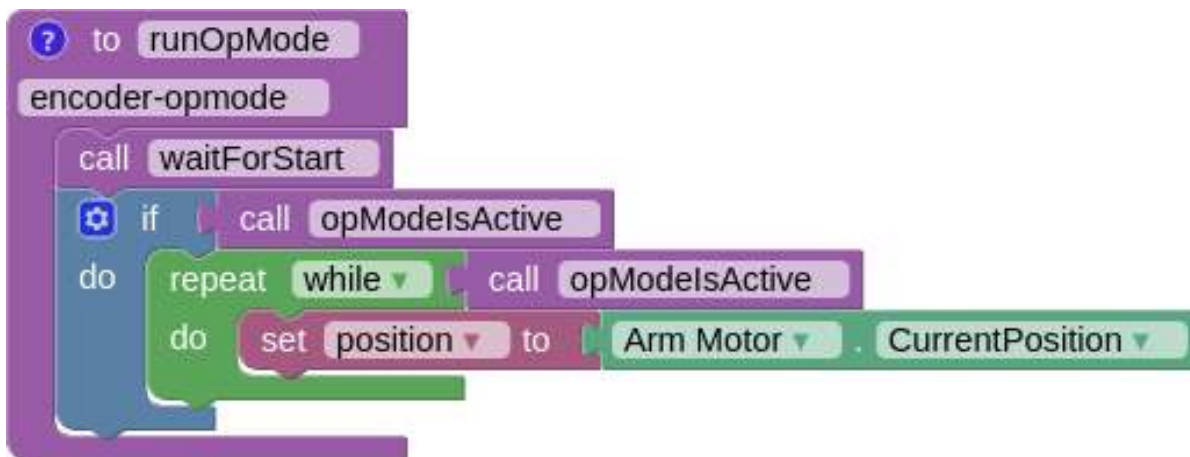


While convenient if one uses the built-in motor encoder, this can easily become confusing if using external encoders. To use external encoders, you must use the motor object associated with the port. For example, if there is a motor in port 1 named "Arm Motor" and an external encoder plugged into encoder port 1, you must do the following to get the encoder's position:

Java

```
DcMotor motor = hardwareMap.dcMotor.get("Arm Motor");
double position = motor.getCurrentPosition();
```

Blocks

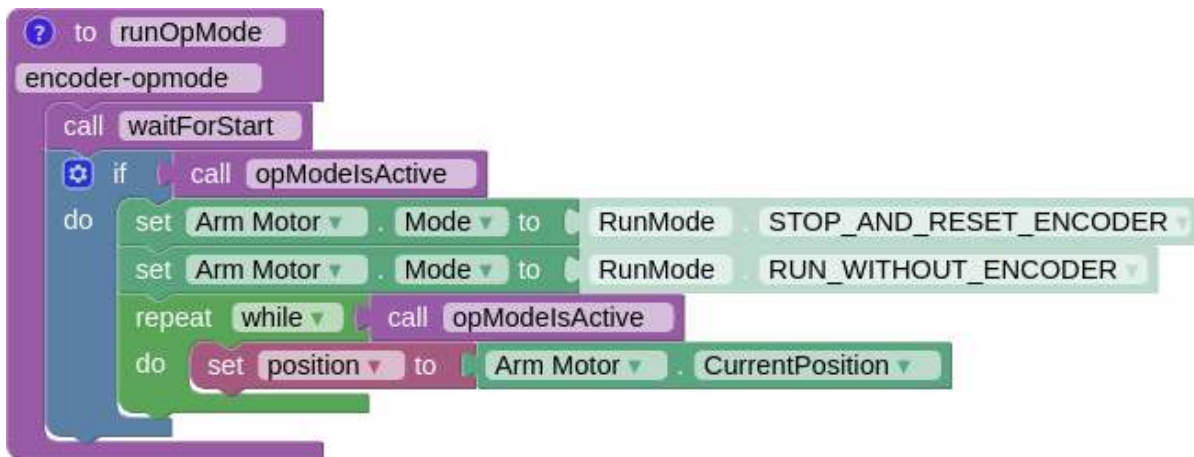


Great! We now have the relative position of our encoder, reported in the number of “counts” it is from what it considers to be zero. However, it is often convenient to have the encoder start at zero at the beginning of the OpMode. While it technically does not change anything, it can help with debugging and simplify future code. To do this, we can add a call to reset the encoders before we read them.

Java

```
DcMotor motor = hardwareMap.dcMotor.get("Arm Motor");
motor.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER); // Reset the motor encoder
motor.setMode(DcMotor.RunMode.RUN_WITHOUT_ENCODER); // Turn the motor back on when we
are done
int position = motor.getCurrentPosition();
```

Blocks



As a note, **RUN_WITHOUT_ENCODER does not disable the encoder. It instead tells the SDK not to use the motor encoder for built-in velocity control.** We will go over what this means in a later section, but for now, know that it turns the motor back on so we can use it after the encoder is reset.

Now we have our position (in counts) relative to the starting angle of the encoder. We can make a quick program to see this in action. Here, we use a motor encoder plugged into a port named “Arm Motor” in the hardware configuration.

Java

```
package org.firstinspires.ftc.teamcode;

import com.qualcomm.robotcore.eventloop.opmode.LinearOpMode;
import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
import com.qualcomm.robotcore.hardware.DcMotor;
@TeleOp
public class EncoderOpmode extends LinearOpMode {
    @Override
    public void runOpMode() throws InterruptedException {
        // Find a motor in the hardware map named "Arm Motor"
```

(continues on next page)

(continued from previous page)

```

DcMotor motor = hardwareMap.dcMotor.get("Arm Motor");

// Reset the motor encoder so that it reads zero ticks
motor.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);

// Turn the motor back on, required if you use STOP_AND_RESET_ENCODER
motor.setMode(DcMotor.RunMode.RUN_WITHOUT_ENCODER);

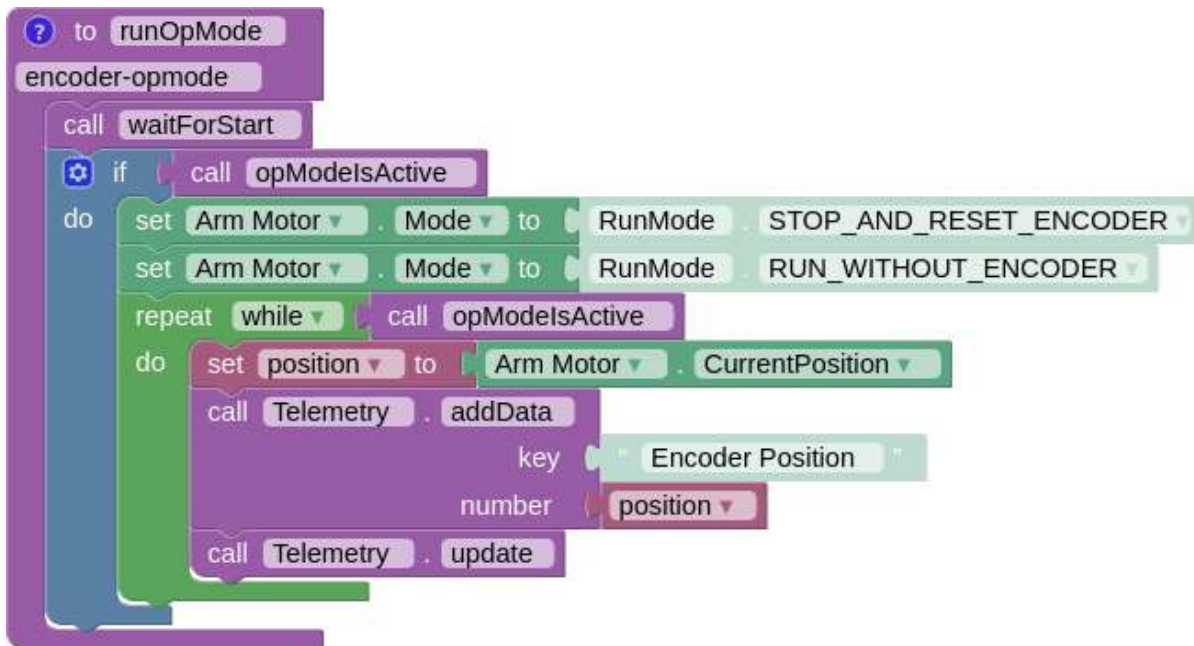
waitForStart();

while (opModeIsActive()) {
    // Get the current position of the motor
    int position = motor.getCurrentPosition();

    // Show the position of the motor on telemetry
    telemetry.addData("Encoder Position", position);
    telemetry.update();
}
}

```

Blocks



If you run the above OpMode and turn the encoder, you should see the values change as you move. If you rotate the shaft back to where it started, you will see the number return to (very close to) zero. As an exercise, rotate the shaft one full revolution (360) degrees and note down the number.

There is one more thing we can do with encoders. While knowing the number of counts something moved is useful, oftentimes one will need a different number, like the number of revolutions the encoder has turned or the angle it is at. To determine these, we need a constant, the encoders Counts Per Revolution or CPR. For external encoders, this number is often provided in a datasheet. For motors, it will generally be on the product page, although some motors (most notably the Rev Ultraplanetary Gearbox) do not provide it plainly.

Tip

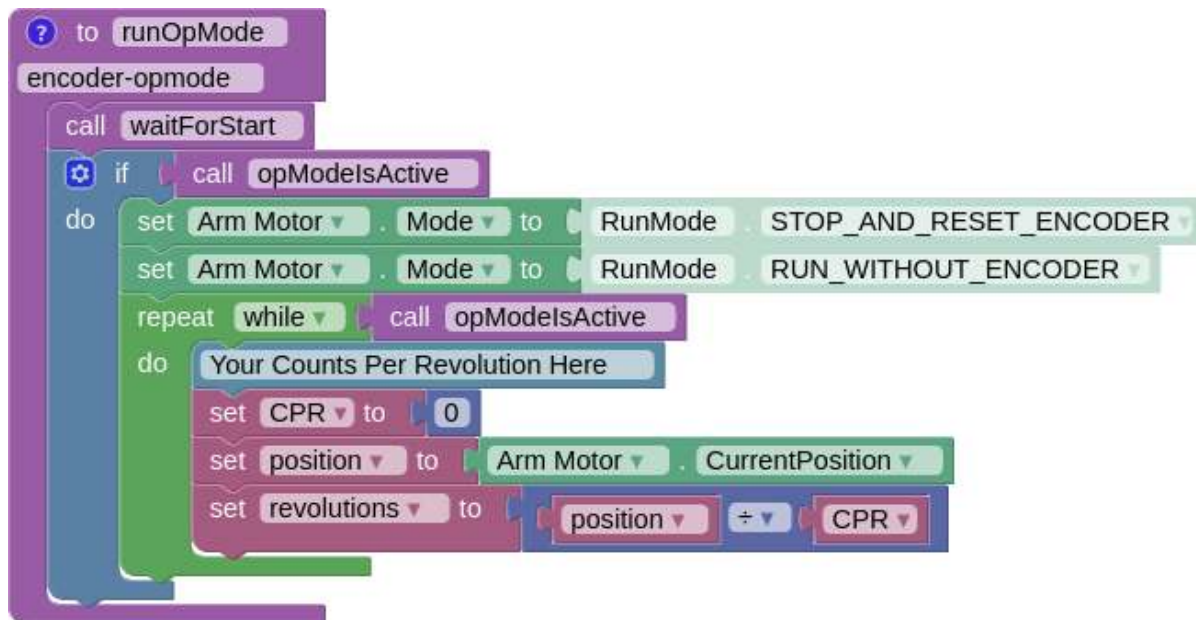
You can calculate a motor's Counts Per Revolution by taking the base motor's Counts Per Revolution and multiplying it by the gearbox ratio. Be careful to use the actual gearbox ratio when doing this! For example, a 5:1 Ultraplanetary motor would have a count per revolution of $28 * (5.23) = 146.44$ because the base motor has 28 Counts Per Revolution and the 5:1 gearbox actually has a 5.23:1 gear ratio. Remember, when using two gearboxes on top of each other, you multiply the gear ratios together.

In the following example, we divide the encoder position by its counts per revolution to obtain the number of revolutions the encoder has rotated. You have to replace [Your Counts Per Revolution Here] with the counts per revolution of your motor, found on its product page or calculated using the above tip.

Java

```
double CPR = [Your Counts Per Revolution Here];

int position = motor.getCurrentPosition();
double revolutions = position/CPR;
```

Blocks

There is one more number we can get: the angle of the shaft. Calculating this number is very simple. We can multiply the number of rotations by 360 (since there are 360 degrees in one revolution). You might notice that this number can go above 360 as the shaft rotates multiple times. As such, we introduce `angleNormalized`, which will always be between 0 and 360.

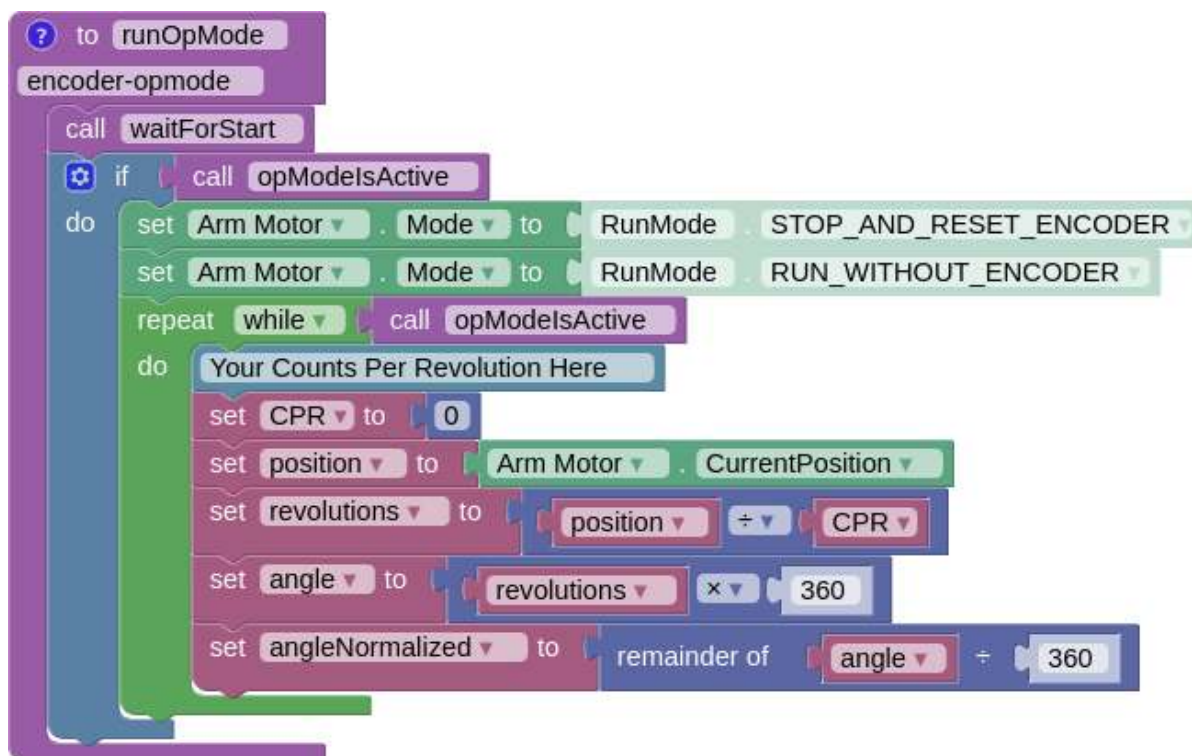
Java

```
double CPR = [Your Counts Per Revolution Here];

int position = motor.getCurrentPosition();
double revolutions = position/CPR;

double angle = revolutions * 360;
double angleNormalized = angle % 360;
```

Blocks



Putting it all together, we get the following testing program.

Java

```
package org.firstinspires.ftc.teamcode;

import com.qualcomm.robotcore.eventloop.opmode.LinearOpMode;
import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
import com.qualcomm.robotcore.hardware.DcMotor;
@TeleOp
public class EncoderOpmode extends LinearOpMode {
    @Override
    public void runOpMode() throws InterruptedException {
        // Find a motor in the hardware map named "Arm Motor"
        DcMotor motor = hardwareMap.dcMotor.get("Arm Motor");
```

(continues on next page)

(continued from previous page)

```

// Reset the motor encoder so that it reads zero ticks
motor.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);

// Turn the motor back on, required if you use STOP_AND_RESET_ENCODER
motor.setMode(DcMotor.RunMode.RUN_WITHOUT_ENCODER);

waitForStart();

while (opModeIsActive()) {
    double CPR = [Your Counts Per Revolution Here];

    // Get the current position of the motor
    int position = motor.getCurrentPosition();
    double revolutions = position/CPR;

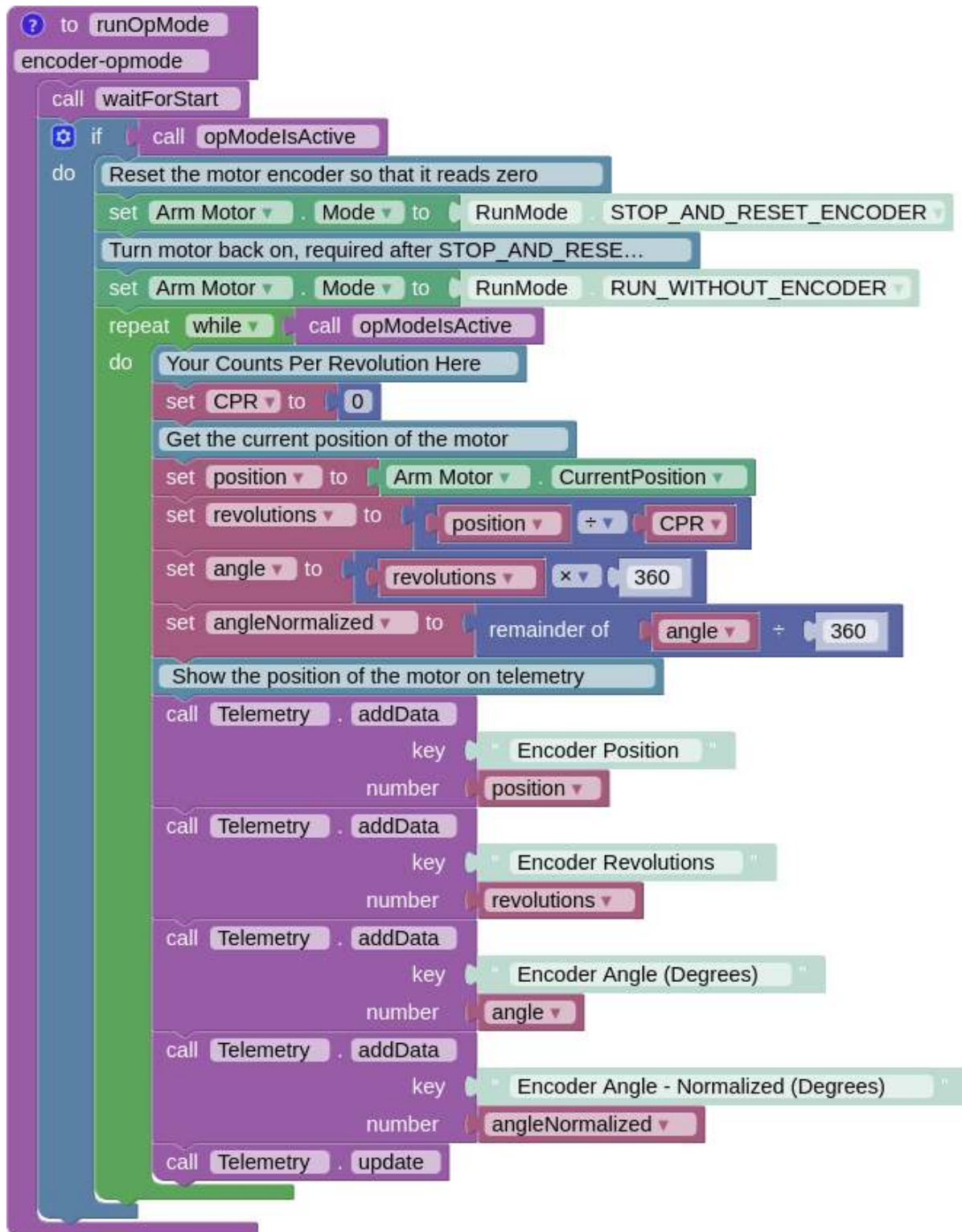
    double angle = revolutions * 360;
    double angleNormalized = angle % 360;

    // Show the position of the motor on telemetry
    telemetry.addData("Encoder Position", position);
    telemetry.addData("Encoder Revolutions", revolutions);
    telemetry.addData("Encoder Angle (Degrees)", angle);
    telemetry.addData("Encoder Angle - Normalized (Degrees)",
↵angleNormalized);
    telemetry.update();
}
}
}

```

Blocks

Blocks file download



Tracking Wheels and Spools

Up to this point, we have mostly been working with motors rotating something. However, many mechanisms in FTC are linear, and it can be desirable to measure these in a linear unit as well. Fortunately, this is very straightforward. All we need to know is the diameter of the object we are measuring.

Be careful when selecting your diameter. For wheels, it is the outer diameter of the wheel, but for spools, it is the inner diameter of the spool, where the string rests. For chain and belts, it is the “pitch diameter” of the sprocket or pulley.

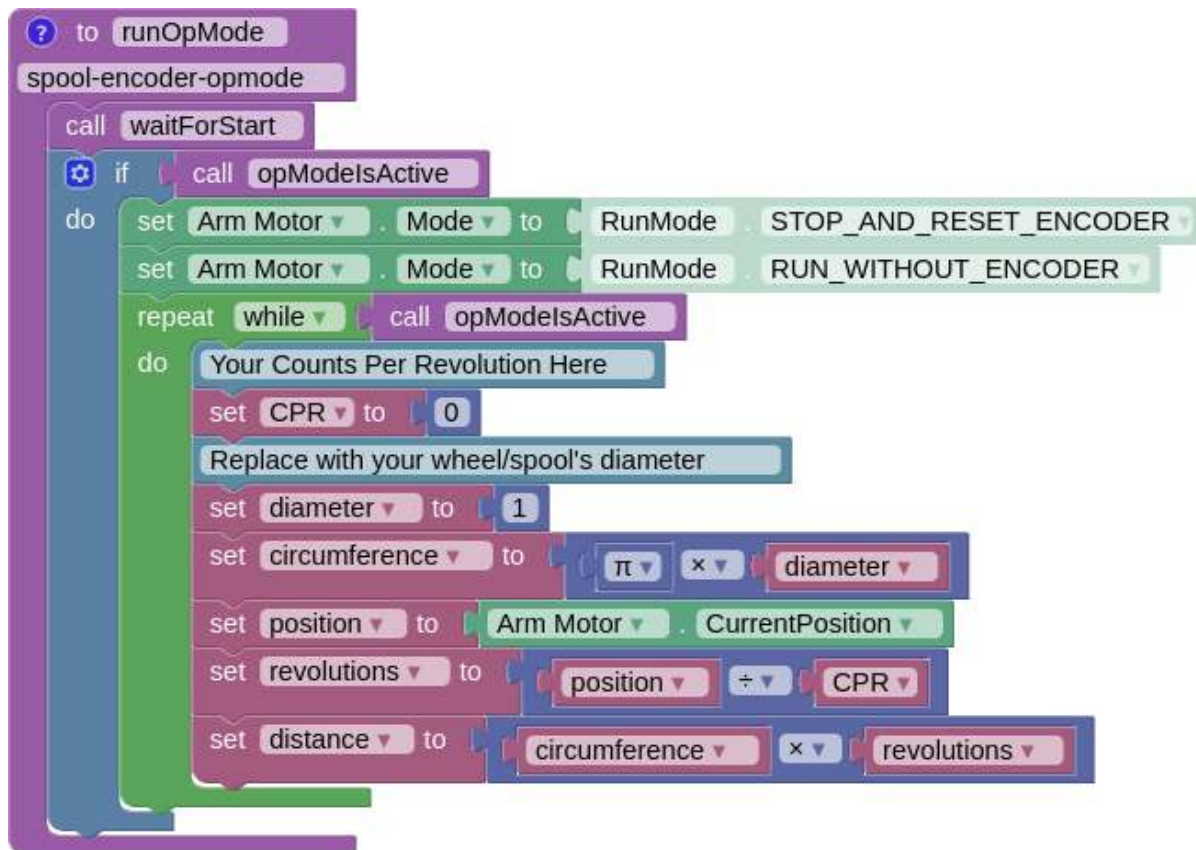
From here, we can calculate the circumference (the length of the arc of the circle, or the distance the wheel/spool will travel in one rotation)

Java

```
double diameter = 1.0; // Replace with your wheel/spool's diameter
double circumference = Math.PI * diameter;

double distance = circumference * revolutions;
```

Blocks



Note

Units are very important when dealing with FTC programming, so make sure they are always consistent! Whatever units you use for the diameter are the units for your calculated distance. So if you measure your diameter in inches, the reported distance will also be in inches.

Putting this all together with our previous testing program, we get

Java

```
package org.firstinspires.ftc.teamcode;

import com.qualcomm.robotcore.eventloop.opmode.LinearOpMode;
import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
import com.qualcomm.robotcore.hardware.DcMotor;
@TeleOp
public class SpoolEncoderOpmode extends LinearOpMode {
    @Override
    public void runOpMode() throws InterruptedException {
        // Find a motor in the hardware map named "Arm Motor"
        DcMotor motor = hardwareMap.dcMotor.get("Arm Motor");

        // Reset the motor encoder so that it reads zero ticks
        motor.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);

        // Turn the motor back on, required if you use STOP_AND_RESET_ENCODER
        motor.setMode(DcMotor.RunMode.RUN_WITHOUT_ENCODER);

        waitForStart();

        while (opModeIsActive()) {
            double CPR = [Your Counts Per Revolution Here];

            double diameter = 1.0; // Replace with your wheel/spool's diameter
            double circumference = Math.PI * diameter;

            // Get the current position of the motor
            int position = motor.getCurrentPosition();
            double revolutions = position/CPR;

            double angle = revolutions * 360;
            double angleNormalized = angle % 360;

            double distance = circumference * revolutions;

            //Show the position of the motor on telemetry
            telemetry.addData("Encoder Position", position);
            telemetry.addData("Encoder Revolutions", revolutions);
            telemetry.addData("Encoder Angle (Degrees)", angle);
            telemetry.addData("Encoder Angle - Normalized (Degrees)", angleNormalized);
            telemetry.addData("Linear Distance", distance);
            telemetry.update();
        }
    }
}
```

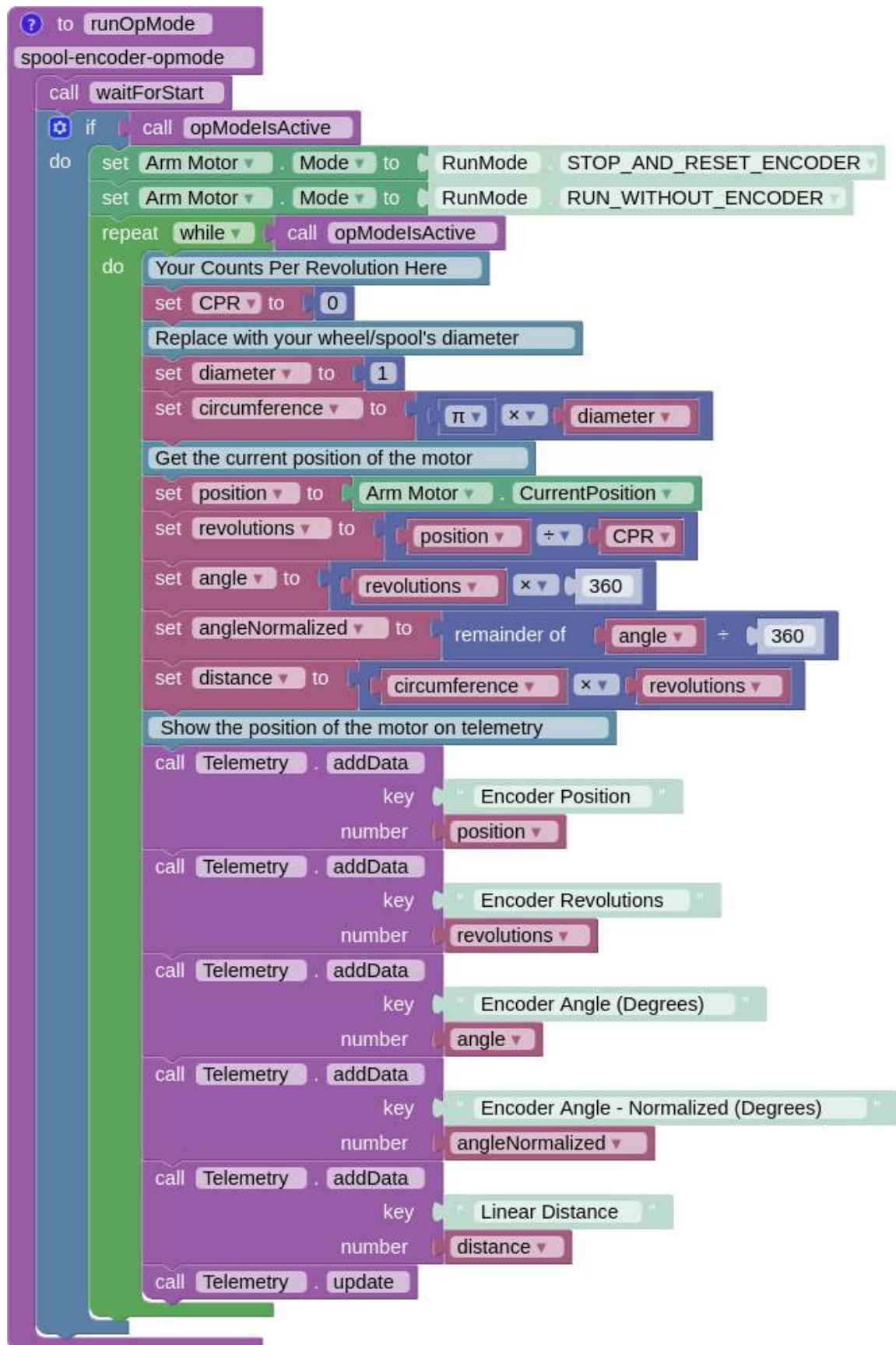
(continues on next page)

(continued from previous page)

```
}  
}
```

Blocks

Blocks file download



Running Motors With Encoders

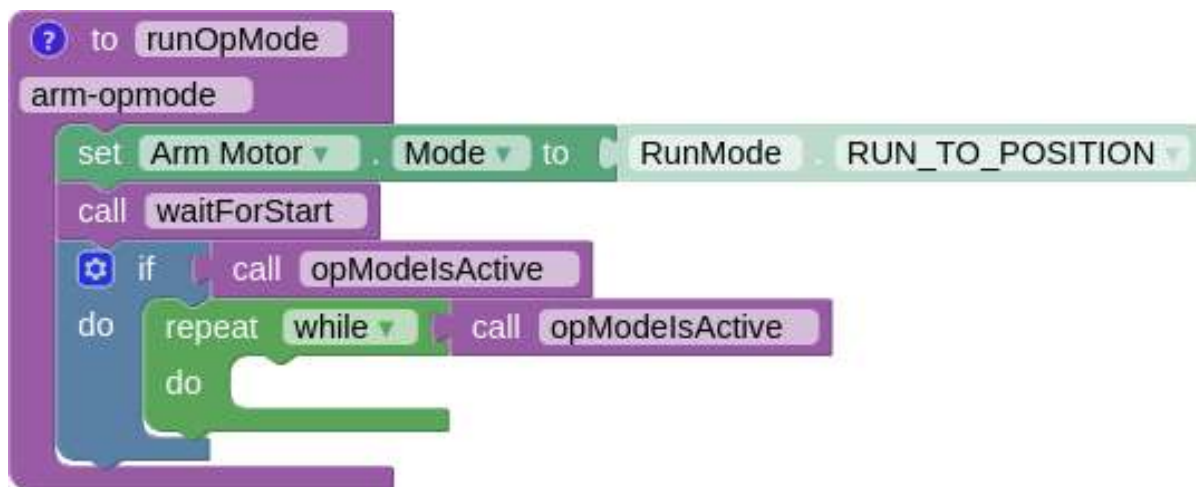
We've learned how to read encoder values, but how do you set where you want to go and tell the motor to go there?

Earlier, we learned about the `RUN_WITHOUT_ENCODER` mode for the motor. We can use another motor mode, `RUN_TO_POSITION`, to tell the motor to run to a specific position in ticks, like so:

Java

```
DcMotor motor = hardwareMap.dcmotor.get("Arm Motor");
motor.setMode(DcMotor.RunMode.RUN_TO_POSITION); // Tells the motor to run to the
↳ specific position
```

Blocks



Tip

You can find out more about run modes at the [official REV Robotics Documentation page](https://docs.revrobotics.com/duo-control/programming/using-encoder-feedback)²⁰²

However, before we tell the motor to go to a position, we have to tell the motor what position to run to. **Note that this value must be an integer.** Let's amend the above code to do that.

Warning

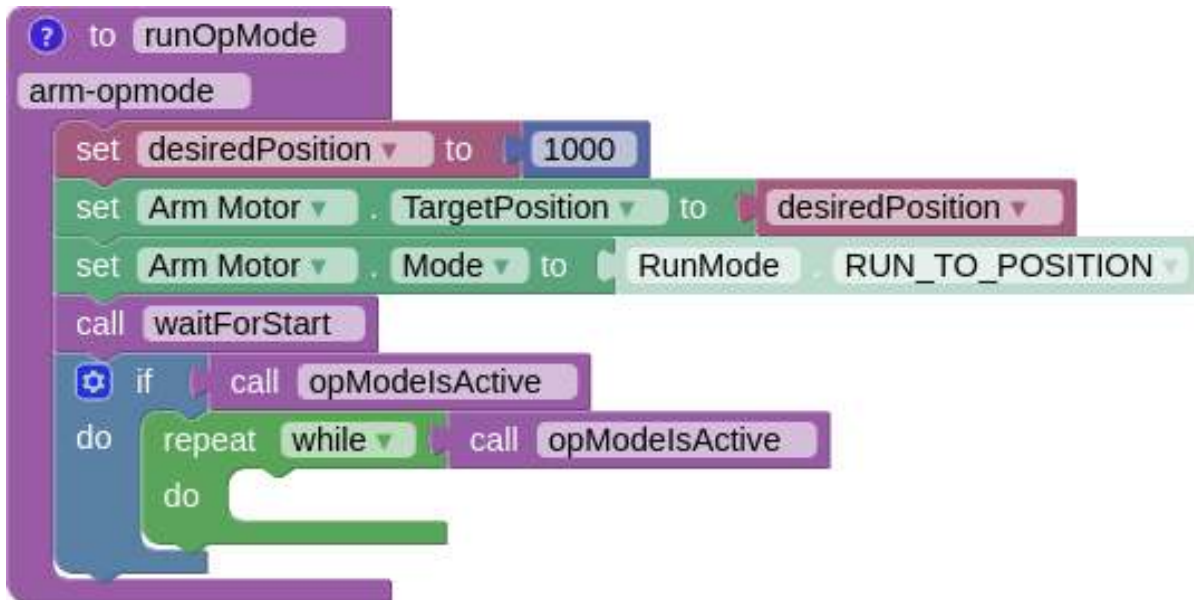
Setting the motor to `RUN_TO_POSITION` mode before setting a target position will throw an error. Be careful not to do that!

²⁰² <https://docs.revrobotics.com/duo-control/programming/using-encoder-feedback>

Java

```
DcMotor motor = hardwareMap.dcmotor.get("Arm Motor");
int desiredPosition = 1000; // The position (in ticks) that you want the motor to
    ↳ move to
motor.setTargetPosition(desiredPosition); // Tells the motor that the position it
    ↳ should go to is desiredPosition
motor.setMode(DcMotor.RunMode.RUN_TO_POSITION);
```

Blocks



This code tells the motor to move to 1000 ticks, using a PID loop to control the motor's position. You can read more about PID loops [here](https://gm0.org/en/latest/docs/software/concepts/control-loops.html#pid).²⁰³

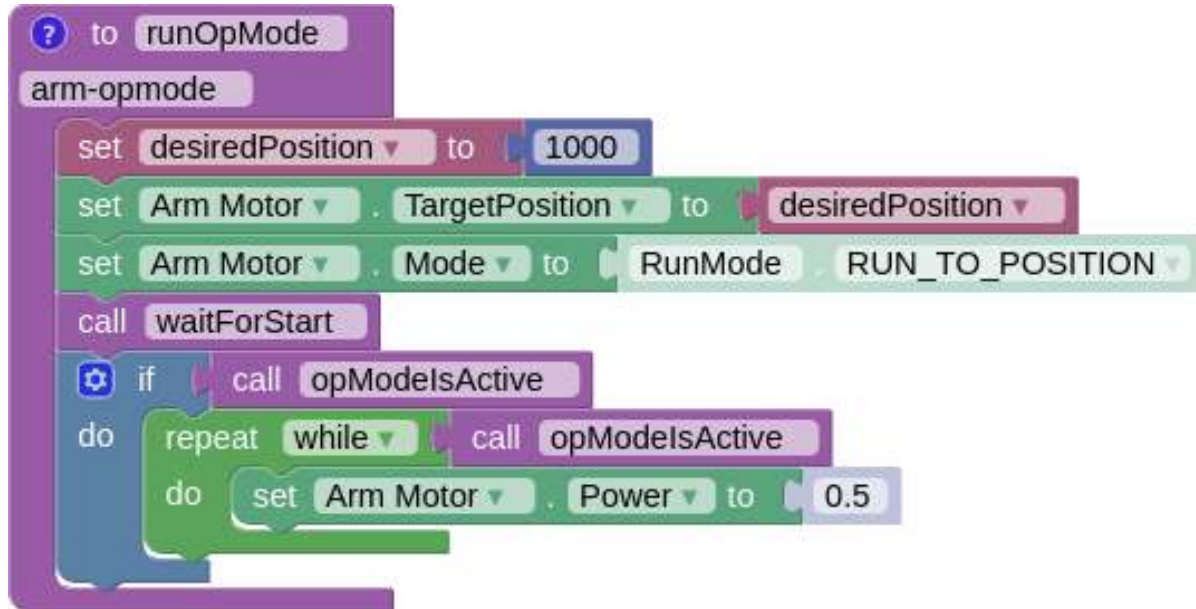
We can cap the speed that the motor runs at using the following code:

Java

```
DcMotor motor = hardwareMap.dcmotor.get("Arm Motor");
int desiredPosition = 1000; // The position (in ticks) that you want the motor to
    ↳ move to
motor.setTargetPosition(desiredPosition); // Tells the motor that the position it
    ↳ should go to is desiredPosition
motor.setMode(DcMotor.RunMode.RUN_TO_POSITION);
motor.setPower(0.5); // Sets the maximum power that the motor can go at
```

²⁰³ <https://gm0.org/en/latest/docs/software/concepts/control-loops.html#pid>

Blocks



Now, let's use this information to control an arm in an OpMode.

Java

```
package org.firstinspires.ftc.teamcode.Tests;

import com.qualcomm.robotcore.eventloop.opmode.LinearOpMode;
import com.qualcomm.robotcore.eventloop.opmode.TeleOp;
import com.qualcomm.robotcore.hardware.DcMotor;

@TeleOp
public class ArmOpMode extends LinearOpMode {
    @Override
    public void runOpMode() throws InterruptedException {
        // Position of the arm when it's lifted
        int armUpPosition = 1000;

        // Position of the arm when it's down
        int armDownPosition = 0;

        // Find a motor in the hardware map named "Arm Motor"
        DcMotor armMotor = hardwareMap.dcMotor.get("Arm Motor");

        // Reset the motor encoder so that it reads zero ticks
        armMotor.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);

        // Sets the starting position of the arm to the down position
        armMotor.setTargetPosition(armDownPosition);
        armMotor.setMode(DcMotor.RunMode.RUN_TO_POSITION);

        waitForStart();
    }
}
```

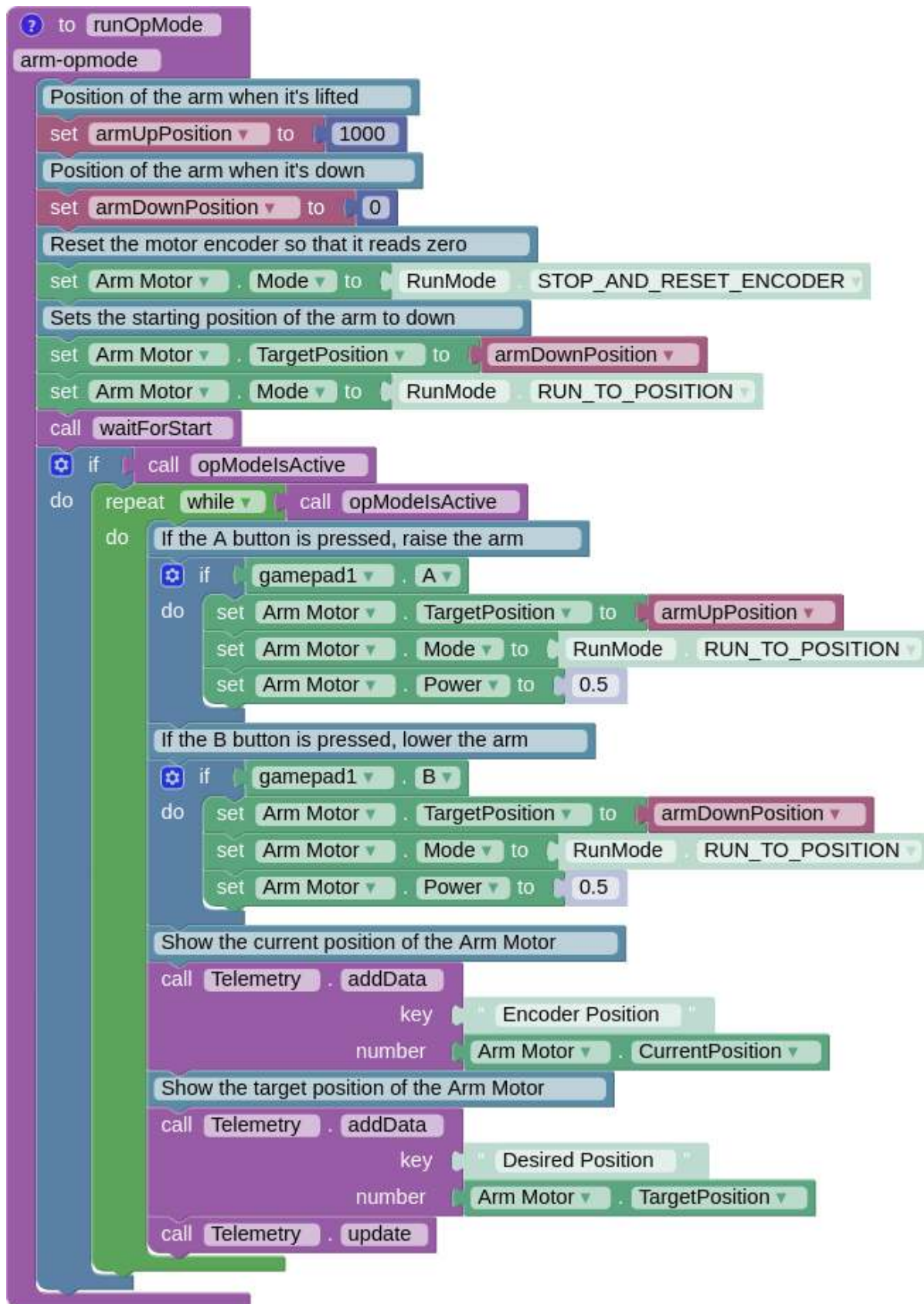
(continues on next page)

(continued from previous page)

```
while (opModeIsActive()) {  
    // If the A button is pressed, raise the arm  
    if (gamepad1.a) {  
        armMotor.setTargetPosition(armUpPosition);  
        armMotor.setMode(DcMotor.RunMode.RUN_TO_POSITION);  
        armMotor.setPower(0.5);  
    }  
  
    // If the B button is pressed, lower the arm  
    if (gamepad1.b) {  
        armMotor.setTargetPosition(armDownPosition);  
        armMotor.setMode(DcMotor.RunMode.RUN_TO_POSITION);  
        armMotor.setPower(0.5);  
    }  
  
    // Get the current position of the armMotor  
    double position = armMotor.getCurrentPosition();  
  
    // Get the target position of the armMotor  
    double desiredPosition = armMotor.getTargetPosition();  
  
    // Show the position of the armMotor on telemetry  
    telemetry.addData("Encoder Position", position);  
  
    // Show the target position of the armMotor on telemetry  
    telemetry.addData("Desired Position", desiredPosition);  
  
    telemetry.update();  
}  
}
```

Blocks

Blocks file download



10.2.5 Bulk Reads

Bulk reads are a LynxCommand that reads all sensor values (except I2C) on a hub at once. This takes the same amount of time to execute as any other LynxCommand, and can therefore save a lot of time in the execution loop; with a bulk read, reading ten sensors takes as much time as reading one sensor (if they are not I2C and are on the same hub).

This became much simpler to do with SDK versions 5.4 and above, with a built-in way to easily access it. Here is [the official example](#)²⁰⁴ on how to use bulk reads.

Off Mode

This is the default, and the most boring; it means bulk reads are not used by the SDK when calling normal hardware-access methods.

Note

Bulk reads can still be accessed by calling the `LynxModule.getBulkInputData()` method, however if one wishes to use bulk reads (which we highly recommend) using AUTO or MANUAL modes is simpler.

To manually set OFF mode, you need to run

```
List<LynxModule> allHubs = hardwareMap.getAll(LynxModule.class);

for (LynxModule hub : allHubs) {
    hub.setBulkCachingMode(LynxModule.BulkCachingMode.OFF);
}
```

Auto Mode

This is the simplest mode to use that utilizes bulk reads; a new bulk read is done when a hardware read is repeated. As an example of this

```
List<LynxModule> allHubs = hardwareMap.getAll(LynxModule.class);

for (LynxModule hub : allHubs) {
    hub.setBulkCachingMode(LynxModule.BulkCachingMode.AUTO);
}

while (opModeIsActive()) {
    // Will run one bulk read per cycle; however, if e.g.
    // frontLeftMotor.getCurrentPosition() was called again,
    // a new bulk read would be issued
    int frontLeftEncoderPos = frontLeftMotor.getCurrentPosition();
    int frontRightEncoderPos = frontRightMotor.getCurrentPosition();
    int backLeftEncoderPos = backLeftMotor.getCurrentPosition();
    int backRightEncoderPos = backRightMotor.getCurrentPosition();
}
```

²⁰⁴ <https://github.com/first-tech-challenge/FtcRobotController/blob/master/FtcRobotController/src/main/java/org/firstinspires/ftc/robotcontroller/external/samples/ConceptMotorBulkRead.java>

However, this can be problematic, if the same hardware read is called more than once in a given loop; an example of this

```
List<LynxModule> allHubs = hardwareMap.getAll(LynxModule.class);

for (LynxModule hub : allHubs) {
    hub.setBulkCachingMode(LynxModule.BulkCachingMode.AUTO);
}

while (opModeIsActive()) {
    // Will run two bulk read per cycles,
    // as frontLeftMotor.getCurrentPosition() is called twice
    int frontLeftEncoderPos = frontLeftMotor.getCurrentPosition();
    int frontLeftEncoderPos2 = frontLeftMotor.getCurrentPosition();
}
```

Overall, this is recommended, as it is very unlikely to mess anything up and can give significant performance improvements for little effort. On the user side, one does not need to manually flush the bulk read cache; however, this means you lose some control.

Manual Mode

In manual mode the cache for bulk reads is only reset once manually reset. This can be useful, as it is the way to absolutely minimize extraneous reads, however if the cache is not reset, stale values will be returned. That being said, here's a proper implementation of MANUAL mode

```
List<LynxModule> allHubs = hardwareMap.getAll(LynxModule.class);

for (LynxModule hub : allHubs) {
    hub.setBulkCachingMode(LynxModule.BulkCachingMode.MANUAL);
}

while (opModeIsActive()) {
    // Will run one bulk read per cycle,
    // even as frontLeftMotor.getCurrentPosition() is called twice
    // because the caches are being handled manually and cleared
    // once a loop
    for (LynxModule hub : allHubs) {
        hub.clearBulkCache();
    }

    int frontLeftEncoderPos = frontLeftMotor.getCurrentPosition();
    int frontLeftEncoderPos2 = frontLeftMotor.getCurrentPosition();
}
```

Warning

When in MANUAL mode, if the cache is not cleared appropriately, stale values will be returned. For that reason, if you are not quite sure what you are doing, we recommend AUTO mode; while MANUAL mode can have some performance improvements if AUTO mode is not used optimally, it has less room for catastrophic error.

 Tip

Bulk reads are issued per hub, so you don't necessarily need to bulk read both hubs at the same time or every loop.

10.2.6 Computer Vision

Computer vision is the process of using computers to understand digital images, such as photographs and videos. Computer vision offers object detection which is commonly needed in FTC®.

Currently, there are 3 commonly used forms of computer vision: TensorFlow Lite, AprilTags, and OpenCV (via VisionPortal/EasyOpenCV).

Officially Supported Solutions

TensorFlow

TensorFlow is Google's machine learning technology, which can be trained to detect objects. The FTC SDK uses TensorFlow Lite, which is a lightweight version of Google's TensorFlow designed to run on mobile devices. Along with Android Studio and OnBot support, TensorFlow has block support which makes it a good choice for most teams.

Sample OpModes for TensorFlow being used for pixel detection (CENTERSTAGE) can be found [here](#)²⁰⁵ (Blocks), and a Java example can be found [here](#)²⁰⁶.

FIRST® has released a tool called FTC-ML to train your own TensorFlow Lite model for detecting custom objects. Details about FTC-ML can be found [on FTC Docs](#)²⁰⁷.

AprilTags

AprilTags detect specific low-resolution, black-and-white images that are placed on various parts of the field. AprilTags are a great way to detect specific parts of the fields at wide angles, distances, and in a variety of lighting conditions. AprilTags are supported in OnBot Java, Android Studio, and Blocks.

You can learn more about implementing AprilTags [on FTC Docs](#)²⁰⁸. Supported AprilTag images are available on this [pdf](#)²⁰⁹.

²⁰⁵ <https://github.com/FIRST-Tech-Challenge/FtcRobotController/wiki/Blocks-Sample-OpMode-for-TFOD>

²⁰⁶ <https://github.com/FIRST-Tech-Challenge/FtcRobotController/wiki/Java-Sample-OpMode-for-TFOD>

²⁰⁷ https://ftc-docs.firstinspires.org/ftc_ml/index.html

²⁰⁸ https://ftc-docs.firstinspires.org/en/latest/apriltag/vision_portal/apriltag_intro/apriltag-intro.html

²⁰⁹ <https://www.dotproduct3d.com/uploads/8/5/1/1/85115558/apriltags1-20.pdf>

VisionPortal

Introduced with FTC SDK v8.2, the VisionPortal API integrates the [EasyOpenCV²¹⁰](#) project into the FTC SDK. With VisionPortal, you can attach multiple VisionProcessors to a single camera. VisionProcessors are an easy way to create OpenCV pipelines, which allows for custom manipulation and processing to be applied to each incoming frame. VisionPortal is the most powerful form of computer vision, but it is also the hardest to use. Therefore, we can only recommend it to more advanced teams.

Additional Vision Resources

EasyOpenCV Simulator

EasyOpenCV Simulator is a straightforward way to test your pipelines directly on your computer. It supports Windows, macOS, and Linux, and simulates a portion of the FTC SDK structure including the VisionPortal API, allowing you to copy and paste pipelines.

You can find EasyOpenCV Simulator [here²¹¹](#).

FTC® Dashboard

FTC Dashboard runs a dashboard webpage on the Control Hub that, among other things, streams a live preview of a connected camera. This is a very useful tool for testing and debugging vision applications.

Warning

FTC Dashboard is not legal to run during matches, make sure it is disabled during a competition. More information can be found [here²¹²](#).

You can find FTC Dashboard [here²¹³](#).

Scrcpy

Scrcpy is an easy way to display and control Android devices connected over USB. It supports Linux, Windows, and macOS, offers low latency, requires low resources, and has great performance. It is extremely useful for debugging vision code on a Control Hub, as it will enable you to see the camera output in near real time. It can also be used with a robot controller phone, however the camera output can be seen on the phone's screen.

You can find scrcpy [here²¹⁴](#).

²¹⁰ <https://github.com/OpenFTC/EasyOpenCV>

²¹¹ <https://github.com/deltacv/EOCV-Sim>

²¹² <https://acmerobotics.github.io/ftc-dashboard/competition>

²¹³ <https://github.com/acmerobotics/ftc-dashboard>

²¹⁴ <https://github.com/Genymobile/scrcpy>

10.3 Programming Concepts

Many of the pages here are on concepts that are not necessarily FTC® specific, and are generally more theoretical than actual tutorials on FTC specific concepts.

10.3.1 Control Loops

Control loops are software used to operate power transmission systems (such as a drivetrain or linear slide) in a fast and controlled fashion. Not only do control loops let you run mechanisms quickly without fear of losing control, in many cases, they help preserve the longevity of mechanisms by reducing rapid change of applied motor voltage.

What is Error?

The first thing that must be defined when discussing control loops is the concept of error.

Error is defined as the difference between where you are and where you want to be. For instance, say you tell your drivetrain to drive at 30 inches per second, but in actuality, at a time, the drivetrain is driving at 28 inches per second. Since $30 - 28 = 2$, the error of the drivetrain's speed at this time T is 2 inches per second. In other words, at a time $t = T$, $e(t) = 2$.

PID

A PID controller (or Proportional Integral Derivative controller) is a control loop that solely uses error to control the system. PID is a form of a **feedback control loop**, or **closed loop control**. This means that data about the variable you are controlling is required in order for the loop to control that variable. In this case, information about the **error** of the system is required to control the system with a PID controller.

The Optional Calculus

The following equation represents the rigorous mathematical definition of the output of a PID controller f at any given time t :

$$f(t) = K_p e(t) + K_i \int_0^t e(t) dt + K_d \frac{de(t)}{dt}$$

where K_p , K_i , and K_d are constants and $e(t)$, as previously mentioned, is the error in the system.

If you have no experience with calculus, don't worry; while PID is fundamentally rooted in calculus, you do not need any calculus experience to be able to understand it, only basic algebra. However, you are still urged to read the rest of the section regardless of calculus experience, as the formula alone doesn't tell you why it works.

Simplification of the PID formula

Here is a simplified version of the PID formula: $f(t) = K_p P(t) + K_i I(t) + K_d D(t)$

All we have done is simply take the full formula and replaced part of the terms with functions: $P(t)$, $I(t)$, and $D(t)$.

The Proportional Term

The first component of the function, $K_p P(t)$, is by far the most simple and easy to understand, as $P(t) = e(t)$. For the sake of example, let's pretend that $K_i = 0$ and $K_d = 0$ (a PID controller with only a proportional constant is known as a **P controller**). How will the system behave? Well, if the error is large, the output will be large. Likewise, if the error is small, the output will be small. Also, ideally, given enough time, the system always approaches its destination, assuming K_p is of the correct sign.

Say we apply this to a drivetrain. You want to drive a distance D , and you decide to set your motor powers using a P controller to accomplish this. In this case, your error is how far away the robot is from the desired location. As you start to drive forward, your error is large, so you drive forward quickly, which is desirable. After all, you aren't concerned with overshooting the target yet if you are far away from it.

But as the robot's distance to the target approaches 0, you will start to slow down, gaining more control over the robot. Once the error is zero, ideally, the robot will stop, and you have reached your destination. If you happen to overshoot, the error will become negative, and the robot will backtrack, repeating the process.

The Derivative Term

This term, $K_d D(t)$, is intended to dampen the rate of change of the error. In other words, it tries to keep the error constant. How is this done?

Well, for those of you with calculus under your belt, $D(t) = \frac{de(t)}{dt}$. For those without calculus experience, it represents how fast the error is changing. Graphically, $D(t)$ is simply the slope of the error at any given time t .

This slope can be calculated by keeping track of the error over successive iterations of the control loop. One iteration occurs at time t_n with an error of $e(t_n)$. At the next iteration, the time is t_{n+1} with an error of $e(t_{n+1})$. Thus, to find $D(t)$, simply find the slope of $e(t)$ given these two points.

The Integral Term

Admittedly, the integral term is the least important term for FTC® PID control loops. With a properly tuned K_p and K_d , you often can just set K_i to 0 and call it a day.

However, it can still be useful in some cases. Just like the derivative term, the integral term intends to correct for overshoot. If the system thinks it reached its destination, it will stop, even when, in fact, the error is not yet 0. Perhaps the motor is no longer being supplied enough power to move. Well, given enough time, the integral term will increase the output (in this case, motor power), causing movement towards the destination.

To explain without calculus, the integral term essentially sums the error over a specific interval of time. To do this, error in each loop iteration is added to a variable (in this case, $I(t)$).

However, summing error this way has an unfortunate side effect: the longer the loop takes to complete one iteration, the more slowly this sum increases, which is obviously not desirable, as we don't want lag to affect

how the robot moves. To compensate for this, before the error is added to $I(t)$, it is multiplied by how long the previous loop took to complete, or $t_{n+1} - t_n$, preventing lag from making the system sum more slowly.

So say the robot stops short of the target. The P and D combination aren't strong enough to move it forward to the destination. You can either tune K_p and K_d to compensate (**this is recommended**), or you can add the integral term to increase output (**this works too, but requires more attention and tuning to achieve the same result**).

PID Pseudocode

```
while True:
    current_time = get_current_time()
    current_error = desire_position-current_position

    p = k_p * current_error

    i += k_i * (current_error * (current_time - previous_time))

    if i > max_i:
        i = max_i
    elif i < -max_i:
        i = -max_i

    D = k_d * (current_error - previous_error) / (current_time - previous_time)

    output = p + i + d

    previous_error = current_error
    previous_time = current_time
```

Tuning a PID Loop

The most important thing to know while tuning a PID loop is how each of the terms affects the output. This can allow you to see which gains need to be adjusted.

For example, if the target is not reached but instead the setpoint begins to oscillate around the target, it means there is not enough D gain. If the target is eventually reached, albeit very slowly, that means there is not enough P gain or the D gain is too high.

In brief, the P variable drives the error towards zero, the I variable corrects for steady state error, and the D variable dampens the effects of the P variable, more so as error approaches zero, which prevents overshoot.

The most common method for tuning a PID controller is as follows:

1. Set the I and D gains to zero
2. Increase the P gain until there are oscillations around the target
3. Increase the D gain until no overshoot occurs
4. If there is steady state error, increase the I gain until it is corrected

An important thing to note is that most systems do not need both I and D control. Generally, systems without a lot of friction do not need an I term, but will need more D control. Systems with a lot of friction, on the other hand, generally do not need D control because the friction facilitates deceleration but need I control because the friction prevents the system from reaching the target otherwise.

For a more in-depth explanation, [click here](#)²¹⁵

Built-In PID Controller

For situations where one needs to control the velocity or position of a single motor, the built in PID controller can be used. PID can be enabled by changing the run mode to `RUN_USING_ENCODER`

Hint

Many misunderstand the use of `RUN_USING_ENCODER`, many may mistake that it is necessary to use this mode for the encoders to work at all, but this is not true. Instead, `RUN_USING_ENCODER` enables velocity feedback using the encoder. If you are using an external PID controller such as one that you implement, generally, it is recommended that you use `RUN_WITHOUT_ENCODER`.

For official documentation on the built in PID controller, [see here](#)²¹⁶

Debugging Built-In PID Controller

Problem	Solution
Motor goes at Full Speed regardless of velocity set-point	Most of the time this occurs when one of two things occurs: #1: Your encoder is not connected properly. Diagnosis: Log your encoder position to telemetry, if the position oscillates between 0 - 1 make sure you have the correct cable and it is seated correctly. #2: Your motor is going in the wrong direction. Diagnosis: Log your velocity to telemetry, if you have a positive reference velocity and the output is negative or vice versa then your motor is plugged in backwards.
Motor does not reach full speed with <code>.setPower</code>	Use the <code>.setVelocity</code> method as part of <code>DcMotorEx</code> or use <code>RUN_WITHOUT_ENCODER</code> with an external PID controller.

²¹⁵ <https://blog.wesleyac.com/posts/intro-to-control-part-two-pid-tuning>

²¹⁶ <https://docs.revrobotics.com/duo-control/programming/using-encoder-feedback>

PID Controller Sample Rate

For teams who desire the most performance out of their PID controller, it is essential to consider the Sample rate of the controller. The Sample rate is when the controller updates its output given new sensor data. Higher Sample rates allow for more stable control and allow for the usage of more significant PID coefficients to reduce settling time. See this [video](#)²¹⁷ to see how sample rate effects stability in a practical motor control example. The inbuilt PID controller is locked at a 20hz refresh rate (50ms sample rate). Many top FTC teams optimize their robot loops to run at up to 80hz, achieving *much* more stable control with an external PID.

Feedforward Control

Feedforward control is a method of what is known as “open-loop” control. This is the opposite of closed-loop control and the primary difference is that feedforward does not actively use sensors to control the system. Instead it “predicts” the desired input based on a model.

Typically feedforward is used to control either rates of change or combat known disturbances from your system.

Feedforward is very powerful because it is immune to noise or other sensor errors. This is because it is not actively measuring the system, but instead predicting the desired input. However, this also means that it is not very good at correcting for errors. This is why it is often used in conjunction with a closed-loop controller such as PID.

Kv Ka Feedforward Model

The most common feedforward and the one used by libraries such as road-runner is the Kv-Ka feedforward model:

$$f(t) = K_v \cdot \text{Velocity} + K_a \cdot \text{Acceleration}$$

Where K_v is the velocity gain, K_a is the acceleration gain, and $f(t)$ is the feedforward output sent to your motors.

These gains can be estimated by giving the controller a series of ramp inputs (such as those computed with a motion profile), measuring the output, and then changing these gains till the robot matches the desired motion.

Note

The gains will change based on the robot’s mass, friction, and other factors. It is recommended to re-estimate these gains every time you make a significant change to your robot.

²¹⁷ <https://www.youtube.com/watch?v=fusr9eTceEo&t=133s>

Kv Ka Feedforward Pseudocode

```
while True:
    targetVelocity = getTargetVelocity(time)
    targetAcceleration = getTargetAcceleration(time)
    output = targetVelocity * Kv + targetAcceleration * Ka;
```

Static Friction Feedforward

In every system there is bound to be some amount of static Friction. This means that the robot mechanism will not move until a certain amount of power is applied. This can be modeled by adding a constant feedforward term in the direction you want to move.

```
while True:
    error = desire_position - current_position;
    sign = signum(error) # sign of error, will be -1, 0, or 1
    output = sign * staticFriction + PID(error); # PID Controller + Friction
    ↪ Feedforward
```

Motion Profiles

Tip

Motion profiles are *not* a specific type of control loop, but rather a technique that works well in combination with other control loops such as PID and feedforward.

Motion profiling is a technique popularized in FRC® that is starting to find its way to FTC. A motion profile is a function used to change the speed of a power transmission system in a controlled and consistent way by changing desired speed gradually rather than instantaneously.

Let's illustrate this with an example: say you want your drivetrain, which is initially unmoving, to drive forward at full speed. Ordinarily, you would set all drivetrain motors to full power in the code. However, this can be problematic because even though you tell the motors to move at full speed instantaneously, the drivetrain takes time to get to full speed. This can lead to uncontrolled movements which have the potential to make autonomous less consistent and, perhaps more importantly, damage mechanisms.

Motion profiling attempts to solve this issue.

Advantages

- More controlled and predictable movements
- Reduces rapid change of applied motor voltage

Disadvantages

- Can be slower

There are two main types of motion profiles: **Trapezoidal** profiles and **S-Curve** profiles. Trapezoidal profiles accelerate the system at a constant rate, and S-Curve profiles assume jerk (the speed acceleration changes) is constant. Given that S-Curve profiles are not optimal for controlling 2d trajectories (such as driving) and exist to reduce slippage (which usually only occurs when driving in FTC), trapezoidal profiles are recommended for most FTC applications.

Trapezoidal profiles get their name from the shape of the graph of velocity over time:

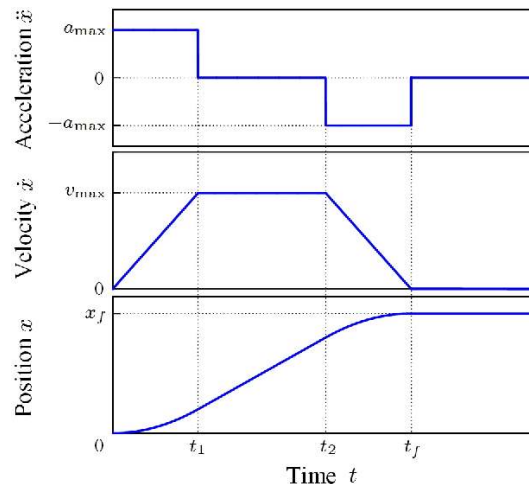


Fig. 1: These are the “magic functions” for velocity and acceleration over time alluded to in the feedforward section.

Here is some pseudocode for a trapezoidal profile:

```
while True:
    current_velocity = get_current_velocity()
    current_time = get_current_time()

    direction_multiplier = 1

    if position_error < 0:
        direction_multiplier = -1

    # if maximum speed has not been reached
    if MAXIMUM_SPEED > abs(current_velocity):
        output_velocity = current_velocity + direction_multiplier * MAX_ACCELERATION *
        ↪ (current_time - previous_time)
        output_acceleration = MAX_ACCELERATION

    #if maximum speed has been reached, stay there for now
    else:
        outputVelocity = MAXIMUM_SPEED
        outputAcceleration = 0

    #if we are close enough to the object to begin slowing down
    if position_error <= (output_velocity * output_velocity) / (2 * MAX_ACCELERATION)):
        output_velocity = current_velocity - direction_multiplier * MAX_ACCELERATION *
        ↪ (current_time - previous_time)
```

(continues on next page)

(continued from previous page)

```

→ (current_time - previous_time)
    output_acceleration = -MAX_ACCELERATION

    previous_time = current_time

```

The results of the above pseudocode are then used in a feedforward and / or PID loop to control the position of the system in a smooth and predictable way.

A more advanced example of the math for motion profile generation as used in the [Road Runner library](https://github.com/acmerobotics/road-runner)²¹⁸ can be found in this [Jupyter Notebook](https://mybinder.org/v2/gh/acmerobotics/road-runner/HEAD?filepath=doc%2Fnotebook%2Froad-runner-lite.ipynb)²¹⁹.

10.3.2 Finite State Machines

Finite State Machines (FSM) are often used while programming in order to allow for more complex series of actions. This is especially useful when one needs multiple tasks to run at the same time, because it allows for tasks to depend on each other's execution in a non-linear fashion.

What is a Finite State Machine?

The name of a finite state machine is very descriptive; it's a state machine, with a finite number of states. It can be in one state at a time, and can transition to a different state once something happens. The [Wikipedia](https://en.wikipedia.org/wiki/Finite_state_machine)²²⁰ example of a turnstile explains the concept very well.

Implementation

Naive Implementation

When first learning about FSMs, it is quite common for programmers to try and use them. Often times, they try to apply an FSM to their autonomous programs by segmenting their autonomous into a giant switch statement, and it often looks something like this:

```

while (opModeIsActive()) {
    switch (state) {
        case DETECT_SKYSTONE:
            // skystone detection code here
            int position = detectSkystone();

            if (position == 0) {
                state = SKYSTONE_POS_0;
            }
            else if (position == 1) {
                state = SKYSTONE_POS_1;
            }
            else {
                state = SKYSTONE_POS_2;
            }
            break;
    }
}

```

(continues on next page)

²¹⁸ <https://github.com/acmerobotics/road-runner>

²¹⁹ <https://mybinder.org/v2/gh/acmerobotics/road-runner/HEAD?filepath=doc%2Fnotebook%2Froad-runner-lite.ipynb>

²²⁰ https://en.wikipedia.org/wiki/Finite_state_machine#Example:_coin-operated_turnstile

(continued from previous page)

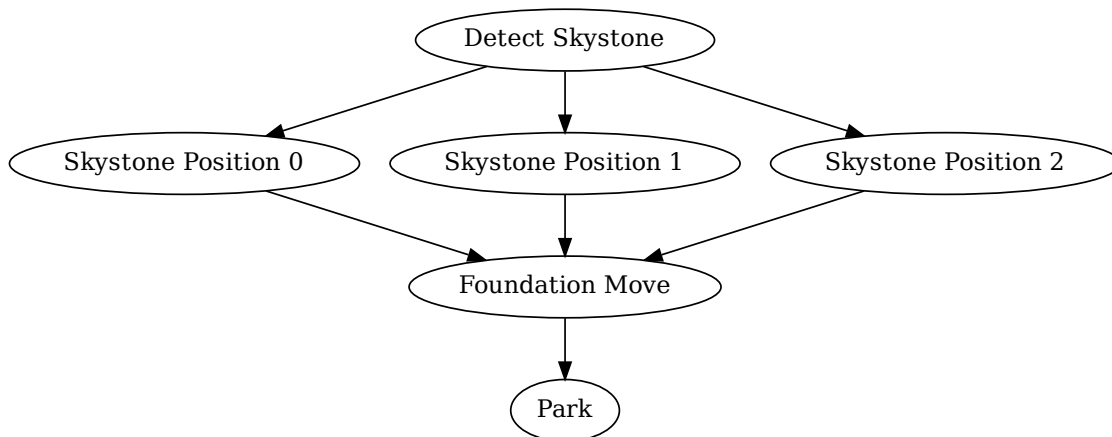
```

    case SKYSTONE_POS_0:
        // skystone position 0 here
        doSkystone(0);
        state = MOVE_FOUNDATION;
        break;
    case SKYSTONE_POS_1:
    case SKYSTONE_POS_2:
        // etc etc
        break;
    case MOVE_FOUNDATION:
        // foundation move code
        state = PARK;
        break;
    case PARK:
        // park the bot
        break;
}
}

```

This however does not really have any benefits compared to if the programmer had simply put each of the code's segments into functions, and executed them in order. In fact, often times, programmers will structure their code like this instead of splitting their code up into functions. The result is an autonomous that's more difficult to debug, and ultimately harder to fix on the fly during a competition or other time crunch.

If one drew out the state transition diagram for each of the states, for the autonomous above it'd be very linear, and the state transitions always occur because the section of the code finished:



In fact, in many implementations, making state transitions for any other reason is often difficult because the code executes linearly and is only in a loop to rerun the switch statements. (Often times, this means the code has a hard time reacting to a stop request in the middle of autonomous.)

Warning

It is inadvisable to write code like this. If your autonomous is synchronous, it is preferable to split your code up into functions and run them in order, as this will be easier to understand and edit on the fly.

Useful Implementation

FSMs are the right tool to use when a robot needs to complete multiple tasks at once; a common example of this is when a robot should have automation in teleop, but still have control over the drivetrain.

Often times, teams have issues because their teleop executes in a loop and their servo logic has sleeps in it. But, we can avoid this if we write code in an **asynchronous** fashion - where instead of waiting for a task to complete before doing the next one, tasks are performed at the same time, and each task's state is checked without stopping the other tasks from executing.

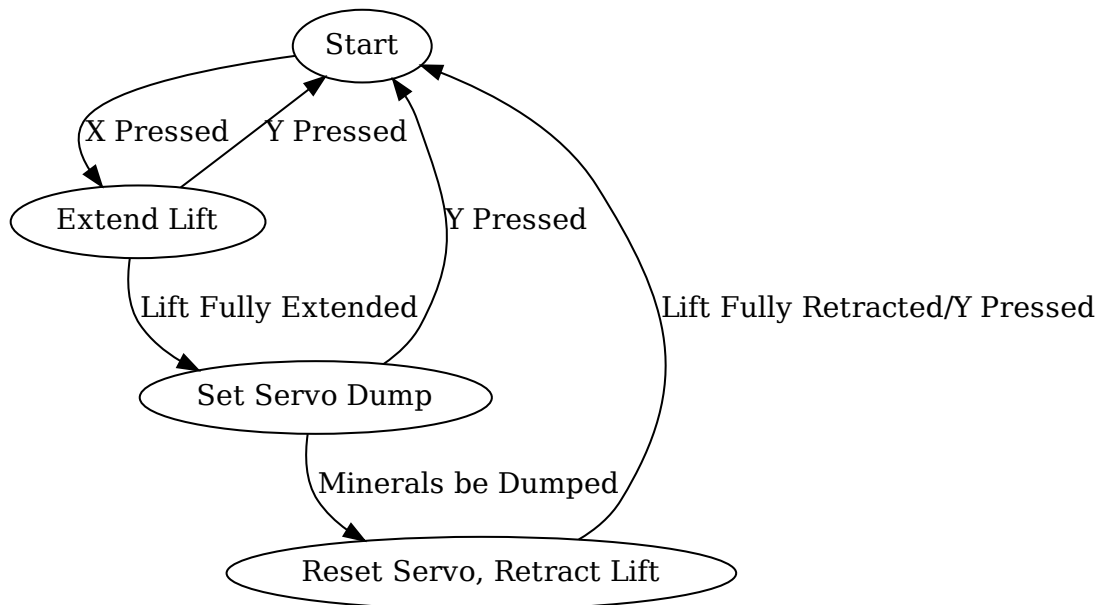
An example of this would be that if one had a robot similar to [Gluten Free's Rover Ruckus Robot²²¹](#), and one wanted to automate the scoring lift so that the drivers don't have to think while the bot deposits the minerals. There are two parts of the bot that are relevant to us in this exercise: the angled scoring lift, and the servo that tips the dumper so the minerals fall out. The goal is to be able to push a button, and then the bot will:

- extend the lift,
- at full lift extension, angle the mineral bucket servo to deposit the minerals,
- wait for the minerals to fall out,
- reset the servo to the original position
- retract the lift

If the drivers press a specific other button, we will stop executing the actions above as a failsafe - in case the robot is breaking somehow and the drivers need to take manual control. All the while, the drivers should still be able to control our drivetrain so we can make adjustments. Now, of course, this is a bit simplified (and probably not entirely what GF did), but it will do for now.

Before anything is programmed, it may be useful draw out the state diagram for this to get a better understanding of what we the robot should actually be doing. This can also add to a Control Award submission.

²²¹ <https://www.youtube.com/watch?v=NQvhvYJXVMA>



Notice how resetting the dump servo and retracting the lift share a state. That's because the robot doesn't need to wait for the servo to reset before moving the lift down; they can both happen at once.

Now, let's get into actually implementing the code for this. In a traditional OpMode, which is commonly used for teleop, code runs repeatedly in a `loop()` function, so instead of waiting for a state transition to happen directly, the code will repeatedly check on each `loop()` call if it should perform a state transition. This kind of "update our state" pattern keeps code from blocking the rest of the `loop()` code from running, such as the drivetrain.

```

/*
 * Some declarations that are boilerplate are
 * skipped for the sake of brevity.
 * Since there are no real values to use, named constants will be used.
 */

@TeleOp(name="FSM Example")
public class FSMEExample extends OpMode {
    // An Enum is used to represent lift states.
    // (This is one thing enums are designed to do)
    public enum LiftState {
        LIFT_START,
        LIFT_EXTEND,
        LIFT_DUMP,
        LIFT_RETRACT
    };

    // The liftState variable is declared out here
    // so its value persists between loop() calls
    LiftState liftState = LiftState.LIFT_START;

    // Some hardware access boilerplate; these would be initialized in init()

```

(continues on next page)

(continued from previous page)

```

// the lift motor, it's in RUN_TO_POSITION mode
public DcMotorEx liftMotor;

// the dump servo
public Servo liftDump;
// used with the dump servo, this will get covered in a bit
ElapsedTime liftTimer = new ElapsedTime();

final double DUMP_IDLE; // the idle position for the dump servo
final double DUMP_DEPOSIT; // the dumping position for the dump servo

// the amount of time the dump servo takes to activate in seconds
final double DUMP_TIME;

final int LIFT_LOW; // the low encoder position for the lift
final int LIFT_HIGH; // the high encoder position for the lift

public void init() {
    liftTimer.reset();

    // hardware initialization code goes here
    // this needs to correspond with the configuration used
    liftMotor = hardwareMap.get(DcMotorEx.class, "liftMotor");
    liftDump = hardwareMap.get(Servo.class, "liftDump");

    liftMotor.setTargetPosition(LIFT_LOW);
    liftMotor.setMode(DcMotor.RunMode.RUN_TO_POSITION);
}

public void loop() {
    liftMotor.setPower(1.0);

    switch (liftState) {
        case LIFT_START:
            // Waiting for some input
            if (gamepad1.x) {
                // x is pressed, start extending
                liftMotor.setTargetPosition(LIFT_HIGH);
                liftState = LiftState.LIFT_EXTEND;
            }
            break;
        case LIFT_EXTEND:
            // check if the lift has finished extending,
            // otherwise do nothing.
            if (Math.abs(liftMotor.getCurrentPosition() - LIFT_HIGH) < 10) {
                // our threshold is within
                // 10 encoder ticks of our target.
                // this is pretty arbitrary, and would have to be
                // tweaked for each robot.

                // set the lift dump to dump
                liftDump.setTargetPosition(DUMP_DEPOSIT);

                liftTimer.reset();
                liftState = LiftState.LIFT_DUMP;
            }
    }
}

```

(continues on next page)

(continued from previous page)

```

        break;
    case LIFT_DUMP:
        if (liftTimer.seconds() >= DUMP_TIME) {
            // The robot waited long enough, time to start
            // retracting the lift
            liftDump.setTargetPosition(DUMP_IDLE);
            liftMotor.setTargetPosition(LIFT_LOW);
            liftState = LiftState.LIFT_RETRACT;
        }
        break;
    case LIFT_RETRACT:
        if (Math.abs(liftMotor.getCurrentPosition() - LIFT_LOW) < 10) {
            liftState = LiftState.LIFT_START;
        }
        break;
    default:
        // should never be reached, as liftState should never be null
        liftState = LiftState.LIFT_START;
}

// small optimization, instead of repeating ourselves in each
// lift state case besides LIFT_START for the cancel action,
// it's just handled here
if (gamepad1.y && liftState != LiftState.LIFT_START) {
    liftState = LiftState.LIFT_START;
}

// mecanum drive code goes here
// But since none of the stuff in the switch case stops
// the robot, this will always run!
updateDrive(gamepad1, gamepad2);
}
}

```

10.3.3 Kinematics

Kinematics is the application of geometry to the control of various robot mechanisms. Kinematics equations are used to control mechanisms by providing specific inputs to achieve a desired output.

Many of the kinematics equations here were taken from [Controls Engineering in the FIRST Robotics Competition \(book\)](https://file.tavsys.net/control/controls-engineering-in-frc.pdf)²²² and [Mobile Robot Kinematics for FTC \(paper\)](https://github.com/acmerobotics/road-runner/blob/master/doc/pdf/Mobile_Robot_Kinematics_for_FTC.pdf)²²³, which contain the relevant derivations. While only tank (differential drive) and mecanum kinematics equations are shown here, these sources also contain derivations for other mechanisms such as swerve and dead wheel odometry.

²²² <https://file.tavsys.net/control/controls-engineering-in-frc.pdf>

²²³ https://github.com/acmerobotics/road-runner/blob/master/doc/pdf/Mobile_Robot_Kinematics_for_FTC.pdf

Forward vs Inverse Kinematics

Mechanisms may have different sets of equations for their forward and inverse kinematics. Forward kinematics are the equations used to determine the state of a system given the state of its outputs, whereas inverse kinematics determines the output of a system given the desired state. For example, in a drivetrain, forward kinematics would determine body velocity of the robot based on the individual velocities of the wheels, whereas inverse kinematics would determine the required wheel velocities for a desired body velocity.

Tank (Differential Drive)

A tank, or differential drive, is a drivetrain consisting of two sets of wheels on either side of the robot that are independently driven. These are described under further detail in the [Tank \(Skid-Steer\) Drivetrains](#) (page 106) section.

Variables

The following variables are used in this section.

- v_r denotes the linear velocity of the right wheel(s)
- v_l denotes the linear velocity of the left wheel(s)
- v_f denotes the forward velocity of the robot, relative to itself
- ω denotes the rotational velocity of the robot in radians/second
- r_b denotes the base track radius, or the distance between the wheel and center of the robot (half of the distance between wheels)

Warning

These variables, with the exception of ω , represent **linear** velocities NOT **rotational** velocities. Wheel rotational velocity in radians/second can be converted to linear velocity by multiplying by the wheel's radius.

Positive rotational velocity (ω) will spin the robot COUNTERCLOCKWISE when viewed from above.

Forward Kinematics

The forward kinematics of a tank drive relate the velocity of the wheels to the forward and rotational velocities of the robot, relative to itself. The forward velocity v_f and the rotational velocity v_θ is:

$$v_f = \frac{v_r + v_l}{2}$$
$$\omega = \frac{v_r - v_l}{2r_b}$$

Inverse Kinematics

The inverse kinematics of a tank drive relate the desired velocity of the robot to the velocity required of the wheels. These velocities are as follows:

$$v_r = v_f + r_d \cdot \omega$$

$$v_l = v_f - r_d \cdot \omega$$

Mecanum Drive

Variables

Mecanum kinematics uses the same variables as differential drive, except with four wheel velocity variables and an additional robot velocity vector for the left to right velocity.

- v_{fr} denotes the linear velocity of the front (leading) right wheel
- v_{br} denotes the linear velocity of the back (trailing) right wheel
- v_{fl} denotes the linear velocity of the front (leading) left wheel(s)
- v_{bl} denotes the linear velocity of the back (trailing) left wheel(s)
- v_f denotes the forward velocity of the robot, relative to itself.
- v_s denotes the strafe (sideways) velocity of the robot, relative to itself.
- ω denotes the rotational velocity of the robot in radians/second
- r_b represents the base track radius, or the distance between the wheel and center of the robot (half of the distance between wheels)

Warning

These variables, with the exception of ω , represent **linear** velocities NOT **rotational** velocities. Wheel rotational velocity in radians/second can be converted to linear velocity by multiplying by the wheel's radius.

Positive rotational velocity (ω) will spin the robot COUNTERCLOCKWISE when viewed from above.

Forward Kinematics

The forward kinematics of a mecanum drive relate the velocity of the wheels to the forward, strafe, and rotational velocities of the robot, relative to itself. These are as follows:

$$v_f = \frac{v_{fr} + v_{fl} + v_{br} + v_{bl}}{4}$$

$$v_s = \frac{v_{bl} + v_{fr} - v_{fl} - v_{br}}{4}$$

$$\omega = \frac{v_{br} + v_{fr} - v_{fl} - v_{bl}}{4 * 2r_b}$$

Inverse Kinematics

The inverse kinematics of a mecanum drive relate the desired velocity of the robot to the velocity required on the wheels. These are as follows:

$$v_{fl} = v_f - v_s - (2r_b \cdot \omega)$$

$$v_{bl} = v_f + v_s - (2r_b \cdot \omega)$$

$$v_{br} = v_f - v_s + (2r_b \cdot \omega)$$

$$v_{fr} = v_f + v_s + (2r_b \cdot \omega)$$

10.3.4 Odometry

Odometry is a form of localization that uses data from sensors like encoders to derive an estimated position relative to a starting point. Localization is a means for being able to locate the position of the bot at some point in time. Odometry is especially useful in autonomous programs because it allows for easier implementation of different tasks on the field due to understanding one's position.

Pose

We refer to pose, which is the position of some body (like a bot), normally in the context two-dimensional space, as the movement of the robot is generally constrained to a single plane. We notate the robot's pose as $\vec{x}$. A pose contains two entries: the robot's position and heading; position is generally in Cartesian coordinates, so the pose can be represented with x , y , and θ . A "heading" is a term for the direction towards which the front of the robot is facing. Because of this, the robot's coordinate frame is set up such that the global x-axis is lined up with the 0 heading.

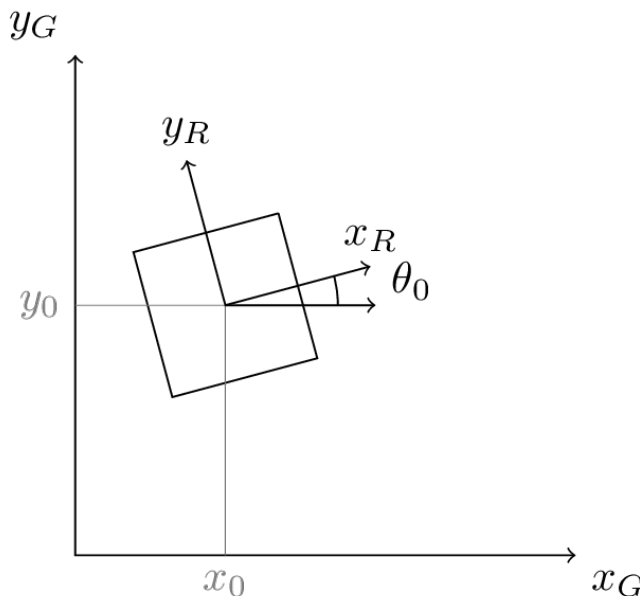


Fig. 2: Road Runner Coordinate Frame Documentation²²⁴

²²⁴ <https://acme-robotics.gitbook.io/road-runner/tour/coordinate-frame>

We can refer to the current pose ($\vec{x}_0$) of the robot as $\begin{pmatrix} x_0 \\ y_0 \\ \theta_0 \end{pmatrix}$. This is just fancy notation for a point on the field (x_0, y_0) with a specified orientation of the robot—the heading θ_0 . A pose generally has some beginning origin in the coordinate frame.

Updating the Pose

The change in pose over some very small amount of time is $\Delta\vec{x}$. The difference in time between the current pose and the last pose should be as small as possible to improve the approximations for the math. Teams should update their robot pose every cycle of their control loop.

Updating the pose is as simple as adding the transformed change to the previous pose where $\varphi = \Delta\theta$

$$\begin{pmatrix} x \\ y \\ \theta \end{pmatrix} = \begin{pmatrix} x_0 \\ y_0 \\ \theta_0 \end{pmatrix} + \begin{pmatrix} \Delta x \\ \Delta y \\ \varphi \end{pmatrix}$$

The idea of odometry is to use sensor data and math to form an approximation for the robot's pose over time.

Finding the Change in Position

In order to determine the current location of the robot and update its pose, the change must be calculated using data read from the sensors. For a robot, there will be three possible sensors that you can use: two that are parallel with the robot's body in the x -direction and one that is aligned with the y -direction of movement (perpendicular to the drive wheels).

Angle and Displacement

The displacement (or change in position) of the left sensor is Δx_l and the displacement of the right sensor is Δx_r . The lateral distance between these two sensors is called the trackwidth, notated as L . This is very important for determining angle for turning approximations. This value will need to be tuned, which means tested repeatedly and then brought to some converging value that is close to the actual measurement.

Deriving the value of φ then becomes simple:

$$\varphi = \frac{\Delta x_l - \Delta x_r}{L}$$

To perform later calculations, we need to know the displacement of the robot in the x -direction relative to its center rather than the two parallel sensors. To do this, we take the average to derive Δx_c , or the center displacement:

$$\Delta x_c = \frac{\Delta x_l + \Delta x_r}{2}$$

The final displacement we need before we can determine the change in pose is the horizontal displacement $\Delta x_\perp$. This is the displacement of the perpendicular sensor Δx_h with a correction for forward offset F . In order to get accurate approximations, the forward offset needs to be considered. When the sensor is closer to the back, the offset is negative, but when it is closer to the front, it is positive. This is to account for the change in its position based on point-turns.

²²⁵ <https://learnroadrunner.com/dead-wheels.html#three-wheel-odometry>

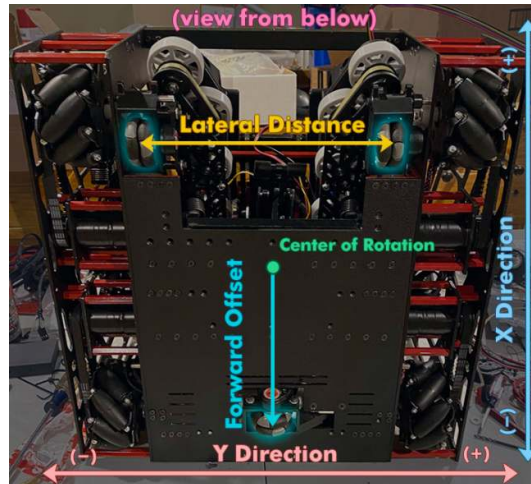


Fig. 3: 17508 Rising Tau's 2019/20 Skystone Bot^{Page 353, 225}

As a result of this, we can define our horizontal displacement as:

$$\Delta x_{\perp} = \Delta x_h - (F * \varphi)$$

Note

If you do not have perpendicular sensors, which are not required if the robot cannot move in the lateral direction, $\Delta x_{\perp}$ is not necessary.

For this value, use 0 if you do not have a horizontal sensor.

Robot-Relative Deltas

Let's come up with a simplified, nonoptimal way to calculate our robot-relative pose deltas which we can then transform into field-relative coordinate changes. To perform this we need to transform the robot-relative deltas via a rotation matrix where we rotate the relative pose difference by the original heading. We can derive the values of Δx and Δy .

$$\begin{pmatrix} \Delta x \\ \Delta y \\ \varphi \end{pmatrix} = \begin{pmatrix} \cos(\theta_0) & -\sin(\theta_0) & 0 \\ \sin(\theta_0) & \cos(\theta_0) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \Delta x_c \\ \Delta x_{\perp} \\ \varphi \end{pmatrix}$$

From this, we can calculate our field-relative change in pose:

$$\begin{pmatrix} \Delta x \\ \Delta y \\ \varphi \end{pmatrix} = \begin{pmatrix} \Delta x_c \cos(\theta_0) - \Delta x_{\perp} \sin(\theta_0) \\ \Delta x_c \sin(\theta_0) + \Delta x_{\perp} \cos(\theta_0) \\ \varphi \end{pmatrix}$$

Note

This method of approximating position is known as Euler integration, but we are using it for strict pose deltas instead of integrating the velocity (essentially, this is a very simplified version of the original theory).

Warning

This is for advanced programmers; while implementing this from scratch is a great learning exercise, it is likely not the optimal way to get the best auto. There are several [resources](#) (page 356) out there for producing great, well-tested, and easy-to-implement odometry.

Odometry Pseudocode

```
while robot_is_active():
    delta_left_encoder_pos = left_encoder_pos - prev_left_encoder_pos
    delta_right_encoder_pos = right_encoder_pos - prev_right_encoder_pos
    delta_center_encoder_pos = center_encoder_pos - prev_center_encoder_pos

    phi = (delta_left_encoder_pos - delta_right_encoder_pos) / trackwidth
    delta_middle_pos = (delta_left_encoder_pos + delta_right_encoder_pos) / 2
    delta_perp_pos = delta_center_encoder_pos - forward_offset * phi

    delta_x = delta_middle_pos * cos(heading) - delta_perp_pos * sin(heading)
    delta_y = delta_middle_pos * sin(heading) + delta_perp_pos * cos(heading)

    x_pos += delta_x
    y_pos += delta_y
    heading += phi

    prev_left_encoder_pos = left_encoder_pos
    prev_right_encoder_pos = right_encoder_pos
    prev_center_encoder_pos = center_encoder_pos
```

Using Pose Exponentials

This method uses differential equations to solve the nonlinear position of the robot given constant curvature. Euler integration assumes that the robot follows a straight path between updates, which can lead to inaccurate approximations when traveling around curves. If you are interested in the math itself, we recommend you check out [this book](#)²²⁶ for FRC® controls.

We'll treat the way it is solved in this page as a black box, and derive the formula by implementing a correction for this nonlinear curvature into our Euler integration robot-relative deltas equation:

$$\begin{pmatrix} \Delta x \\ \Delta y \\ \varphi \end{pmatrix} = \begin{pmatrix} \cos(\theta_0) & -\sin(\theta_0) & 0 \\ \sin(\theta_0) & \cos(\theta_0) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \frac{\sin(\varphi)}{\varphi} & \frac{\cos(\varphi)-1}{\varphi} & 0 \\ \frac{1-\cos(\varphi)}{\varphi} & \frac{\sin(\varphi)}{\varphi} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \Delta x_c \\ \Delta x_{\perp} \\ \varphi \end{pmatrix}$$

²²⁶ <https://file.tavsys.net/control/controls-engineering-in-frc.pdf>

Resources for Odometry

There are several great resources out there for odometry. We highly recommend [Road Runner](#)²²⁷. For the math behind Road Runner (which utilizes pose exponentials), you can also read [Ryan's paper](#)²²⁸. An additional resource for Road Runner is [Learn Road Runner](#)²²⁹ which is a step-by-step procedural guide that explains how to work with the [Road Runner quickstart](#)²³⁰.

We also recommend [Tyler's book](#)²³¹ as it goes into great detail about various controls in *FIRST*® robotics.

If you're using other resources, it is important that you do not use ones that utilize Euler integration as it is less optimal for real life approximations of robot pose.

10.4 Advanced Control System

This section covers advanced details about the SDK and control system.

10.4.1 Control System Electronics

This page contains a breakdown of the known electronics in the control system, as well as any notes regarding them

Expansion Hub Internals

Warning

Don't take apart a Control or Expansion Hub unless you really know what you are doing. They can be damaged in the process, especially if one does not know how to properly reassemble it. THIS WILL VOID YOUR WARRANTY!

Lynx Board

"Lynx" is the codename of the board within the Expansion Hub and Control Hub that interacts with hardware. References to "Lynx" are made in the FTC® SDK refer to this board. It appears to have been developed by both REV and DEKA, possibly for use in *FIRST*® Global (judging by the *FIRST* Global silkscreen on the PCB).

Warning

Don't take apart a Control or Expansion Hub unless you really know what you are doing. They can be damaged in the process, especially if one does not know how to properly reassemble it.

²²⁷ <https://acme-robotics.gitbook.io/road-runner/>

²²⁸ https://github.com/acmerobotics/road-runner/blob/master/doc/pdf/Mobile_Robot_Kinematics_for_FTC.pdf

²²⁹ <https://learnroadrunner.com/>

²³⁰ <https://github.com/acmerobotics/road-runner-quickstart>

²³¹ <https://file.tavsys.net/control/controls-engineering-in-frc.pdf>



Fig. 4: A Lynx board that was removed from its case

Processor

The main processor of the Expansion Hub is a Texas Instruments ARM Cortex M4 running at 80 MHz.

Ports

The expansion hub has the following ports

- Two UART Debug Ports
 - The top port outputs a continuous high speed CSV stream of data from the various subsystems in the hub.
 - The bottom port outputs data at a baud rate of 115200 at a user specified verbosity.
- Four I2C Ports
 - These ports are 100/400 kHz compliant and are connected to a separate bus so there is no need to be concerned about address collisions
 - There are integrated pull up resistors on this port
- Eight DIO Ports
 - 3.3V only, current limited, can briefly supply more than the rated current spec. Pulled up internally.
- Four Analog Ports
 - 5V compliant, you can use a level shifter to supply 5v to the sensor, but take care the analog line bypasses the level shifter. The VIN and GND lines must pass through the level shifter for it to work however.
- Six Servo Ports
 - The 5V supply on the servo ports is default OFF, and will only enable once a servo is used. All six 5V pins are switched on and off together, and disabling PWM on one port will switch all of the ports off unless another servo is used.
- Two 5V Power Ports
 - The 5V supply is shared with the servos
- Four Motor Ports

- Current limited and have overtemperature protection on the chip. The output does not brake the h-bridge during the PWM off cycle. What this means is that there might be linearity issues while deaccelerating, because of momentum in the motor.
- The motor driver is a ST Microelectronics VN15050 motor driver IC, which is capable of handling well over the maximum amount of current an FTC motor can draw. It has integrated current sensing and has been used since the modern robotics era. It has built in thermal and current safety limits. This motor controller has been used for years, and may even date back to the early HiTechnic based control system.
- Four Encoder Ports
 - **IMPORTANT: Only two of the encoder ports (Ports 0 and 3) appear to be connected via hardware and are reliable at high speed.** There are two methods of connecting an encoder internally to the texas instruments microprocessor, through hardware and software. Hardware ports use the integrated quadrature decoder chip and are extremely accurate at high speed, whereas encoders decoded in software are not reliable at high speeds. As a result, high Count Per Revolution (CPR) encoders, those with more than 4000 counts per revolution, should NOT be used on ports 1 and 2, the ports connected in software.
- Two XT30 Connectors
 - Care should be taken when moving around cables as these connectors have been known to fail and break off the board.
 - In addition, XT30s will wear and get loose with time. Hot glue is recommended if cables come loose.
- Mini USB B Connector
 - Capable of full speed USB 2.0, and 5V output to charge a phone. The 5v output may be unreliable, it depends on the hub.
- Internal Connector
 - This connector is what is used to connect to the android daughterboard. It presumably has UART capability as well as power and ground. All expansion hubs have this connector, whether or not they are a control hub. Presumably this is because the control hub and REV hub were developed together at the same time.

Control Hub

The Control Hub is an Expansion Hub with an embedded Android single-board computer daughterboard connected to it. This enables it to not need a separate Robot Controller phone, as the daughterboard functions as the Robot Controller. Internally, LynxCommands are sent over from the daughterboard to the Lynx board over an internal UART connection.

The control hub daughterboard contains a RK3328 Quad-core ARM Cortex-A53 running a custom version of Android/Linux. This software is open source, but has nothing notable except for some software that manages the wifi access point. It has no thermal spreading, such as heat sinks, on the chip, which can cause thermal throttling when heavy applications such as tensorflow are used.

Warning

Don't take apart a Control or Expansion Hub unless you really know what you are doing. They can be damaged in the process, especially if one does not know how to properly reassemble it.

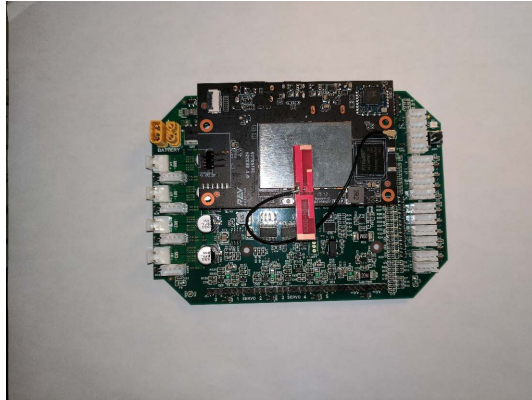


Fig. 5: The single board computer and Lynx board from a Control Hub

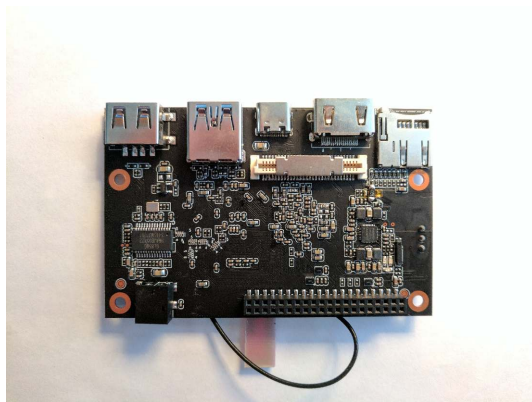


Fig. 6: The android board, removed from a control hub

10.4.2 SDK Communication

When using any method in the FTC® SDK that accesses hardware, be that setting motor power, reading an encoder, a sensor, etc., a LynxCommand is sent.

Note

LynxCommands are not sent directly from the Robot Controller to an Expansion Hub through USB; in an expansion hub they are sent through USB to an FTDI, which converts the USB signal to a UART one. In the control hub, this USB step is skipped, instead the control hub board sends the data directly over UART to the internal expansion hub.

Warning

LynxCommands being blocking (and more specifically a master lock being present on each usb device) means that multithreading hardware calls is at best not helpful and typically harmful to performance.

If an Android phone and Expansion Hub is used, LynxCommands are sent over USB; however if a Control Hub is used, LynxCommands are sent over UART. This is very important, not just because of the increased reliability with UART instead of USB, but also because LynxCommands take approximately 3 milliseconds over USB and approximately 2 milliseconds over UART.

Any expansion hubs connected via RS485 receive their commands via that connector. Lynx hubs do not have to retransmit packets, so the added latency from this process isn't significant, but there will be some added latency. Up to 255 expansion hubs can be connected together in theory.

Note

Interacting with I2C devices takes significantly longer; upwards of 7 milliseconds over USB. However, this is not because each LynxCommand takes longer, but because multiple LynxCommands must be sent to interact with I2C.

Please note that since version 5.5 of the SDK, I2C calls on the Control Hub are much faster than those on the Expansion Hub. This is because the polling rate was dramatically increased, which can cut down on unnecessary wasted time.

10.4.3 SDK Motors

The SDK offers several methods of controlling and communicating with motors, as well as a couple of hidden methods that can be easily accessed.

General Explanation

Motor Controller

All motor ports are controlled with what is called an H-bridge motor controller, a circuit that can be used to vary the output voltage as well as signage (negative or positive) of the voltage. Negative voltage through a DC Motor reverses a motor, whereas positive motor will make the motor go forward. SDK Motor power (from -1 to 1), represents a multiplier of the input voltage that is output through the motor port. The different voltages are created via PWM, where the port is turned on and off rapidly to create a lower average voltage.

In addition, the zero power behavior of the motor, that is the behavior of the motor when no power is applied, can be configured. In FLOAT mode, the motor controller simply turns off, providing minimal additional resistance. In BRAKE mode, the two motor leads are shorted together internally. Due to the inherent property that all DC brushed motors generate electricity when the shaft spins, shorting the leads causes a reverse power that stops the motor quickly and is resistant to external forces.

Note

The motor controller uses the input voltage, this means on a 13 volt battery, setting the power to 1 will create an output voltage of 13 volts. Similarly, on an 11 volt battery, a power of 1 will create an output voltage of 11 volts.

Motor Encoder

Important

There is no real standardized terminology when dealing with quadrature encoders. Here, we use the terms “count” and “tick” to represent a single rising or falling action in the quadrature wave. You may also see some datasheets list “pulses”, which can indicate anything from 1 “count” to 4 “counts”. Be careful when reading datasheets!

FTC® Encoders use the two wire quadrature format for transmitting relative encoder information. In quadrature, there are two signal wires, A and B. When moving, both A and B generate square waves that are 90 degrees apart, that is one square wave starts half way through the other square wave, and ends half way through the other square wave. When traveling in one direction, the A wire square wave leads the B wire square wave, and in the other direction the B wire square wave leads the A wire square wave. The two waves are combined in XOR to produce the output wave, where each rising and falling action is one “tick”, and the faster the wave the faster the encoder is moving.

The REV hub counts the pulses and calculates velocity by using a 5 value “ring buffer”, which has a new value added to it every 10 ms. These 5 values are then used to calculate the current velocity.

Warning

It is recommended that quadrature encoders be hooked up to special hardware decoded ports to allow them to be read correctly. The expansion hub contains special hardware for reading quadrature encoders, but because there are only two of those controllers, two of the pins are hooked up in “software” instead (the ports are decoded in software instead of in hardware). Effectively, this means that ports 0 and 3, the two ports connected to the special quadrature ports, will always read accurately. **Ports 1 and 2 are connected to the less accurate “software” ports, meaning that with high CPR encoders (encoders that produce more than 4000 counts per revolution, such as the REV Through Bore Encoder or Talon SRX Encoder) they can “lose steps” and drift.**

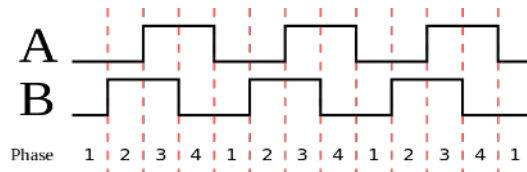


Fig. 7: An example of a quadrature wave, with channel A leading channel B. Each count is a “count” or “tick”

Hidden Methods

DcMotorEx

All REV hub DC motors are instances of `DcMotorEx`, which exposes some more methods to the user, such as velocity control and current draw measurement.

Note

There is no downside to using `DcMotorEx`, in order to convert a `DcMotor` to a `DcMotorEx` the user simply just needs to cast the `DcMotor` returned by the `hardwareMap` to a `DcMotorEx`.

Tips

- While current readings are not bulk read, current alerts (`isMotorOverCurrent()`) is bulk read.
- PID/PIDF coefficients use internal units for the output, a two byte short from -32767 to 32767, instead of the user -1 to 1.
- The default `getVelocity()` method returns the encoder velocity in ticks per second.
- The `RunMode STOP_AND_RESET_ENCODERS` is not actually a run mode. Instead, it just sets the power to zero and sends a `LynxResetMotorEncoderCommand`. This command can be issued manually if one wants to easily reset a motor encoder without changing the run mode.

10.4.4 SDK Servos

The SDK offers several methods of controlling and communicating with servos, as well as a couple of hidden methods that can be easily accessed.

General Explanation

PWM Explanation

Servos are controlled via a PWM signal. PWM signal is one where the signal turns on for a number of microseconds, then off for a number of microseconds. Servos are controlled by sending variable length pulses every 20 ms, with the length of the pulses dictating what angle the servo should move to (or with CR servos, what speed it should move at). This length of the pulse, expressed in microseconds, is called the PWM pulse width. By default, the sdk generates signals from 600 to 2400 microseconds (with SDK 0 being 600 microseconds, and SDK 1 being 2400 microseconds). However, the expansion hub can actually generate between 500 and 2500 microseconds if the range is set manually.

5V Power

All servos require a minimum of 5V to operate, and so the expansion hub generates 5V to power the servos. The 5V power is shared between pairs of ports (0-1, 2-3, 4-5). By default, this 5V power is OFF, and doesn't provide power. However, doing any servo operation, such as setting the position of a servo, will turn ON the 5V power to both shared ports. Calling disable PWM seems to turn OFF shared power, however if another servo is used or is active the 5V power will remain ON instead.

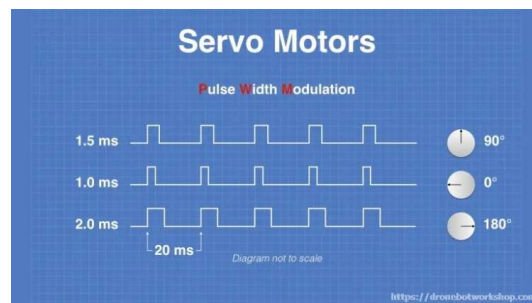


Fig. 8: An example of servo PWM waves. NOTE: Servo angles are arbitrarily selected and will not be accurate to all servos

Warning

Every pair of **two ports**, (0-1, 2-3, 4-5) share the SAME 2 amp limitation, so care should be taken that pairs of servos do not pull more than 2 amps. Putting servos in every other port is recommended when possible.

Hidden Methods

ServoImplEx

All REV hub servos are instances of `ServoImplEx`, which exposes some more methods to the user, such as setting the servo PWM range.

Note

There is no downside to using `ServoImplEx`, in order to convert a `Servo` to a `ServoImplEx` the user simply just needs to cast the `Servo` returned by the `hardwareMap` to a `ServoImplEx`.

Tips:

- You can use `setPwmRange()` to increase the servo range to a maximum of 500 to 2500 microseconds. This changes the SDK mapping, with 0 becoming the minimum microseconds and 1 becoming the maximum microseconds. **Increasing the range to 500-2500 can give more range when controlling servos that use 500-2500 PWM Range, such as goBILDA servos and REV Smart Robot Servo**
- `setPwmDisable()` and `setPwmEnable()` can be used to turn on and off the PWM signal to a servo. On some servos, this will cause the servo to de-energize and “turn off” with no holding power. Other servos may stay on and hold their position. Behavior will vary from servo manufacturer to servo manufacturer. It will also disable 5V power if the servo is the only one in its group of ports.

Tip

Both REV Smart Robot Servos and goBILDA servos will stop holding their position when `setPwmDisable` is called, regardless of if the 5v power is still on or not.

10.4.5 SDK Sensors

The SDK offers several methods of controlling and communicating with sensors.

General Explanation

I2C Explanation

I2C is a two wire serial communication bus that is designed to communicate with multiple devices. The two wires are the SCL, or clock line, and SDA, or data line. The REV hub connector also contains two more pins for 3.3v and GND. The protocol can address multiple devices using addresses, where the master device sends the address it is communicating with, then the data it wants to send. The device it is talking to then responds. This means any number of devices can be connected to one I2C port, so long as no two devices have the same I2C addresses.

I2C has multiple speed specifications, but the REV hub supports both 100khz and 400khz communication standards. The specification also requires the lines to be pulled up, so the REV hub has dual 2.49 kΩ pull up resistors, negating the need for external pull up resistors.

 Tip

Devices generally have their I2C addresses hard coded, meaning they cannot be changed. Check the devices data sheet to make sure addresses will not conflict if connecting multiple I2C devices together.

 Note

I2C is often called slower than other forms of sensors. It is not inherently slower, but due to the way I2C communication works in the SDK it takes multiple commands to read one I2C sensor, which means one I2C read can take 2-3x longer than one digital or analog read.

Analog Input

Reads the input voltage to the expansion hub. These ports are the ONLY sensor ports that are 5v tolerant.

Digital I/O

Digital I/O can be configured to either act as an INPUT, reading whether the port is HIGH or LOW, or an OUTPUT, sending HIGH or LOW signals. The digital input is pulled HIGH to prevent floating.

 Note

Floating occurs when you attempt to read from a digital input pin but the signal is disconnected or not pulled HIGH or to ground. As a result, the pin is considered “floating” and can have inconsistent results. To prevent this, a resistor bridges between the port and 3.3V, so that instead of floating the pin instead reads HIGH when nothing is connected.

 Danger

When wiring limit switches or other digital devices, DO NOT connect the limit switch to 3.3V and the digital port, like most datasheets recommend. Instead, the REV hub expects a connection between GROUND and the digital pin. **Connecting between 3.3V and the digital pin may cause instability or the hub to die.**

10.4.6 Lynx Module

The Lynx Module is an object that represents the Control Hub and Expansion Hub. One Lynx Module object is created per connected hub.

General Explanation

Obtaining Lynx Module Object

A list of connected Lynx Module objects can be obtained using `hardwareMap.getAll(LynxModule.class);`. The Control Hub or the Expansion Hub connected to the phone via USB will have `isMaster()` set to true, and the hub connected over RS-485 will have it set to false.

LED

The REV hub's LED can be set using `setConstant()` or `setPattern()`. `setConstant()` sets a constant color to the LED, and `setPattern()` allows the user to define a pattern of colors for the REV hub to follow.

Power

The total amount of current that the REV hub is pulling can be measured using `getCurrent()`. This is the current pull of the 12V input in the specified units, so it includes everything connected to the hub as well as the hub itself. In addition, the input voltage to the expansion hub can be measured using `getInputVoltage()`, which returns the battery voltage in the specified units.

Bulk Reads

Bulk reads can also be managed through the lynx module object. See [Bulk Reads](#) (page 333) for more information on this.

10.5 Software Glossary

Disconnect

A disconnect (DC) is when, for any reason, the robot is not able to be controlled from the gamepad. This can happen for many reasons - static buildup on the robot, a loose cable, or an error in code.

Generally, most DCs are caused by improper wiring, so wire stress relief is encouraged for all teams ([USB Retention Mount](#)). They can also be caused by WiFi disconnects, or an ESD (electrostatic discharge) shock to the electronics.

Driver Station

The Driver Station (DS) refers to both the software and device that is used by the drive team and connects to the gamepad(s). The device can either be one of the Android phones specified in Game Manual Part 1 or a REV Robotics Driver Hub.

Robot Controller

The Robot Controller (RC) refers to both the software and device that is on the robot that controls it. This can either be a REV Robotics Control Hub or one of the Android phones specified in Game Manual Part 1 connected to an Expansion Hub via a Micro USB cable.